OSTIM TECHNICAL UNIVERSITY FACULTY OF ENGINEERING CAPSTONE PROJECT REPORT

PROJECT TITLE

Acadia: Academic Performance and Stress Monitoring System for Modern Students

Project Team

NOURADDIN ABDURAHMAN ADEN, 210208782
AHMED ADEEB AHMED ABBAD, 210208817
MOHSEN NAEF MOHSEN MOHAMMED QAHTAN, 210208968
SEDDIK MOHEY ABDELHAMID SEDDIK, 210208739
AHMED K.A. ALMATHLOUM, 230208861

Project Advisor:

Dr. Öğr. İlker Yoncacı

Undergraduate Project

Department of Software Engineering
MFBP 402 Graduation Project II

May 2026

APPROVAL

This undergraduate capstone project, entitled "**Acadia: Academic Performance and Stress Monitoring System for Modern Students,**" Prepared by **Nouraddin Abdurahman Aden, Ahmed Adeeb Ahmed Abbad, Mohsen Naef Qahtan, Seddik Mohey Abdelhamid Seddik, Ahmed K.A. Almathloum**. Under the supervision of **Dr. İlker Yoncacı**, has been examined by the committee and accepted as an undergraduate capstone project in terms of scope and quality.

Prof. Dr. Jury Member Name Surname
Supervisor ……

Prof. Dr. Jury Member Name Surname
Member ……

Assoc. Prof. Dr. Jury Member Name Surname
Member ……

Asst. Prof. Dr. Name Surname
Head of Department ……

ACKNOWLEDGEMENTS

The authors would like to express their sincere gratitude to our project advisor, **Dr. İlker Yoncacı**, for his continuous guidance, valuable feedback, and support throughout the development of the **Acadia project**. His recommendations and evaluations helped improve both the technical and academic quality of the project during all development phases.

We would also like to thank the Software Engineering **Department at OSTİM Technical University** for providing the academic environment, resources, and support necessary for completing this graduation project.

Special thanks are extended to all team members for their collaboration, commitment, and contributions during the planning, design, implementation, testing, and documentation stages of the project. The successful completion of Acadia was made possible through effective teamwork, sprint-based development, and continuous communication throughout the project lifecycle.

Finally, we would like to thank the students and participants who provided valuable feedback during testing and evaluation sessions. Their observations and suggestions contributed significantly to improving the usability, reliability, and overall user experience of the system.

ABSTRACT (LOCAL LANGUAGE VERSION)

Bu rapor, Scrum tabanlı bir akademik üretkenlik ve öğrenci iyi oluş yönetim sistemi olan Acadia'nın Minimum Viable Product (MVP) olarak tasarımını ve geliştirilmesini sunmaktadır. Proje, üniversite öğrencilerinin yaşadığı verimsiz ders çalışma planlaması, düzensiz görev takibi, artan stres seviyeleri, zayıf son teslim tarihi yönetimi ve akademik performans ile iş yükü üzerinde sınırlı görünürlük gibi yaygın akademik yönetim problemlerini çözmeye odaklanmaktadır.

Önerilen çözüm; çapraz platform destekli Flutter uygulaması, Supabase tabanlı backend altyapısı, gerçek zamanlı senkronizasyon servisleri, analiz motorları ve birleşik bir yazılım mimarisi içerisinde entegre edilen yapay zeka destekli akademik öneri modüllerinden oluşmaktadır. Sistem, dönem bazlı akademik organizasyonu desteklemekte ve ders yönetimi, görev planlama, çalışma oturumu takibi, ruh hali ve stres kaydı, akademik analizler ve akıllı karar destek araçları sunmaktadır.

Geliştirme süreci; sprint tabanlı uygulama geliştirme, yinelemeli özellik teslimi ve sürekli test ile değerlendirme süreçlerini içeren Agile Scrum metodolojisi takip edilerek yürütülmüştür. Fonksiyonel ve fonksiyonel olmayan gereksinimler; akademik iş akışı analizi, öğrenci üretkenlik problemleri ve sistem kullanılabilirlik hedeflerinden elde edilmiştir. MVP sürümü; güvenli kimlik doğrulama, dönem ve ders yönetimi, Pomodoro ve derin çalışma sistemleri, gerçek zamanlı bildirimler, analiz panelleri, risk puanlama mekanizmaları ve yapay zeka destekli çalışma önerileri gibi temel işlevleri sunmaktadır.

Sistem; çalışma tutarlılığı, iş yükü baskısı, gecikmiş görevler, akademik risk göstergeleri ve öğrenci stres eğilimlerini değerlendirebilen analiz ve karar motorlarını içermektedir. Bu motorlar, kişiselleştirilmiş öneriler ve üretkenlik analizleri oluşturmaktadır. Senaryo tabanlı testler ve gerçek kullanım simülasyonları ile yapılan değerlendirmeler sonucunda; akademik organizasyon, iş yükü farkındalığı, üretkenlik takibi ve genel kullanıcı deneyiminde iyileşmeler gözlemlenmiştir.

Anahtar Kelimeler: Akademik üretkenlik, Scrum, MVP, Flutter, Supabase, Akademik analiz, Öğrenci iyi oluşu, Yazılım mühendisliği

ABSTRACT

This report presents the design and implementation of Acadia, a Scrum-based academic productivity and student wellbeing management system developed as a Minimum Viable Product (MVP). The project focuses on solving common academic management problems experienced by university students, including inefficient study planning, inconsistent task tracking, increasing stress levels, poor deadline management, and limited visibility into academic performance and workload.

The proposed solution consists of a cross-platform Flutter application, a Supabase-powered backend infrastructure, realtime synchronization services, analytics engines, and AI-assisted academic recommendation modules integrated within a unified software architecture. The system supports semester-based academic organization and provides tools for course management, task scheduling, study session tracking, mood and stress logging, academic analytics, and intelligent decision support.

The development process followed Agile Scrum methodology with sprint-based implementation, iterative feature delivery, and continuous testing and evaluation. Functional and non-functional requirements were derived from academic workflow analysis, student productivity challenges, and system usability objectives. The MVP delivers essential functionality including secure authentication, semester and course management, Pomodoro and deep-work study systems, realtime notifications, analytics dashboards, risk scoring mechanisms, and AI-powered study recommendations.

The system incorporates analytics and decision engines capable of evaluating study consistency, workload pressure, overdue tasks, academic risk indicators, and student stress trends to generate personalized recommendations and productivity insights. Evaluation through scenario-based testing and real usage simulations demonstrated improvements in academic organization, workload awareness, productivity tracking, and overall user experience.

Keywords: Academic productivity, Scrum, MVP, Flutter, Supabase, Academic analytics, Student wellbeing, Software engineering

TABLE OF CONTENTS

Sayfa

TABLE OF CONTENTS iv
LIST OF TABLES vi
LIST OF FIGURES vii
LIST OF SYMBOLS AND ABBREVIATIONS viii
1. Introduction 1
1.1. Problem Statement 1
1.2. Project Objectives 1
1.3. Project Scope 1
1.4. Report Organization 1
2. General Concepts 2
2.1. Domain Definition 2
2.2. Technologies Used 2
2.3. Fundamental Concepts 2
3. Related Work 3
3.1. Existing Systems 3
3.2. Academic Studies 3
3.3. Comparative Analysis 3
4. Software Process Model 4
4.1. Selected Software Development Model 4
4.2. Definition of Scrum Roles 4
4.3. Scrum Process 4
4.4. Scrum Implementation in the Project 4
5. Requirements Analysis 5
5.1. Stakeholders 5
5.2. User Requirements 5
5.2.1. Functional Requirements 5
5.2.2. Non-Functional Requirements 5
5.3. System Specifications 5
6. System Modeling 6
6.1. Context Diagram 6
6.2. Use Case Diagram 6
6.3. Use Case Descriptions 6
6.4. Sequence Diagrams 6
7. System Architecture and Design 7

7.1. System Architecture 7
7.2. Structural Models 7
7.2.1. Class Diagram 7
7.2.2. Inheritance (Generalization) 7
7.2.3. Aggregation / Composition 7
7.3. Behavioral Models 7
7.3.1. Event-Driven Model 8
7.3.2. State Diagram 8
8. System Implementation 9
8.1. Development Environment 9
8.2. System Components 9
8.3. Code Structure 9
9. MVP Development 10
9.1. MVP Definition 10
9.2. Implemented Features 10
9.3. System Interfaces 10
9.4. System Demonstration 10
10. Testing and Evaluation 11
10.1. Testing Strategy 11
10.2. Test Scenarios 11
10.3. Test Results 11
11. Results and Discussion 12
12. Conclusion and Future Work 13
12.1. Conclusion 13
12.2. Future Work 13
13. Appendices 16
13.1. Appendix A: Source Code 16
13.2. Appendix B: Sprint Backlog 16
13.3. Appendix C: User Guide 16

LIST OF TABLES

Sayfa

Table 2.1. Technologies Used in Acadia Development

Table 3.1. Comparative Analysis of Existing Systems and Acadia

Table 4.1. Scrum Roles and Responsibilities

Table 4.2. Scrum Sprints Implemented in Acadia Development

Table 5.1. Stakeholders of the Acadia System

Table 5.2. Functional Requirements of Acadia

Table 5.3. Non-Functional Requirements of Acadia

Table 5.4. Main System Technologies and Specifications

Table 10.1. Representative Test Scenarios

LIST OF FIGURES

Sayfa

Figure 4.1. Agile Scrum Development Lifecycle

Figure 4.2. Scrum Team Structure and Role Distribution used in Acadia

Figure 4.3. Scrum Workflow Used in Acadia Project

Figure 4.4. Notion-Based Scrum Sprint Management System Used in Acadia Development

Figure 6.1. Context Diagram of the Acadia Platform

Figure 6.2. Use Case Diagram of the Acadia System

Figure 6.3. User Authentication Sequence Diagram

Figure 6.4. Study Session Workflow Sequence Diagram

Figure 6.5. Notification Delivery Sequence Diagram

Figure 7.1. Layered Acadia Architecture

Figure 7.2. Acadia Class Diagram

Figure 7.3. Inheritance Model

Figure 7.4. Aggregation and Composition Relationships

Figure 7.5. Event-Driven Notification Model

Figure 7.6. Acadia State Diagram

Figure 10.1. Testing and Quality Assurance Activities in Acadia

Figure 10.2.1. Analytics Dashboard After Realtime Data Synchronization

Figure 10.2.2. Realtime Deadline Notification Received on Mobile Device

Figure 11.1. Realtime Smart Notifications Delivered by the Acadia Notification Engine

Figure A.1. Supabase Authentication Integration

Figure A.2. Database Schema and Supabase Tables

Figure A.3. Semester and Course Management System

Figure A.4. Study Session Persistence System

Figure A.5. Analytics Dashboard Implementation

Figure A.6. Smart Notification Delivery System

Figure A.7. AI Assistant Integration Workflow

Figure B.1. Acadia Sprint Management Workspace

Figure B.2. Sprint Completion and Progress Tracking

Figure B.3. GitHub Commit History

Figure C.1. Language Selection Interface

Figure C.2. User Login Interface

Figure C.3. User Registration Interface

Figure C.4. OTP Verification Workflow

Figure C.5. Password Recovery Interface

Figure C.6. Academic Profile Setup Interface

Figure C.7. Semester Configuration Screen

Figure C.8. Course Setup Interface

Figure C.9. Academic Task Dashboard

Figure C.10. Task Creation Interface

Figure C.11. Deadline Management Interface

Figure C.12. Pomodoro Study Session Interface

Figure C.13. Active Focus Session Workflow

Figure C.14. Study History and Productivity Analytics

Figure C.15. Academic Analytics Dashboard

Figure C.16. Academic Risk Monitoring Interface

Figure C.17. Burnout and Productivity Visualization

Figure C.18. Smart Notification Interface

Figure C.19. Realtime Push Notification Example

Figure C.20. AI Assistant Interface

Figure C.21. AI Recommendation System

LIST OF SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CI/CD	Continuous Integration / Continuous Deployment
ERD	Entity Relationship Diagram
ETA	Estimated Time of Arrival
FCM	Firebase Cloud Messaging
KPI	Key Performance Indicator
LLM	Large Language Model
MFA	Multi-Factor Authentication
MVP	Minimum Viable Product
OTP	One-Time Password
REST	Representational State Transfer
RLS	Row Level Security
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
UX	User Experience
UUID	Universally Unique Identifier
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
GPA	Grade Point Average

ECTS	European Credit Transfer System
DBMS	Database Management System
JWT	JSON Web Token
MVC	Model View Controller
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
SaaS	Software as a Service
NLP	Natural Language Processing
QA	Quality Assurance
ORM	Object Relational Mapping
BaaS	Backend as a Service
PWA	Progressive Web Application
CPU	Central Processing Unit
RAM	Random Access Memory
UML	Unified Modeling Language

1. Introduction

1.1. Problem Statement

University students face increasing difficulties in managing academic responsibilities across multiple courses, assignments, exams, and study schedules. Many students rely on disconnected tools such as paper notes, calendar applications, messaging platforms, and simple task managers that fail to provide a centralized academic management experience. As a result, students often struggle with poor organization, missed deadlines, inconsistent study habits, increasing stress levels, and weak visibility into their academic performance.

Existing productivity applications usually focus on isolated features such as scheduling or note-taking and lack integrated academic analytics, semester-based organization, realtime study monitoring, and personalized academic recommendations. Students rarely receive intelligent feedback regarding overdue tasks, workload pressure, declining study consistency, or burnout risk. This absence of centralized monitoring and decision support negatively affects productivity, wellbeing, and academic performance.

There is therefore a need for a unified academic productivity system that combines task management, study tracking, mood and stress monitoring, analytics, realtime notifications, and AI-assisted recommendations within a single integrated platform specifically designed for university students.

1.2. Project Objectives

The primary objective of this project is to design and implement a practical, scalable, and intelligent academic productivity management system that helps university students organize academic activities, monitor performance, and improve study consistency through realtime analytics and integrated productivity tools.

The specific objectives of the project are:

i. Develop a cross-platform mobile application for academic productivity management.

ii. Provide semester-based organization for courses, tasks, deadlines, exams, and study sessions.

iii. Implement task and deadline management features to improve academic scheduling and workload tracking.

iv. Develop Pomodoro and deep-work study session systems for focus and time management.

v. Provide mood and stress tracking capabilities for monitoring student wellbeing.

vi. Implement realtime notification and reminder systems for academic activities and risk alerts.

vii. Design analytics dashboards capable of visualizing productivity trends, workload pressure, study consistency, and stress indicators.

viii. Develop an academic risk engine that evaluates overdue tasks, study deficits, workload intensity, and stress levels.

ix. Integrate AI-assisted recommendation mechanisms capable of generating personalized study guidance and productivity suggestions.

x. Apply Agile Scrum methodology throughout the software development lifecycle using sprint-based implementation and iterative feature delivery.

1.3. Project Scope

The project focuses on the development of a university-oriented academic productivity and student wellbeing management system implemented as a Minimum Viable Product (MVP). The system supports core academic workflows required for managing semesters, courses, tasks, deadlines, study sessions, mood tracking, analytics, and academic notifications within a centralized mobile application.

The scope of the project includes:

- User authentication and onboarding
- Semester and course management
- Task and deadline tracking
- Study session management
- Pomodoro and deep-work timers
- Mood and stress logging
- Academic analytics dashboards
- Realtime notification systems
- Academic risk evaluation
- AI-assisted recommendation features
- Realtime synchronization using cloud backend services
- Cross-platform mobile support

The system is designed primarily for university students and academic productivity use cases within a single-user academic environment.

The project scope excludes:

- Integration with university ERP systems
- Online examination systems
- Financial or payment modules
- Wearable device integration
- Full learning management system functionality
- Career planning systems
- Multi-institution administrative infrastructures

These features are considered potential future extensions of the project.

1.4. Report Organization

This report is organized into multiple sections covering the complete software engineering lifecycle of the Acadia project.

Section 2 introduces the academic productivity domain, enabling technologies, and fundamental concepts used throughout the system.

Section 3 reviews existing academic productivity systems, related academic studies, and comparative analyses relevant to the project domain.

Section 4 explains the selected software development methodology, Scrum roles, sprint processes, and the implementation of Agile practices within the project.

Section 5 presents the requirements analysis process, including stakeholder identification, functional requirements, non-functional requirements, and system specifications.

Section 6 describes the system modeling process through context diagrams, use case diagrams, sequence diagrams, and detailed use case descriptions.

Section 7 explains the overall system architecture, database design, structural models, behavioral models, and backend infrastructure of the Acadia platform.

Section 8 presents the implementation environment, development technologies, code structure, and software components developed during the project lifecycle.

Section 9 focuses on the MVP implementation, including completed features, user interfaces, sprint outcomes, and system demonstrations.

Section 10 discusses the testing strategy, testing scenarios, evaluation methods, and system validation results.

Section 11 presents the final results, findings, and discussion of the developed system.

Section 12 concludes the report and introduces possible future improvements and expansion opportunities for the Acadia platform.

2. General Concepts

2.1. Domain Definition

Academic productivity and student wellbeing management systems are software platforms designed to help students organize academic responsibilities, monitor study performance, manage workloads, and improve productivity through centralized digital tools. These systems combine academic planning, task organization, analytics, and behavioral monitoring to support students throughout their educational process.

Modern university students manage multiple academic activities simultaneously, including assignments, exams, projects, study sessions, and semester planning. As academic workloads increase, students often experience stress, poor time management, inconsistent study habits, and difficulty maintaining productivity. Academic productivity systems aim to address these challenges by providing structured tools for organization, monitoring, and decision support.

The domain of this project combines several interconnected areas:

- Academic management systems
- Productivity and task management systems
- Study tracking systems
- Student wellbeing monitoring
- Realtime analytics systems
- Mobile application development
- Cloud-based backend systems
- AI-assisted recommendation systems

Acadia specifically focuses on the intersection between academic productivity and student wellbeing. Unlike traditional task management applications, the system integrates academic workflows, semester-based organization, study analytics, mood and stress tracking, realtime notifications, and intelligent recommendation mechanisms within a unified platform.

The system supports core academic activities such as:

- Course management
- Semester organization
- Assignment and deadline tracking
- Study session monitoring
- Pomodoro-based productivity tracking
- Academic analytics visualization

- Mood and stress monitoring
- Academic risk evaluation
- Personalized study recommendations

The project domain also includes realtime mobile systems and cloud synchronization technologies that allow users to access updated academic information across devices while maintaining secure and scalable data management.

2.2. Technologies Used

The Acadia project was developed using modern cross-platform mobile development technologies, cloud backend services, realtime synchronization systems, and analytics-driven software engineering practices. The selected technologies were chosen based on scalability, performance, maintainability, realtime capabilities, and development efficiency.

Table 2.1 presents the main technologies used in the development process.
Table 2.1. Technologies Used in Acadia Development

Technology	**Purpose**
Flutter	Cross-platform mobile application development
Dart	Primary programming language
Supabase	Backend-as-a-Service infrastructure
PostgreSQL	Relational database management system
Riverpod	State management solution
GoRouter	Navigation and routing management
Firebase Cloud Messaging (FCM)	Push notification delivery
Hive	Local offline storage
SharedPreferences	Lightweight local persistence
Supabase Edge Functions	Serverless backend functions
TypeScript	Backend function development
fl_chart	Analytics and chart visualization
GitHub	Version control and collaboration
Notion	Sprint management and project planning
Android Studio	Main development environment
Xcode	iOS build and deployment environment
Figma	UI/UX design and prototyping

The frontend application was developed using Flutter because of its ability to support Android, iOS, Web, Windows, macOS, and Linux using a single codebase. Flutter also provides high-performance rendering, responsive UI development, and strong support for custom animations and modern interface design.

The backend infrastructure was implemented using Supabase, which provides authentication, PostgreSQL database management, realtime synchronization, storage services, and serverless edge functions. Supabase simplified backend development while supporting scalable cloud-based architecture and realtime updates.

Riverpod was selected as the state management solution because it provides reactive dependency management, modular provider architecture, and maintainable application state organization. The project also uses GoRouter for route management and navigation handling. Firebase Cloud Messaging was integrated for realtime push notifications related to deadlines, academic alerts, study reminders, and risk notifications.

The analytics and AI-related backend services were implemented using Supabase Edge Functions and TypeScript, allowing server-side processing for recommendation systems, analytics calculations, and intelligent decision support mechanisms.

The project development process also included GitHub for source control management and Notion for Agile Scrum sprint planning, task assignment, and progress tracking.

2.3. Fundamental Concepts

The Acadia system is built upon several important software engineering, productivity, analytics, and mobile computing concepts. Understanding these concepts is necessary to understand the architecture and functionality of the developed system.

Academic Productivity

Academic productivity refers to a student's ability to effectively manage academic tasks, study schedules, deadlines, and learning activities while maintaining consistent performance and efficient time management. Productivity in this context involves organization, focus, workload balance, and study consistency.

Student Wellbeing Monitoring

Student wellbeing monitoring focuses on tracking emotional and behavioral factors such as stress, mood, motivation, and workload pressure that may affect academic performance. Acadia integrates mood and stress tracking mechanisms to help students identify negative academic patterns and maintain healthier study habits.

Minimum Viable Product (MVP)

A Minimum Viable Product (MVP) is a software development approach where the system is built with essential core features necessary to validate functionality and usability before implementing advanced extensions. Acadia was developed as an MVP that delivers the main academic productivity features while leaving advanced functionality for future development phases.

Agile Scrum Methodology

Scrum is an Agile software development framework based on iterative development, sprint planning, continuous feedback, and incremental feature delivery. The Acadia project was developed using sprint-based Scrum practices managed through Notion project boards and structured team collaboration.

Realtime Synchronization

Realtime synchronization allows data changes to appear instantly across devices and application interfaces without requiring manual refresh actions. Acadia uses Supabase realtime services to synchronize tasks, notifications, analytics, study sessions, and academic data dynamically.

State Management

State management refers to controlling and synchronizing application data across multiple screens and components. The project uses Riverpod for reactive state management, provider dependency handling, and modular architecture organization.

Pomodoro Technique

The Pomodoro Technique is a time management method that divides work into focused study intervals followed by short breaks. Acadia integrates Pomodoro and deep-work study session systems to improve concentration and study consistency.

Academic Risk Evaluation

Academic risk evaluation refers to analyzing student activity patterns such as overdue tasks, low study consistency, workload intensity, and stress levels to estimate potential academic difficulties. The system includes a risk engine that generates risk scores and productivity alerts based on behavioral indicators.

Artificial Intelligence Assisted Recommendations

AI-assisted recommendation systems generate personalized suggestions based on user behavior, analytics, and activity history. Acadia uses intelligent recommendation mechanisms to provide study suggestions, workload guidance, and academic productivity insights.

Cloud-Based Backend Architecture

Cloud backend architecture refers to hosting backend services, databases, authentication systems, and APIs on remote infrastructure accessible through the internet. Acadia uses Supabase cloud services for backend operations, authentication, database management, and realtime communication.

Cross-Platform Mobile Development

Cross-platform development enables applications to run on multiple operating systems using a shared codebase. Flutter allows Acadia to support Android, iOS, Web, Windows, macOS, and Linux while maintaining consistent user experience and functionality across platforms.

3. Related Work

3.1. Existing Systems

Several academic productivity and task management systems currently exist to help students organize schedules, manage assignments, and improve study habits. Popular applications such as Notion, Todoist, Google Calendar, MyStudyLife, and Microsoft To Do provide productivity-related features including task management, reminders, scheduling, and note organization. While these systems are widely used, many of them are designed as general productivity platforms rather than specialized academic management systems.

Notion is one of the most flexible productivity applications and allows users to create custom dashboards, databases, and academic planners. Students commonly use Notion for note-taking, assignment tracking, and personal organization. Despite its flexibility, the system requires extensive manual customization and does not provide built-in academic analytics, stress monitoring, academic risk evaluation, or intelligent study recommendations.

Todoist and Microsoft To Do focus mainly on task and reminder management. These systems help users organize deadlines and activities but lack semester-based academic structures, study tracking systems, and productivity analytics specifically designed for university students.

Google Calendar is widely used for scheduling academic activities and reminders. Although it supports event organization and notifications, it does not provide integrated workload analysis, study session tracking, or academic performance monitoring.

MyStudyLife is one of the few platforms designed specifically for students. It supports class schedules, assignments, and exam tracking. However, the system provides limited analytics capabilities and does not integrate mood tracking, realtime productivity evaluation, AI-assisted recommendations, or academic risk analysis.

Forest and Focus To-Do are applications designed to improve concentration using Pomodoro-based productivity techniques. These applications encourage focused study sessions but primarily concentrate on timer functionality rather than complete academic management workflows.

Most existing systems focus on isolated productivity features and do not provide a unified environment that combines academic organization, wellbeing monitoring, analytics, realtime synchronization, intelligent notifications, and AI-supported decision-making within a single integrated platform.

The **Acadia** system was developed to address these limitations by integrating semester management, task tracking, study analytics, mood monitoring, academic risk evaluation, realtime notifications, and AI-assisted recommendations into one centralized academic productivity ecosystem.

3.2. Academic Studies

Academic productivity, student wellbeing, and educational technology systems have been widely studied in software engineering, human-computer interaction, and educational research. Many studies emphasize the relationship between productivity management, stress monitoring, and academic performance among university students.

Research related to time management and academic performance indicates that students with structured study schedules and organized task management systems generally achieve better academic outcomes and lower stress levels. Several studies highlight that poor time management is strongly associated with academic burnout, procrastination, and reduced learning efficiency.

Studies on mobile learning and productivity applications demonstrate that mobile platforms improve accessibility and encourage students to engage more frequently with academic planning tools. Mobile productivity systems also support realtime notifications, personalized reminders, and continuous monitoring of student activities.

Research on the Pomodoro Technique and focus-based productivity systems suggests that structured study intervals combined with scheduled breaks can improve concentration, reduce mental fatigue, and increase study efficiency. Many modern productivity applications integrate Pomodoro-based timers because of their positive impact on focus and consistency. Recent studies involving student wellbeing monitoring show that emotional factors such as stress, motivation, and workload pressure significantly affect academic performance. Systems capable of tracking mood and behavioral patterns can help identify early signs of burnout and productivity decline.

Artificial Intelligence and analytics-driven educational systems have also become an important research area. Intelligent recommendation systems and predictive analytics are increasingly used to analyze study behaviors, estimate academic risks, and generate personalized learning support. These systems aim to provide proactive guidance rather than reactive academic intervention.

Cloud computing and realtime synchronization technologies are also widely discussed in educational software research because they improve scalability, accessibility, and multi-device support. Modern educational applications increasingly rely on cloud-based infrastructures to support realtime data updates, secure authentication, and centralized information management.

The **Acadia** project combines concepts from these research areas by integrating academic productivity management, realtime analytics, mood and stress tracking, study monitoring, and AI-assisted recommendation mechanisms within a unified mobile platform designed specifically for university students.

3.3. Comparative Analysis

Table 3.1 presents a comparison between Acadia and several existing productivity and academic management systems.

Table 3.1. Comparative Analysis of Existing Systems and Acadia

Features	**Notion**	**Todoist**	**MyStudyLife**	**Forest**	**Acadia**
Task Management	Yes	Yes	Yes	Limited	Yes
Semester Management	Limited	No	Yes	No	Yes
Course Management	Limited	No	Yes	No	Yes
Study Session Tracking	Limited	No	No	Yes	Yes
Pomodoro Timer	No	Limited	No	Yes	Yes
Mood & Stress Tracking	No	No	No	No	Yes
Academic Analytics	No	No	Limited	No	Yes
Academic Risk Evaluation	No	No	No	No	Yes
AI Recommendations	No	No	No	No	Yes
Realtime Synchronization	Yes	Yes	Limited	Yes	Yes
Notification System	Yes	Yes	Yes	Limited	Yes
Cross-Platform Support	Yes	Yes	Yes	Yes	Yes
Unified Academic Ecosystem	No	No	Partial	No	Yes

The comparison shows that existing systems typically focus on individual productivity features rather than integrated academic ecosystems. Some applications specialize in task management, while others focus on focus timers or scheduling. Very few systems combine academic organization, wellbeing monitoring, analytics, and intelligent recommendations within a single unified platform.

Acadia differentiates itself by integrating:

- Semester-based academic management
- Realtime productivity analytics
- Mood and stress monitoring
- Academic risk evaluation
- AI-assisted recommendations
- Study consistency tracking
- Intelligent notification systems
- Centralized academic dashboards

Unlike traditional productivity applications, Acadia is specifically designed for university academic workflows and student wellbeing support. The system combines productivity management with behavioral analytics and realtime decision support mechanisms, providing a more comprehensive academic management experience.

4. Software Process Model

4.1. Selected Software Development Model

The Acadia project was developed using the Agile Scrum software development model. Scrum was selected because the project required iterative feature development, continuous testing, rapid feedback integration, and flexible sprint-based planning throughout the development lifecycle.

The project involved multiple interconnected modules such as authentication, semester management, study tracking, analytics, notifications, mood monitoring, AI recommendations, and realtime synchronization. Developing all these components using a traditional sequential model would have reduced flexibility and slowed adaptation to new requirements and implementation changes. Scrum allowed the development team to divide the project into manageable sprint cycles and incrementally deliver features while continuously improving the system architecture and user experience.

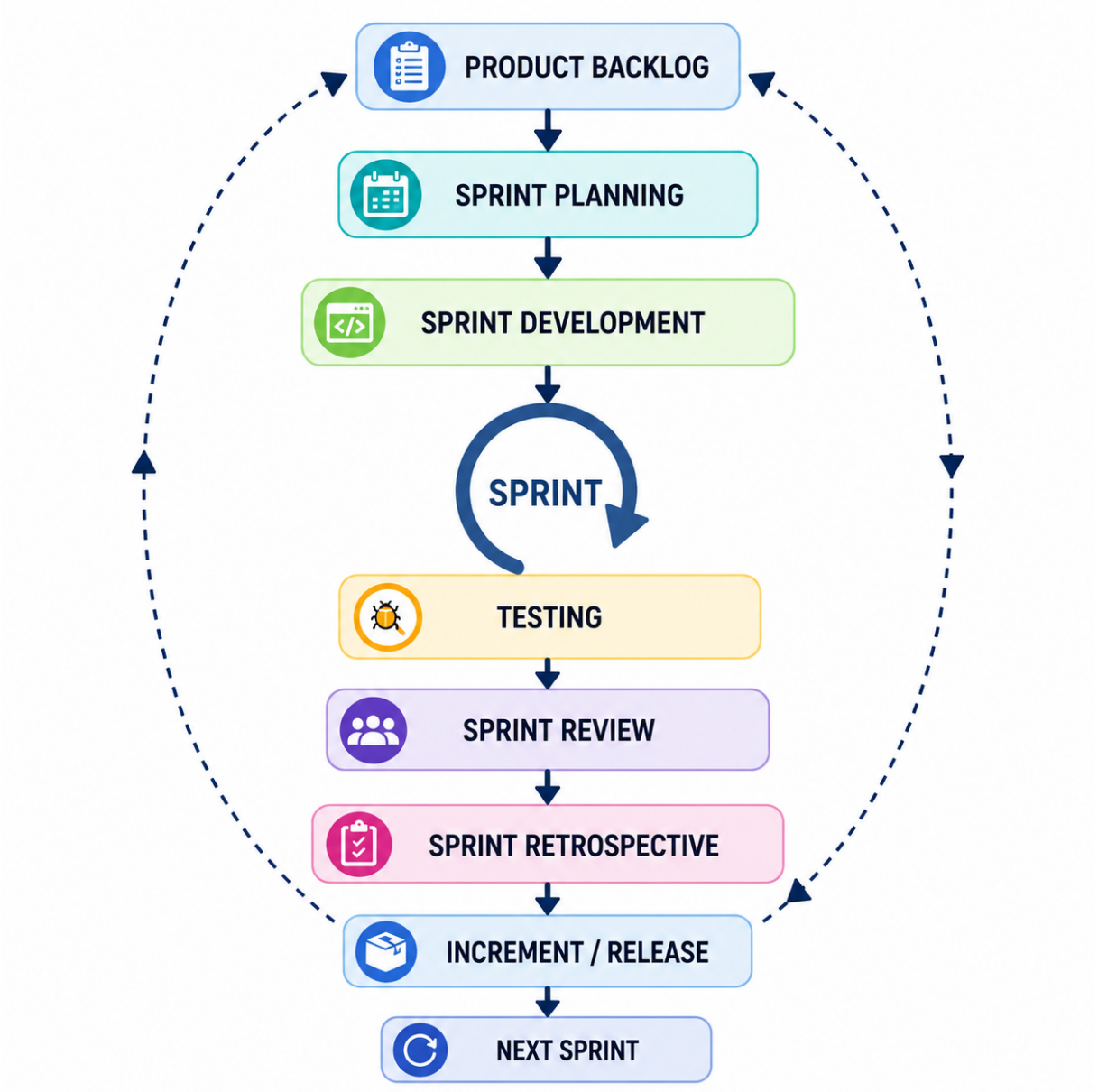
The Agile Scrum model also supported:

- Incremental feature delivery
- Continuous testing and evaluation
- Frequent feedback and improvements
- Better team collaboration
- Sprint-based task organization
- Prioritized backlog management
- Faster problem identification
- Flexible adaptation to changing requirements

Each sprint focused on a specific group of system functionalities such as authentication, semester management, study sessions, analytics, notifications, or AI integration. Features were planned, implemented, tested, reviewed, and refined during iterative development cycles.

The selected development model was especially suitable for Acadia because the project combined frontend development, backend integration, database design, analytics systems, and mobile UI implementation within a rapidly evolving academic productivity platform.

Figure 4.1. Agile Scrum Development Lifecycle



4.2. Definition of Scrum Roles

The Acadia project team followed Scrum role distribution principles to organize project responsibilities, sprint execution, and development activities throughout the software development lifecycle.

The project roles were divided into the following categories:

Table 4.1. Scrum Roles and Responsibilities

Role	Responsibilities
Project Advisor	Academic supervision, technical guidance, project evaluation
Scrum Team	Feature development, testing, documentation, sprint implementation
Product Owner Responsibilities	Feature prioritization, requirement analysis, backlog organization
Scrum Master Responsibilities	Sprint coordination, task tracking, workflow management

The project advisor**, Dr. İlker Yoncacı**, supervised the overall development process, reviewed technical progress, and provided academic guidance throughout the project lifecycle.

The student development team was responsible for:

- frontend implementation
- backend integration
- database management
- analytics systems
- UI/UX development
- testing
- sprint planning
- documentation

Product Owner responsibilities were distributed among the team members during sprint planning sessions. These responsibilities included defining system requirements, prioritizing features, managing the product backlog, and ensuring that implemented features aligned with project objectives.

Scrum Master responsibilities included:

- organizing sprint workflows
- managing sprint boards
- monitoring progress
- tracking completed tasks
- coordinating team communication
- ensuring sprint goals were achieved on schedule

The project used collaborative planning and shared sprint management through the Notion project management workspace.

Figure 4.2. Scrum Team Structure and Role Distribution used in Acadia.

4.3. Scrum Process

The Acadia project followed an iterative Scrum workflow consisting of sprint planning, feature implementation, testing, review, and continuous improvement phases. The project development lifecycle was organized into multiple sprint cycles, where each sprint focused on implementing a specific set of system functionalities.

The Scrum process used in the project included the following stages:

Product Backlog Preparation

The development process began with requirement analysis and backlog preparation. Features were identified based on academic productivity challenges, user needs, and project **objectives. The product backlog included tasks related to:**

- authentication
- onboarding
- semester management
- task tracking
- study sessions
- analytics
- notifications
- AI recommendations
- testing
- deployment

Sprint Planning

Before each sprint, the development team selected a group of prioritized backlog items and divided them into implementation tasks. Sprint goals, responsibilities, and timelines were clearly defined.

Sprint Development

During the implementation phase, team members developed frontend interfaces, backend services, database structures, providers, analytics systems, and realtime synchronization mechanisms related to the sprint objectives.

Testing and Validation

Each sprint included testing activities to validate functionality, identify bugs, and improve system stability. Features were tested continuously during development to ensure compatibility and correct system behavior.

Sprint Review

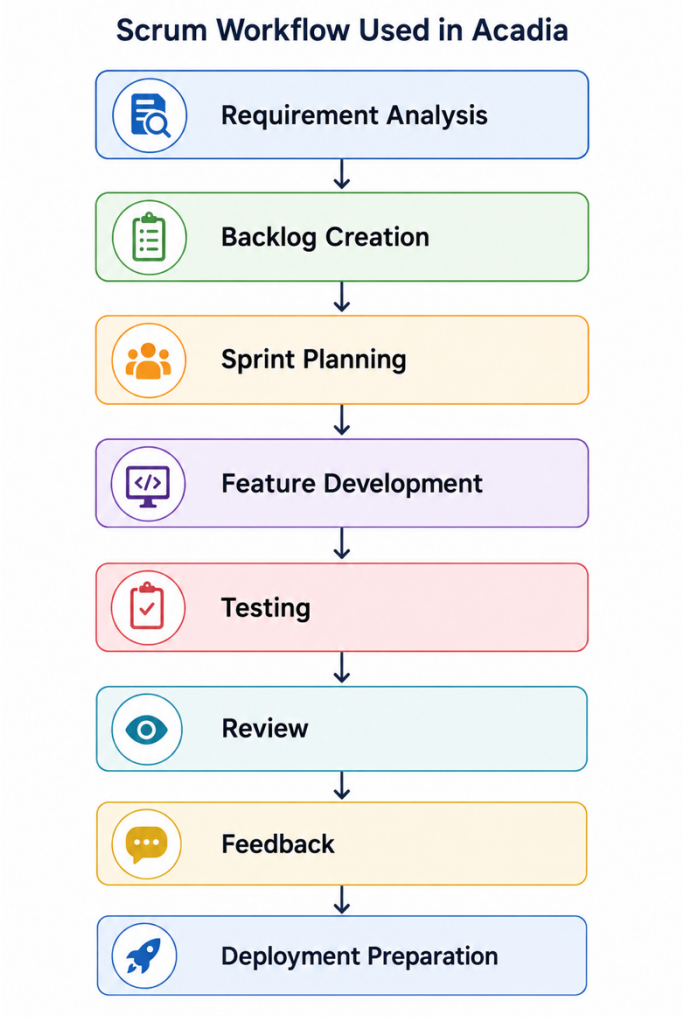
At the end of each sprint, completed features were reviewed and evaluated. The development team analyzed sprint outcomes, discussed implementation issues, and identified possible improvements for future sprints.

Sprint Retrospective

Retrospective discussions helped evaluate development efficiency, communication, workflow organization, and technical challenges. These evaluations improved sprint planning and task management during later development phases.

The Scrum process enabled the project to evolve incrementally while maintaining flexibility, continuous feedback integration, and stable feature delivery throughout the development lifecycle.

Figure 4.3. Scrum Workflow Used in Acadia project.



4.4. Scrum Implementation in the Project

The Scrum methodology was implemented throughout the entire Acadia development lifecycle using sprint-based planning, iterative development, and continuous integration practices. The development process was managed through the Notion project management workspace, where tasks, sprint goals, implementation progress, and deliverables were monitored systematically.

The project was divided into structured weekly sprints. Each sprint focused on a specific subsystem or functional area of the application. Sprint planning sessions were conducted to define implementation goals, assign responsibilities, estimate workloads, and organize feature priorities.

The Scrum workflow supported:

- defining sprint objectives
- assigning responsibilities to team members
- monitoring implementation progress
- managing feature dependencies
- reviewing completed deliverables
- validating implemented functionality
- improving collaboration between frontend, backend, and AI components

The project management structure included:

- sprint backlogs
- feature prioritization
- implementation tracking
- task assignments
- sprint reviews
- progress monitoring
- completion status indicators
- testing and bug tracking

The following Scrum sprints were implemented during the development of Acadia:

Table 4.2. Scrum Sprints Implemented in Acadia Development

Sprint	**Main Objectives**
Sprint 1	Project setup and architecture initialization
Sprint 2	Authentication and onboarding system
Sprint 3	Semester and course management
Sprint 4	Task and deadline management
Sprint 5	Study session and Pomodoro systems
Sprint 6	Mood and stress tracking
Sprint 7	Analytics dashboards and visualization
Sprint 8	Academic risk and decision engines
Sprint 9	AI assistant and prediction systems
Sprint 10	Smart notification and background systems
Sprint 11	Realtime synchronization and optimization
Sprint 12	App stabilization and final integration
Sprint 13	Testing, QA, and bug fixing
Sprint 14	Documentation, poster, and presentation preparation

Each sprint produced incremental and testable deliverables that were integrated into the existing system architecture. The sprint-based structure allowed the team to continuously improve application stability, user experience, analytics accuracy, backend performance, and AI-assisted features.

The Notion-based Scrum management system was used to organize:

• sprint boards
• task timelines
• assigned responsibilities
• feature tracking
• implementation priorities
• bug tracking
• testing workflows
• progress reports

• documentation evidence

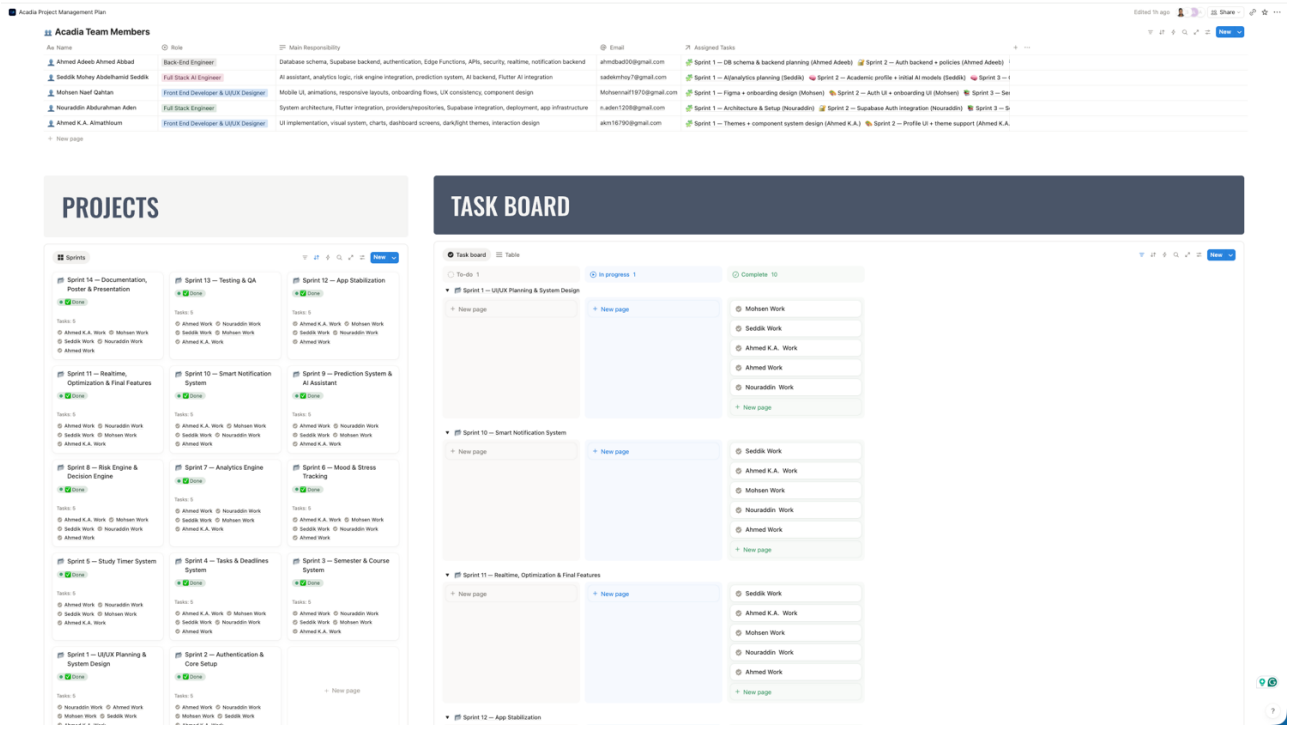
GitHub was also used together with the Scrum workflow to manage version control, feature branches, integration processes, and collaborative development activities. The project followed a structured branch strategy including development, feature, and hotfix branches.

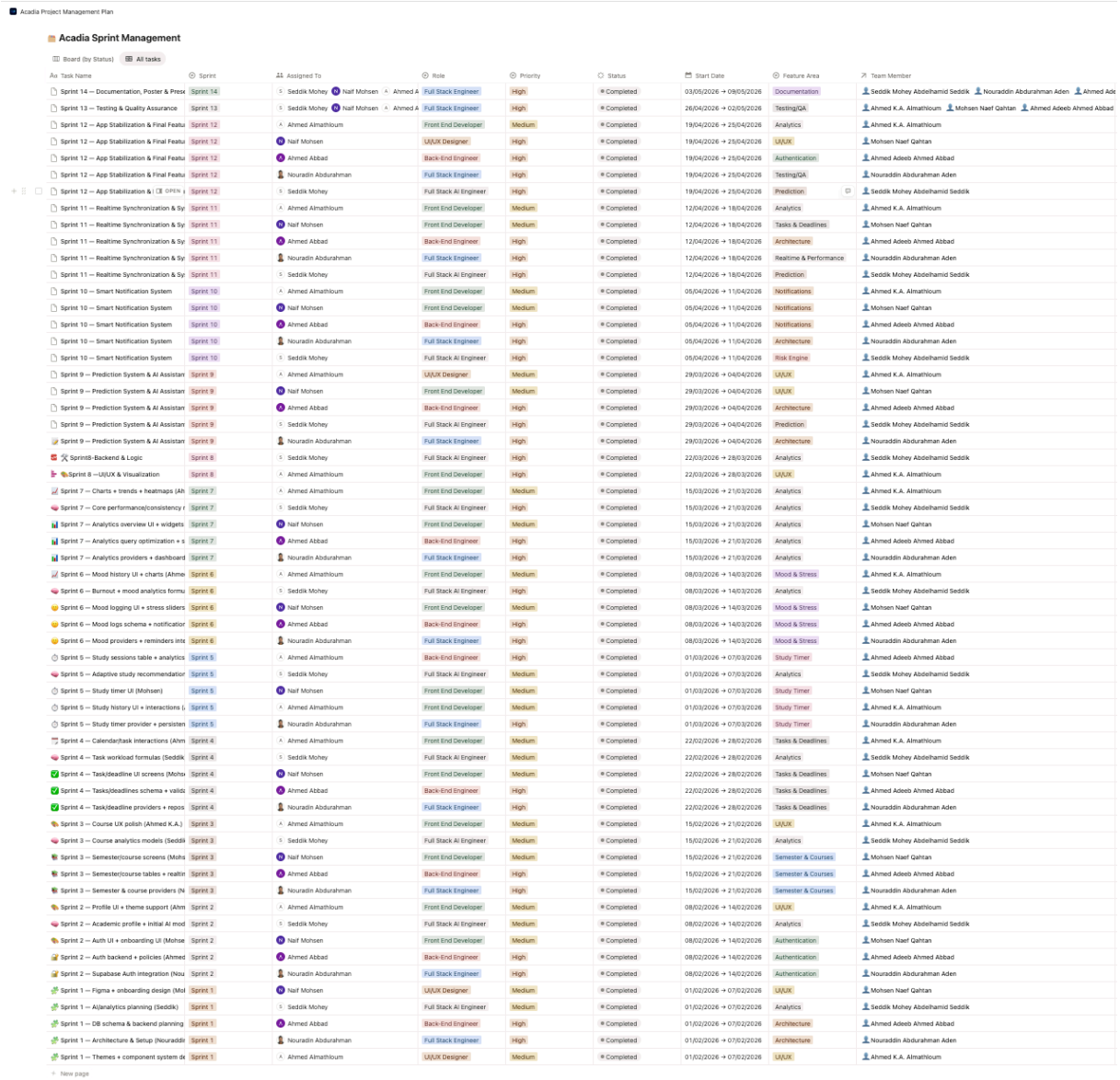
The Scrum implementation also supported:

• iterative feature delivery
• continuous backend integration
• realtime testing and validation
• UI/UX refinement
• analytics verification
• AI integration improvements
• database optimization
• deployment preparation

The sprint-based workflow significantly improved project organization and enabled the team to manage a complex multi-module academic monitoring platform more efficiently. It also provided clear evidence of project management activities, development progress, testing procedures, and team collaboration throughout the graduation project process.

Figure 4.4. Notion-Based Scrum Sprint Management System Used in Acadia Development





Acadia Project Management Plan

Acadia Sprint Management

Board (by Status) | All tasks

Task Name	Sprint	Assigned To	Role	Priority	Status	Start Date	Feature Area	Team Member
Sprint 14 — Documentation, Poster & Prese	Sprint 14	Seddik Mohey, Naif Mohsen, Ahmed A	Full Stack Engineer	High	Completed	03/05/2026 → 09/05/2026	Documentation	Seddik Mohey Abdelhamid Seddik, Nouraddin Abdurahman Aden, Ahmed Ade
Sprint 13 — Testing & Quality Assurance	Sprint 13	Seddik Mohey, Naif Mohsen, Ahmed A	Full Stack Engineer	High	Completed	26/04/2026 → 02/05/2026	Testing/QA	Ahmed K.A. Almathloum, Mohsen Naef Qahtan, Ahmed Adeeb Ahmed Abbad
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Ahmed Almathloum	Front End Developer	Medium	Completed	19/04/2026 → 25/04/2026	Analytics	Ahmed K.A. Almathloum
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Naif Mohsen	UI/UX Designer	High	Completed	19/04/2026 → 25/04/2026	UI/UX	Mohsen Naef Qahtan
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Ahmed Abbad	Back-End Engineer	High	Completed	19/04/2026 → 25/04/2026	Authentication	Ahmed Adeeb Ahmed Abbad
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Nouradin Abdurahman	Full Stack Engineer	High	Completed	19/04/2026 → 25/04/2026	Testing/QA	Nouraddin Abdurahman Aden
Sprint 12 — App Stabilization & | OPEN	Sprint 12	Seddik Mohey	Full Stack AI Engineer	High	Completed	19/04/2026 → 25/04/2026	Prediction	Seddik Mohey Abdelhamid Seddik
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Ahmed Almathloum	Front End Developer	Medium	Completed	12/04/2026 → 18/04/2026	Analytics	Ahmed K.A. Almathloum
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Naif Mohsen	Front End Developer	Medium	Completed	12/04/2026 → 18/04/2026	Tasks & Deadlines	Mohsen Naef Qahtan
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Ahmed Abbad	Back-End Engineer	High	Completed	12/04/2026 → 18/04/2026	Architecture	Ahmed Adeeb Ahmed Abbad
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Nouradin Abdurahman	Full Stack Engineer	High	Completed	12/04/2026 → 18/04/2026	Realtime & Performance	Nouraddin Abdurahman Aden
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Seddik Mohey	Full Stack AI Engineer	High	Completed	12/04/2026 → 18/04/2026	Prediction	Seddik Mohey Abdelhamid Seddik
Sprint 10 — Smart Notification System	Sprint 10	Ahmed Almathloum	Front End Developer	Medium	Completed	05/04/2026 → 11/04/2026	Notifications	Ahmed K.A. Almathloum
Sprint 10 — Smart Notification System	Sprint 10	Naif Mohsen	Front End Developer	Medium	Completed	05/04/2026 → 11/04/2026	Notifications	Mohsen Naef Qahtan
Sprint 10 — Smart Notification System	Sprint 10	Ahmed Abbad	Back-End Engineer	High	Completed	05/04/2026 → 11/04/2026	Notifications	Ahmed Adeeb Ahmed Abbad
Sprint 10 — Smart Notification System	Sprint 10	Nouradin Abdurahman	Full Stack Engineer	High	Completed	05/04/2026 → 11/04/2026	Architecture	Nouraddin Abdurahman Aden
Sprint 10 — Smart Notification System	Sprint 10	Seddik Mohey	Full Stack AI Engineer	High	Completed	05/04/2026 → 11/04/2026	Risk Engine	Seddik Mohey Abdelhamid Seddik
Sprint 9 — Prediction System & AI Assistant	Sprint 9	Ahmed Almathloum	UI/UX Designer	Medium	Completed	29/03/2026 → 04/04/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 9 — Prediction System & AI Assistant	Sprint 9	Naif Mohsen	Front End Developer	Medium	Completed	29/03/2026 → 04/04/2026	UI/UX	Mohsen Naef Qahtan
Sprint 9 — Prediction System & AI Assistant	Sprint 9	Ahmed Abbad	Back-End Engineer	High	Completed	29/03/2026 → 04/04/2026	Architecture	Ahmed Adeeb Ahmed Abbad
Sprint 9 — Prediction System & AI Assistant	Sprint 9	Seddik Mohey	Full Stack AI Engineer	High	Completed	29/03/2026 → 04/04/2026	Prediction	Seddik Mohey Abdelhamid Seddik
Sprint 9 — Prediction System & AI Assistant	Sprint 9	Nouradin Abdurahman	Full Stack Engineer	High	Completed	29/03/2026 → 04/04/2026	Architecture	Nouraddin Abdurahman Aden
🛠️ Sprint8–Backend & Logic	Sprint 8	Seddik Mohey	Full Stack AI Engineer	High	Completed	22/03/2026 → 28/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
🎨Sprint 8 —UI/UX & Visualization	Sprint 8	Ahmed Almathloum	Front End Developer	High	Completed	22/03/2026 → 28/03/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 7 — Charts + trends + heatmaps (Ah	Sprint 7	Ahmed Almathloum	Front End Developer	Medium	Completed	15/03/2026 → 21/03/2026	Analytics	Ahmed K.A. Almathloum
Sprint 7 — Core performance/consistency (	Sprint 7	Seddik Mohey	Full Stack AI Engineer	High	Completed	15/03/2026 → 21/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 7 — Analytics overview UI + widgets	Sprint 7	Naif Mohsen	Front End Developer	Medium	Completed	15/03/2026 → 21/03/2026	Analytics	Mohsen Naef Qahtan
Sprint 7 — Analytics query optimization + s	Sprint 7	Ahmed Abbad	Back-End Engineer	High	Completed	15/03/2026 → 21/03/2026	Analytics	Ahmed Adeeb Ahmed Abbad
Sprint 7 — Analytics providers + dashboard	Sprint 7	Nouradin Abdurahman	Full Stack Engineer	High	Completed	15/03/2026 → 21/03/2026	Analytics	Nouraddin Abdurahman Aden
Sprint 6 — Mood history UI + charts (Ahme	Sprint 6	Ahmed Almathloum	Front End Developer	Medium	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Ahmed K.A. Almathloum
Sprint 6 — Burnout + mood analytics formu	Sprint 6	Seddik Mohey	Full Stack AI Engineer	High	Completed	08/03/2026 → 14/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 6 — Mood logging UI + stress sliders	Sprint 6	Naif Mohsen	Front End Developer	Medium	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Mohsen Naef Qahtan
Sprint 6 — Mood logs schema + notification	Sprint 6	Ahmed Abbad	Back-End Engineer	High	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Ahmed Adeeb Ahmed Abbad
Sprint 6 — Mood providers + reminders inte	Sprint 6	Nouradin Abdurahman	Full Stack Engineer	High	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Nouraddin Abdurahman Aden
Sprint 5 — Study sessions table + analytics	Sprint 5	Ahmed Almathloum	Back-End Engineer	High	Completed	01/03/2026 → 07/03/2026	Study Timer	Ahmed Adeeb Ahmed Abbad
Sprint 5 — Adaptive study recommendation	Sprint 5	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	01/03/2026 → 07/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 5 — Study timer UI (Mohsen)	Sprint 5	Naif Mohsen	Front End Developer	Medium	Completed	01/03/2026 → 07/03/2026	Study Timer	Mohsen Naef Qahtan
Sprint 5 — Study history UI + interactions (	Sprint 5	Ahmed Almathloum	Front End Developer	Medium	Completed	01/03/2026 → 07/03/2026	Study Timer	Ahmed K.A. Almathloum
Sprint 5 — Study timer provider + persisten	Sprint 5	Nouradin Abdurahman	Full Stack Engineer	High	Completed	01/03/2026 → 07/03/2026	Study Timer	Nouraddin Abdurahman Aden
Sprint 4 — Calendar/task interactions (Ahm	Sprint 4	Ahmed Almathloum	Front End Developer	Medium	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Ahmed K.A. Almathloum
Sprint 4 — Task workload formulas (Seddik	Sprint 4	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	22/02/2026 → 28/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
✅ Sprint 4 — Task/deadline UI screens (Mohs	Sprint 4	Naif Mohsen	Front End Developer	Medium	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Mohsen Naef Qahtan
✅ Sprint 4 — Tasks/deadlines schema + validz	Sprint 4	Ahmed Abbad	Back-End Engineer	High	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Ahmed Adeeb Ahmed Abbad
✅ Sprint 4 — Task/deadline providers + repos	Sprint 4	Nouradin Abdurahman	Full Stack Engineer	High	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Nouraddin Abdurahman Aden
Sprint 3 — Course UX polish (Ahmed K.A.)	Sprint 3	Ahmed Almathloum	Front End Developer	Medium	Completed	15/02/2026 → 21/02/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 3 — Course analytics models (Seddik	Sprint 3	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	15/02/2026 → 21/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 3 — Semester/course screens (Mohs	Sprint 3	Naif Mohsen	Front End Developer	Medium	Completed	15/02/2026 → 21/02/2026	Semester & Courses	Mohsen Naef Qahtan
Sprint 3 — Semester/course tables + realtin	Sprint 3	Ahmed Abbad	Back-End Engineer	High	Completed	15/02/2026 → 21/02/2026	Semester & Courses	Ahmed Adeeb Ahmed Abbad
Sprint 3 — Semester & course providers (N	Sprint 3	Nouradin Abdurahman	Full Stack Engineer	High	Completed	15/02/2026 → 21/02/2026	Semester & Courses	Nouraddin Abdurahman Aden
Sprint 2 — Profile UI + theme support (Ahm	Sprint 2	Ahmed Almathloum	Front End Developer	Medium	Completed	08/02/2026 → 14/02/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 2 — Academic profile + initial AI mod	Sprint 2	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	08/02/2026 → 14/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 2 — Auth UI + onboarding UI (Mohse	Sprint 2	Naif Mohsen	Front End Developer	Medium	Completed	08/02/2026 → 14/02/2026	Authentication	Mohsen Naef Qahtan
Sprint 2 — Auth backend + policies (Ahmed	Sprint 2	Ahmed Abbad	Back-End Engineer	High	Completed	08/02/2026 → 14/02/2026	Authentication	Ahmed Adeeb Ahmed Abbad
Sprint 2 — Supabase Auth integration (Nou	Sprint 2	Nouradin Abdurahman	Full Stack Engineer	High	Completed	08/02/2026 → 14/02/2026	Authentication	Nouraddin Abdurahman Aden
Sprint 1 — Figma + onboarding design (Mol	Sprint 1	Naif Mohsen	UI/UX Designer	Medium	Completed	01/02/2026 → 07/02/2026	UI/UX	Mohsen Naef Qahtan
Sprint 1 — AI/analytics planning (Seddik)	Sprint 1	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	01/02/2026 → 07/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 1 — DB schema & backend planning	Sprint 1	Ahmed Abbad	Back-End Engineer	High	Completed	01/02/2026 → 07/02/2026	Architecture	Ahmed Adeeb Ahmed Abbad
Sprint 1 — Architecture & Setup (Nouraddi	Sprint 1	Nouradin Abdurahman	Full Stack Engineer	High	Completed	01/02/2026 → 07/02/2026	Architecture	Nouraddin Abdurahman Aden
Sprint 1 — Themes + component system de	Sprint 1	Ahmed Almathloum	UI/UX Designer	Medium	Completed	01/02/2026 → 07/02/2026	UI/UX	Ahmed K.A. Almathloum

New page

5. Requirements Analysis

5.1. Stakeholders

The Acadia project involves several stakeholders who directly or indirectly interact with the system throughout its operation and development lifecycle. These stakeholders were identified during the requirements analysis and planning phases of the project.

The primary stakeholders of the system are university students who use the application to manage academic activities, monitor study performance, organize tasks, and receive productivity insights. Students interact with nearly all modules of the system, including semester management, course tracking, study sessions, notifications, analytics dashboards, and AI-assisted recommendations.

Secondary stakeholders include project developers, academic supervisors, and future administrators who may manage, evaluate, maintain, or extend the platform in future development phases.

Table 5.1 presents the main stakeholders of the Acadia system.
Table 5.1. Stakeholders of the Acadia System

Stakeholder	**Role in the System**
Students	Manage academic activities, track tasks, monitor productivity, receive analytics and recommendations
Development Team	Design, implement, test, and maintain the system
Project Advisor	Supervise project progress and provide academic guidance
Future System Administrators	Manage backend services, deployment, and maintenance
Researchers and Evaluators	Analyze system performance and academic productivity results

Students represent the most important stakeholder group because the system was specifically designed to improve academic organization, productivity tracking, and student wellbeing management.

The development team acted as both system developers and internal testers during iterative sprint-based implementation and evaluation processes.

5.2. User Requirements

User requirements were identified through academic workflow analysis, productivity challenges commonly experienced by university students, and iterative Scrum planning sessions conducted throughout the project lifecycle. The requirements were continuously refined during sprint implementation, testing, and feature evaluation phases.
The system requirements focused on creating a centralized academic productivity platform capable of supporting task organization, study monitoring, semester management, analytics visualization, stress tracking, and intelligent academic recommendations.

The identified requirements were divided into functional requirements and non-functional requirements.

5.2.1. Functional Requirements

Functional requirements describe the core functionalities and services that the Acadia system must provide to users.

Table 5.2. Functional Requirements of Acadia

Requirement ID	Functional Requirement
FR-1	Users shall create and manage academic semesters.
FR-2	Users shall create, edit, and delete courses within semesters.
FR-3	Users shall create and manage academic tasks and deadlines.
FR-4	Users shall receive notifications for upcoming deadlines and overdue tasks.
FR-5	Users shall start and monitor Pomodoro and deep-work study sessions.
FR-6	Users shall track completed study sessions and study duration statistics.
FR-7	Users shall log mood, stress, and energy levels.
FR-8	Users shall view analytics dashboards related to productivity and academic performance.
FR-9	The system shall calculate academic risk indicators based on workload and study behavior.
FR-10	The system shall generate personalized study recommendations using analytics and AI logic.
FR-11	Users shall securely authenticate using login and signup functionality.
FR-12	The system shall synchronize academic data in realtime using cloud backend services.
FR-13	Users shall manage exams, assignments, and academic deadlines.
FR-14	Users shall access AI-assisted academic guidance through the integrated assistant module.

FR-15	The system shall support dark mode and responsive mobile user interfaces.

These functional requirements formed the foundation of the sprint backlog and guided the implementation of system modules throughout the development process.

5.2.2. Non-Functional Requirements

Non-functional requirements define the quality attributes, operational constraints, and performance expectations of the Acadia system.

Table 5.3. Non-Functional Requirements of Acadia

Requirement ID	Non-Functional Requirement
NFR-1	The system shall provide responsive user interactions with minimal interface delay under normal usage conditions.
NFR-2	The platform shall support realtime synchronization of academic data across application screens.
NFR-3	The system shall maintain secure authentication and protected user data access using Row Level Security (RLS).
NFR-4	The application shall support cross-platform execution on Android and iOS devices.
NFR-5	The system shall provide scalable backend infrastructure using cloud-based services.
NFR-6	The platform shall maintain reliable notification delivery for deadlines, reminders, and risk alerts.
NFR-7	The system shall support modular and maintainable software architecture.
NFR-8	The application shall provide consistent UI/UX behavior across different screen sizes and devices.
NFR-9	The backend services shall support concurrent user operations and realtime event processing.
NFR-10	The system shall provide data persistence and recovery mechanisms for user sessions and study activities.
NFR-11	The application shall support secure API communication using authenticated requests.
NFR-12	The system shall maintain high usability and accessibility for university students.

The non-functional requirements significantly influenced architectural decisions, backend infrastructure selection, state management organization, and cloud integration strategies used throughout the project.

5.3. System Specifications

The Acadia system was implemented using a layered client-server architecture supported by cloud backend services, realtime synchronization, and modular application components. The platform consists of a Flutter-based mobile frontend, Supabase backend infrastructure, PostgreSQL database services, realtime communication channels, analytics engines, and AI-assisted recommendation modules.

The frontend application was developed using Flutter and Dart to support cross-platform deployment across Android and iOS devices. State management was implemented using Riverpod, while application navigation and route handling were managed using GoRouter.

The backend infrastructure uses Supabase services, including:

- Supabase Authentication
- PostgreSQL Database
- Realtime Services
- Edge Functions
- Cloud Storage

The database architecture includes multiple interconnected entities responsible for academic organization and productivity tracking.

Main database entities include:

- users
- student_profiles
- semesters
- courses
- tasks
- academic_deadlines
- study_sessions
- mood_logs
- notifications
- exams
- analytics
- course_results

The backend also supports realtime synchronization between frontend interfaces and cloud services, allowing instant updates for notifications, analytics, tasks, and study sessions.

Table 5.4. Main System Technologies and Specifications

Component	**Specification**
Frontend Framework	Flutter
Programming Language	Dart
Backend Platform	Supabase
Database System	PostgreSQL
State Management	Riverpod
Notification Service	Firebase Cloud Messaging
Authentication	Supabase Auth
Backend Functions	Supabase Edge Functions
Local Storage	Hive, SharedPreferences
Version Control	GitHub
Project Management	Notion
Supported Platforms	Android, iOS, Web, Windows, macOS, Linux

The system architecture was designed to support modular scalability, maintainability, realtime processing, and future expansion of AI-assisted academic intelligence features.

6. System Modeling

6.1. Context Diagram

The Acadia system operates as a centralized academic productivity and student wellbeing management platform that interacts with multiple external actors and cloud-based services. The system was designed using a client-server architecture where users interact with the mobile application while backend services handle authentication, realtime synchronization, analytics processing, notifications, and database management.

The primary external actor of the system is the student user. Students interact with the application to manage semesters, courses, tasks, deadlines, study sessions, mood logs, analytics dashboards, and AI-assisted recommendations.

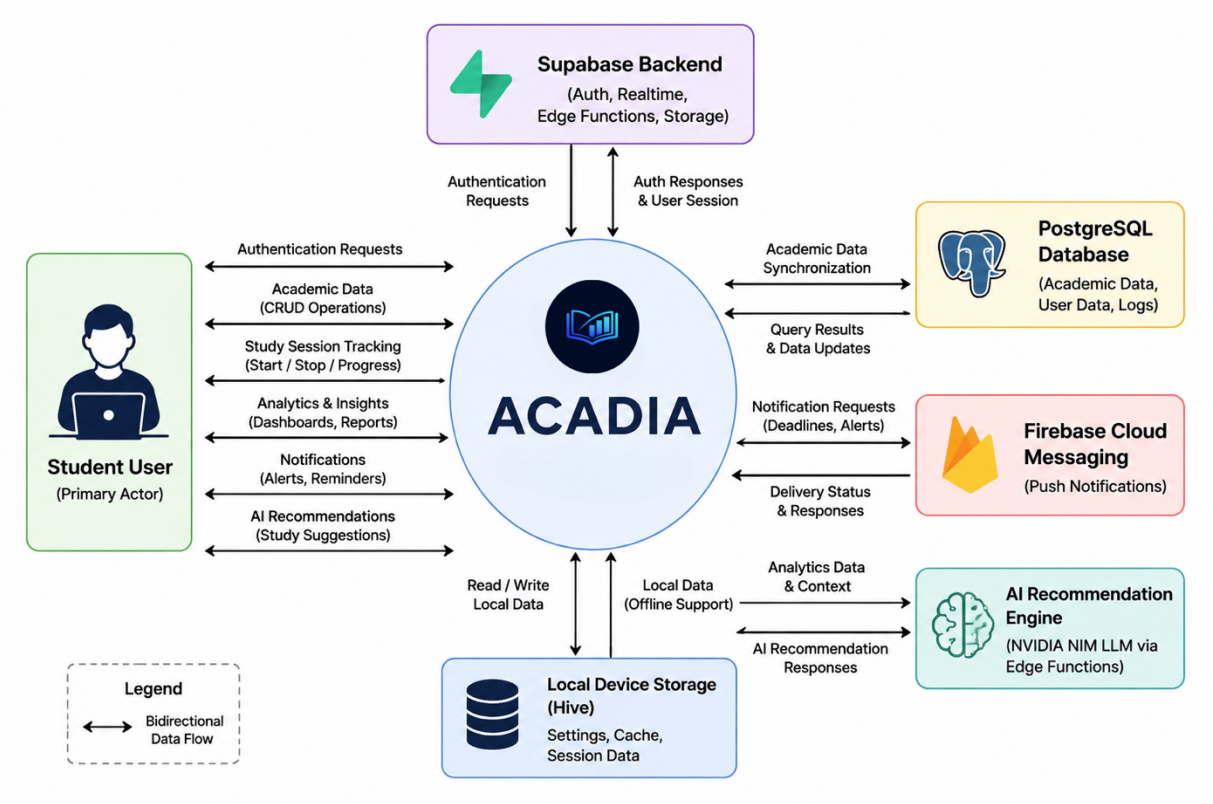
Several external backend services are also integrated into the platform to support realtime functionality and cloud infrastructure operations.

The main external systems integrated into Acadia include:

- Supabase Authentication Services
- PostgreSQL Database Services
- Firebase Cloud Messaging (FCM)
- Supabase Realtime Services
- AI Recommendation Services
- Local Device Storage

The context model illustrates how the student interacts with the frontend application while the backend infrastructure manages data processing, synchronization, notifications, analytics generation, and AI-assisted recommendation workflows.

Figure 6.1. Context Diagram of the Acadia Platform



6.2. Use Case Diagram

The Acadia system contains multiple interconnected use cases that support academic management, productivity tracking, analytics generation, and intelligent recommendation workflows. The use case model was designed to represent the primary interactions between students and the system.

The main actor in the system is the student user, who interacts with the application through various academic and productivity-related operations.

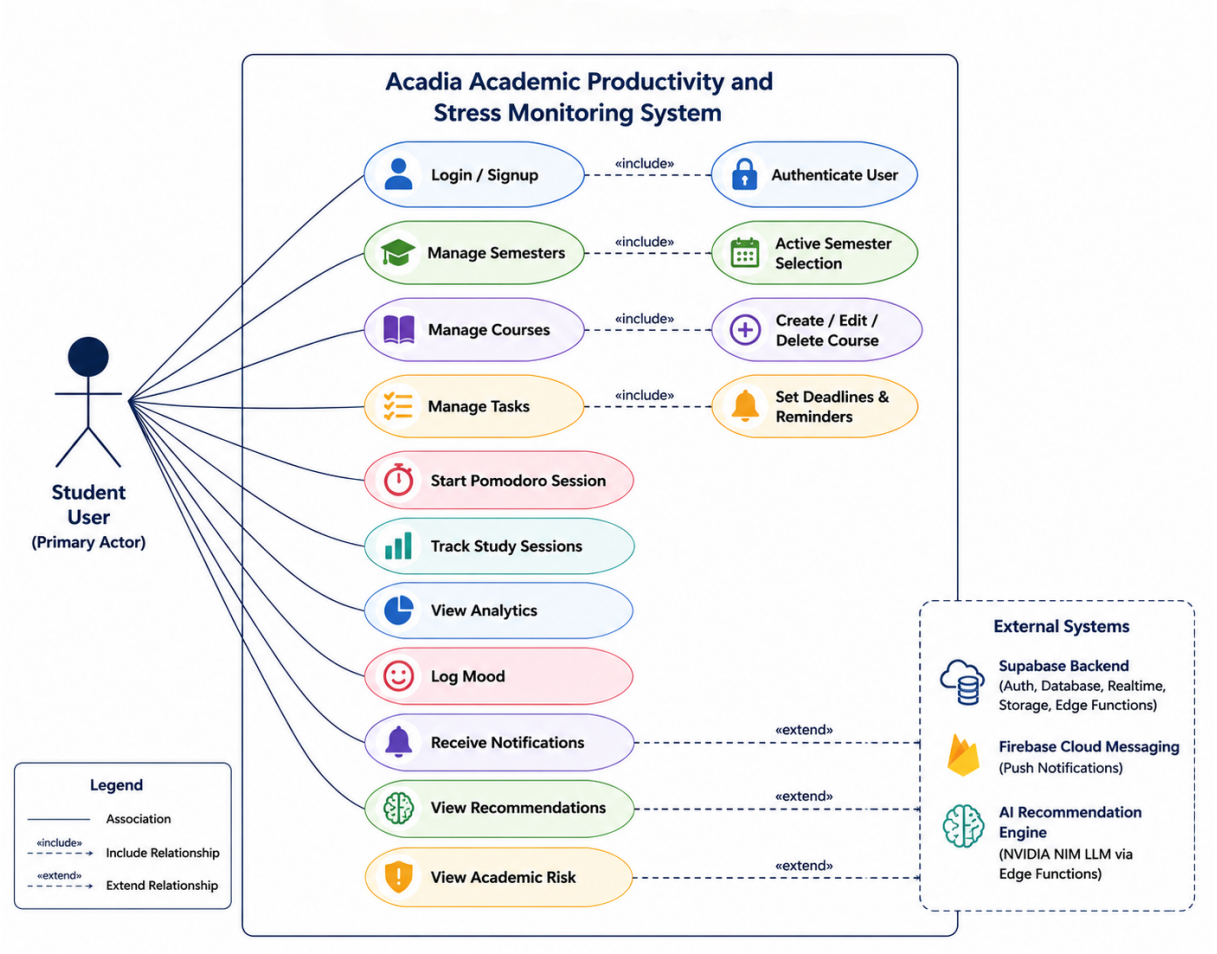
The major use cases implemented in Acadia include:

- Authenticate User
- Complete Academic Onboarding
- Manage Semesters
- Manage Courses
- Create and Manage Tasks
- Manage Academic Deadlines
- Start Study Session
- Track Study Progress
- Log Mood and Stress

- View Analytics Dashboard
- Receive Notifications
- View Academic Risk Status
- Receive AI Recommendations
- Manage Profile Settings
- Review Productivity Insights

These use cases collectively support the core objectives of the system related to academic organization, productivity management, stress monitoring, and intelligent decision support.

Figure 6.2. Use Case Diagram of the Acadia System



6.3. Use Case Descriptions

Each use case in Acadia defines a specific interaction between the student and the system. The use case descriptions include actors, preconditions, main workflows, alternative flows, and postconditions.
The following use cases represent some of the main system operations implemented in the project.

Use Case 1: Authenticate User

Field	Description
Use Case Name	Authenticate User
Primary Actor	Student User
Preconditions	User has internet access and registered account
Main Flow	User enters email and password → System validates credentials → User session is created → Dashboard is loaded
Alternative Flow	Invalid credentials → Error message displayed
Postconditions	Authenticated user gains access to system features

Use Case 2: Manage Academic Tasks

Field	Description
Use Case Name	Manage Academic Tasks
Primary Actor	Student User
Preconditions	User is authenticated and active semester exists
Main Flow	User creates task → Selects course and deadline → Task stored in database → Dashboard updated
Alternative Flow	Missing required fields → Validation warning displayed
Postconditions	Task appears in task lists and analytics system

Use Case 3: Start Study Session

Field	Description
Use Case Name	Start Study Session
Primary Actor	Student User
Preconditions	User is authenticated and course exists
Main Flow	User selects session type → Timer starts → Session tracked in realtime → Session data stored after completion
Alternative Flow	User exits session early → Session saved as incomplete
Postconditions	Study session analytics updated

Use Case 4: Log Mood and Stress

Field	Description

Use Case Name	Log Mood and Stress
Primary Actor	Student User
Preconditions	User is authenticated
Main Flow	User selects mood and stress values → Optional notes added → Data stored in backend
Alternative Flow	User skips optional note fields
Postconditions	Mood analytics and stress indicators updated

Use Case 5: View Analytics Dashboard

Field	Description
Use Case Name	View Analytics Dashboard
Primary Actor	Student User
Preconditions	User has existing academic data
Main Flow	User opens analytics screen → System processes metrics → Charts and insights displayed
Alternative Flow	No available data → Empty analytics state shown
Postconditions	User receives productivity and performance insights

Use Case 6: Receive AI Recommendations

Field	Description
Use Case Name	Receive AI Recommendations
Primary Actor	Student User
Preconditions	Analytics and academic data available
Main Flow	System evaluates workload and study metrics → Recommendation engine generates suggestions → Results displayed
Alternative Flow	Recommendation service unavailable → Default suggestions displayed
Postconditions	User receives personalized productivity recommendations

6.4. Sequence Diagrams

Sequence diagrams were used to model the interaction flow between the frontend application, providers, backend services, and database infrastructure during major system operations.

The sequence models illustrate how requests move through the layered architecture of

Acadia, including:

- Flutter UI Layer
- Riverpod Providers
- Repository Layer
- Service Layer
- Supabase Backend
- PostgreSQL Database

The following major sequence flows were modeled during system analysis and implementation:

- User Authentication Flow
- Task Creation Flow
- Study Session Tracking Flow
- Notification Delivery Flow
- Analytics Processing Flow
- AI Recommendation Flow

One of the most important sequence flows in the system is the study session workflow.
The study session sequence begins when the student selects a session type and starts the study timer. The request is processed through Riverpod providers and backend services. Session progress is synchronized in realtime, and after completion, analytics and productivity metrics are updated automatically.

Another important sequence flow involves analytics processing. The frontend requests productivity metrics from the analytics provider, which retrieves academic data from repositories and backend services before calculating and displaying productivity indicators and charts.

Figure 6.3. User Authentication Sequence Diagram

User → Login Screen → Auth Provider → Supabase Auth → Database → Dashboard

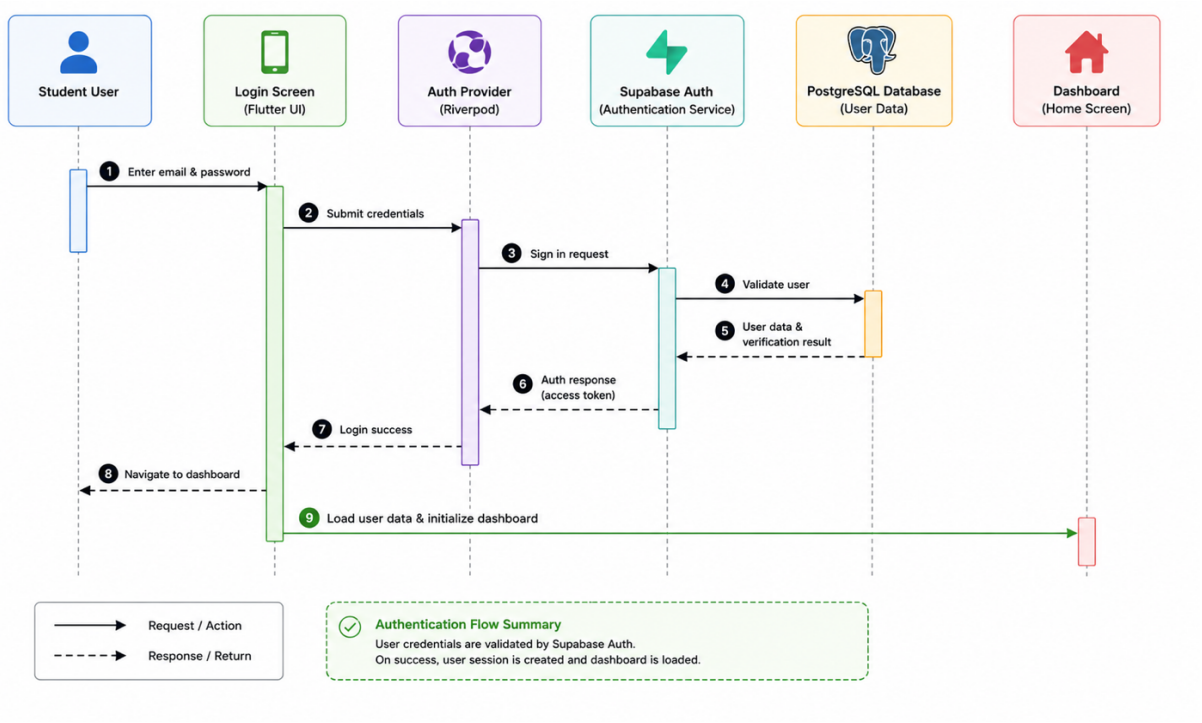


Figure 6.4. study session workflow sequence diagram.

- User → Study Screen → Session Provider → Backend Service → Database → Analytics Engine → Dashboard Update

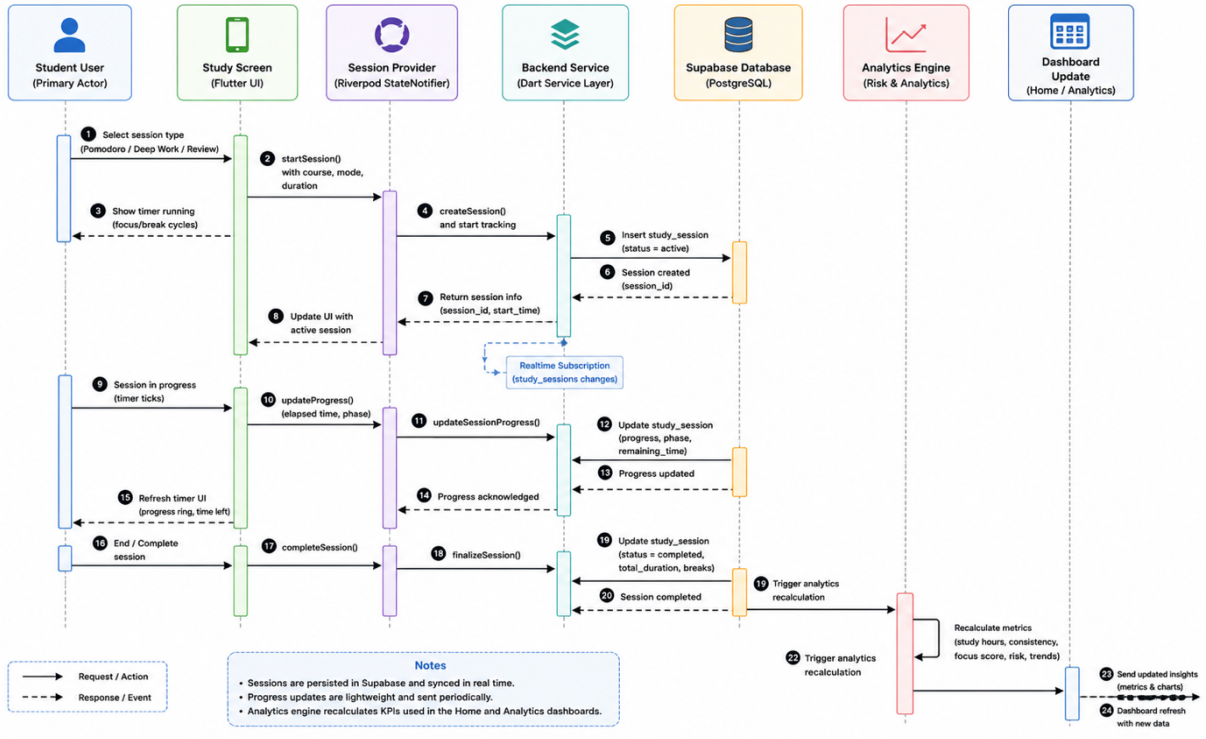
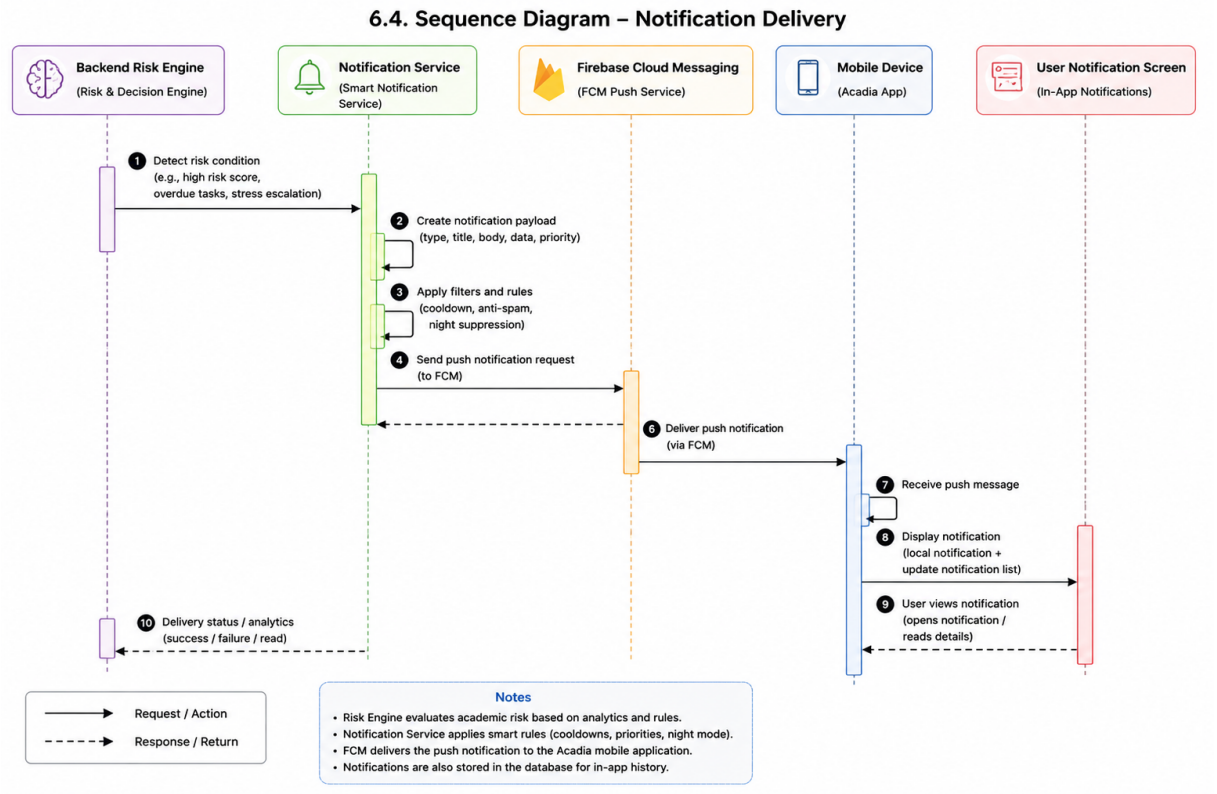


Figure 6.5 Notification delivery sequence diagram.

- Backend Risk Engine → Notification Service → Firebase Cloud Messaging → Mobile Device → User Notification Screen



These sequence diagrams help visualize the interaction logic and layered architecture used throughout the Acadia platform.

7. System Architecture and Design

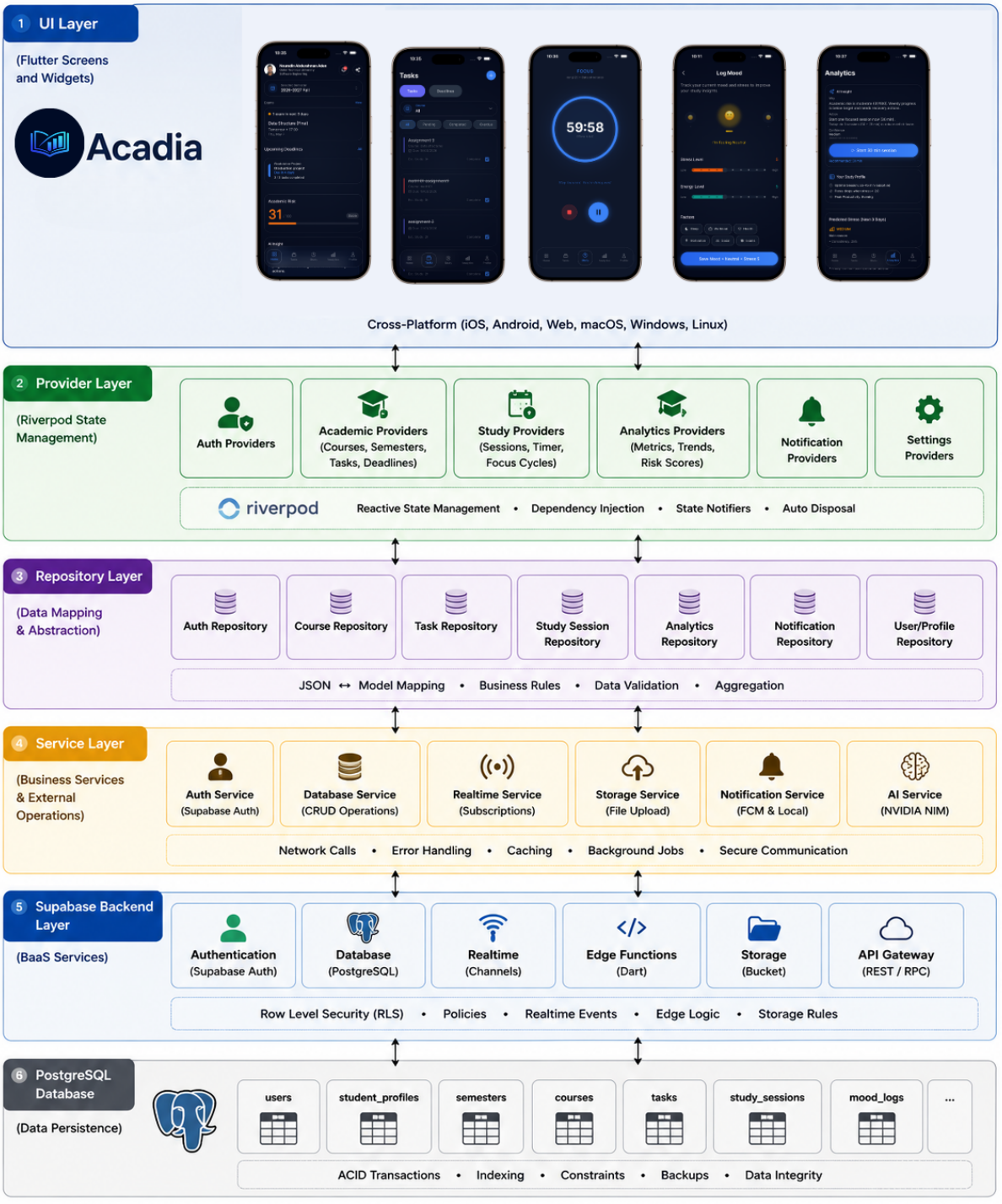
7.1. System Architecture

Acadia adopts a layered client–server architecture designed to support cross-platform academic monitoring, analytics processing, and real-time synchronization. The system consists of Flutter-based mobile and desktop clients connected to a Supabase backend through repositories, providers, and service layers.

The frontend layer handles user interaction, visualization, and state rendering. Riverpod providers manage business orchestration and reactive state updates. Repository classes abstract data access and model mapping, while service classes handle authentication, database communication, and background operations. The backend layer provides PostgreSQL persistence, authentication, realtime synchronization, edge functions, and file storage.

The architecture also integrates Firebase Cloud Messaging for push notifications and Supabase Realtime for live updates across tasks, analytics, study sessions, and notifications.

Figure 7.1. The layered Acadia Architecture



7.2. Structural Models

The structural design of Acadia is centered around academic management, analytics processing, and intelligent recommendation systems. The database schema contains interconnected entities responsible for courses, semesters, tasks, study sessions, analytics, notifications, mood tracking, and academic results.

Core entities include:

• Users
• Student Profiles
• Semesters
• Courses
• Tasks
• Academic Deadlines
• Study Sessions
• Mood Logs
• Notifications
• Course Results
• Analytics

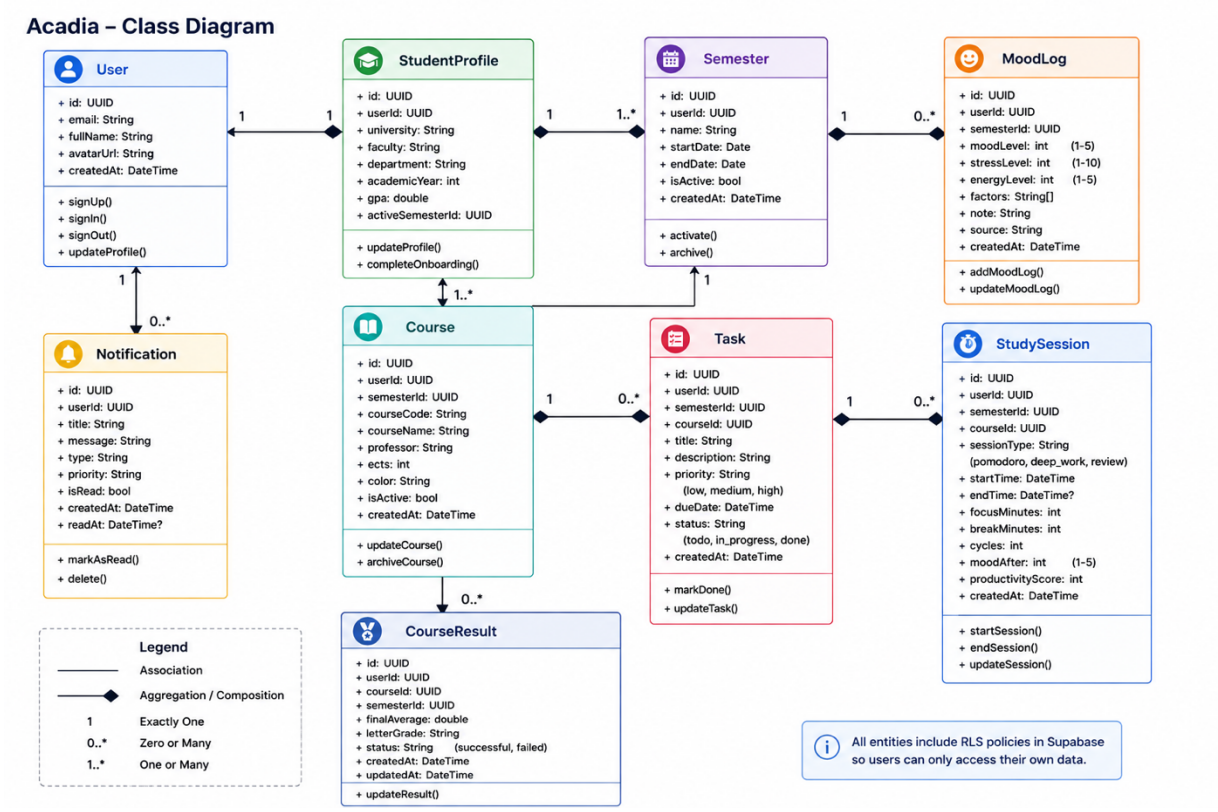
The system uses relational mappings and semester isolation to maintain academic consistency and prevent cross-semester data corruption.

7.2.1. Class Diagram

The class model defines the relationships between the primary academic entities inside Acadia. The StudentProfile entity is associated with semesters, while each semester contains multiple courses, tasks, deadlines, exams, and study sessions. Courses are linked to study sessions and course results for GPA and performance analysis. Notifications and mood logs are attached to users and semesters to support intelligent monitoring and behavioral analytics.

The model reduces redundancy through normalized structures and foreign key relationships. This design improves scalability, analytics computation, and data integrity.

Figure 7.2. Acadia class diagram.

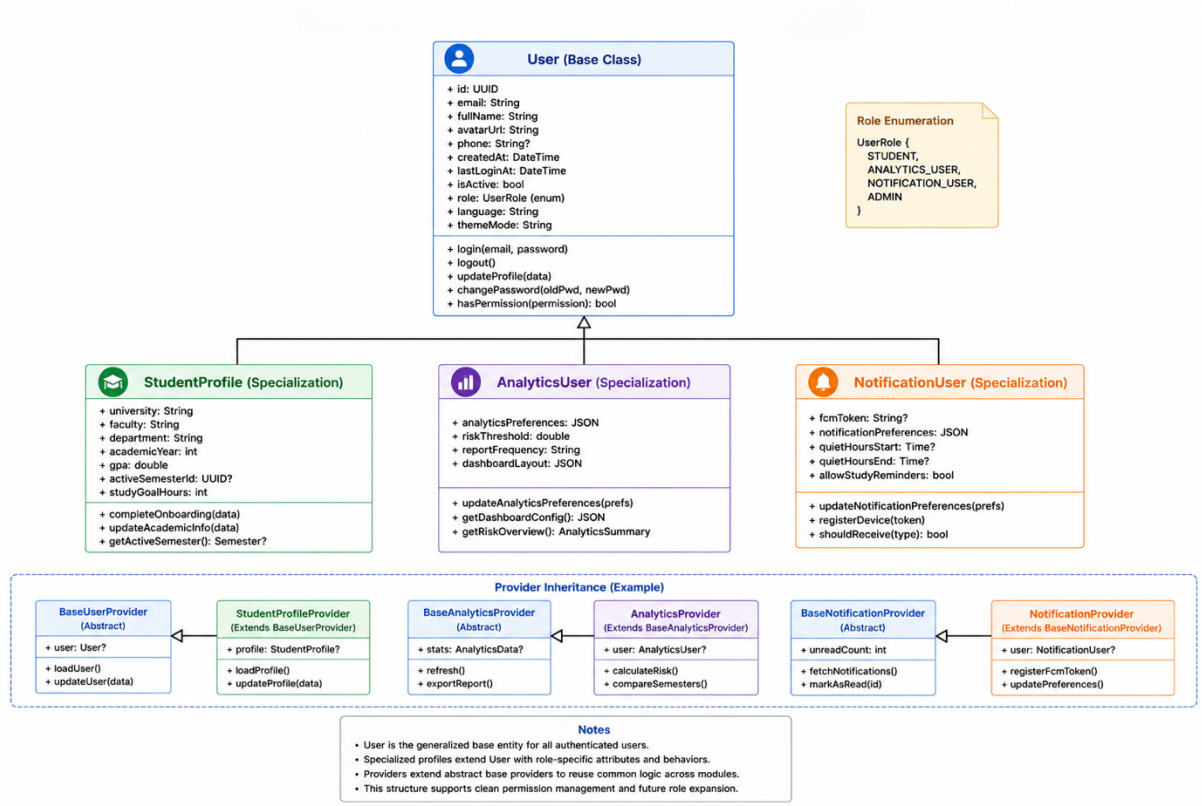


7.2.2. Inheritance (Generalization)

The system applies inheritance principles through role abstraction and modular provider structures. A generalized User entity represents all authenticated users, while the student-specific behavior is extended through the StudentProfile entity. Additional logical specialization exists inside analytics, notification, and AI modules, where different provider classes extend shared base functionality for reusable business logic.

The generalized structure simplifies permission management, authentication handling, and future expansion for additional academic roles.

Figure 7.3. Inheritance Model

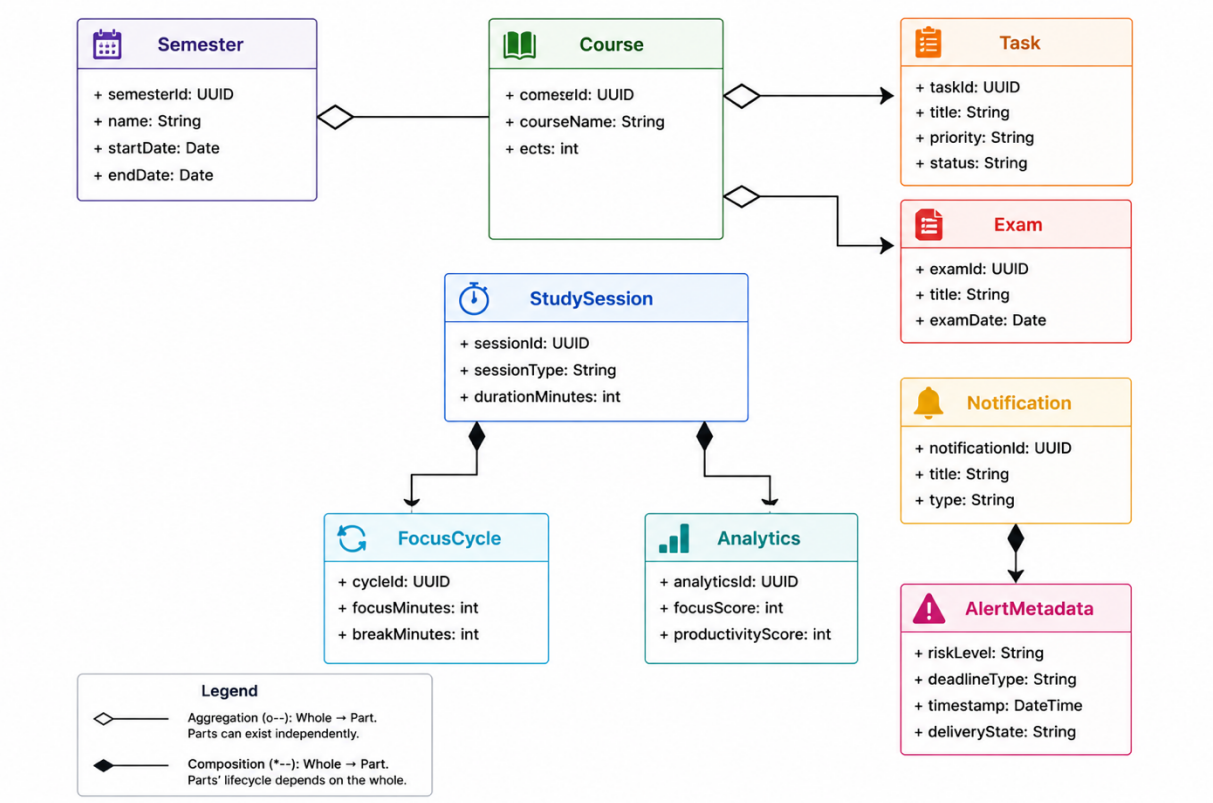


7.2.3. Aggregation / Composition

Acadia uses aggregation and composition relationships to preserve academic lifecycle consistency. A semester aggregates multiple courses, deadlines, tasks, exams, and study sessions. Study sessions compose detailed focus-cycle information, mood values, break tracking, and productivity statistics.

The notification system also composes alert metadata such as risk level, deadline type, timestamps, and delivery state. This structure ensures that dependent records remain logically connected and synchronized throughout the application lifecycle.

Figure 7.4. Aggregation and composition relationships.



7.3. Behavioral Models

Behavioral models describe how Acadia reacts to user actions, analytics changes, and background intelligence events. The system uses reactive state management, realtime subscriptions, and event-driven processing to update analytics, notifications, and recommendations dynamically.

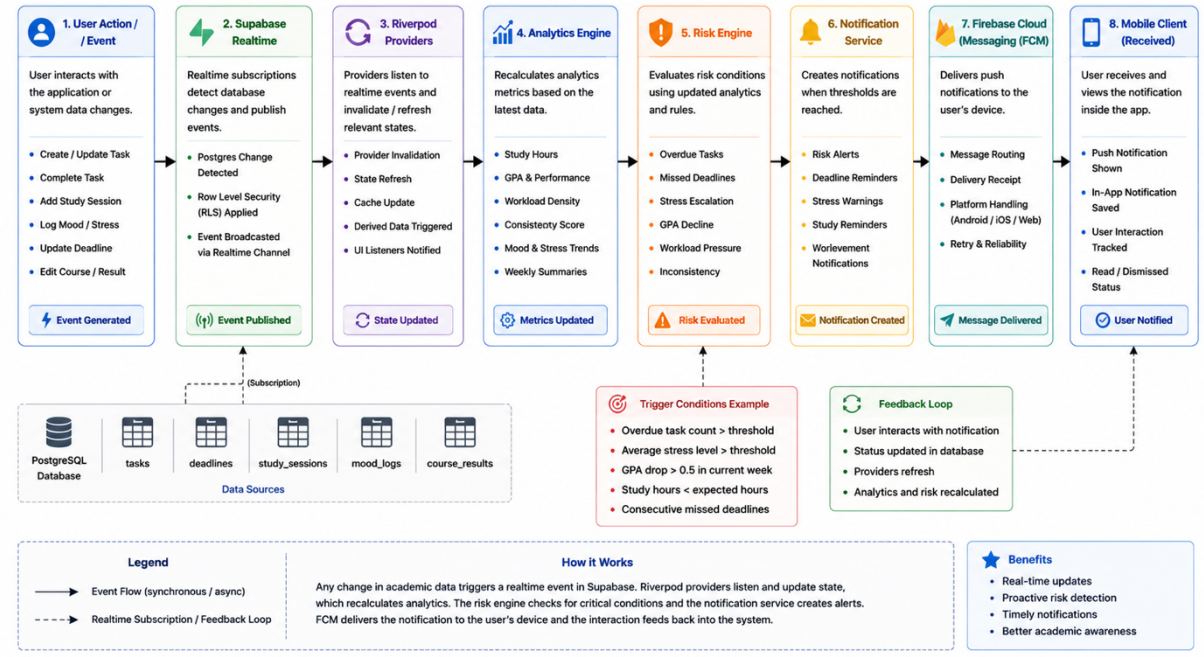
Behavioral flows include:
• Task completion updates
• Risk score recalculation
• Mood tracking events
• Smart notification triggering
• Study recommendation generation
• Analytics refresh operations

7.3.1. Event-Driven Model

Acadia follows an event-driven architecture for realtime synchronization and intelligent monitoring. Changes in tasks, deadlines, study sessions, or mood logs trigger provider invalidation and analytics recalculation through Supabase Realtime subscriptions.

The smart notification engine evaluates overdue tasks, stress escalation, GPA decline, and workload pressure. When threshold conditions are reached, the backend notification service sends alerts through Firebase Cloud Messaging to the user's mobile device.

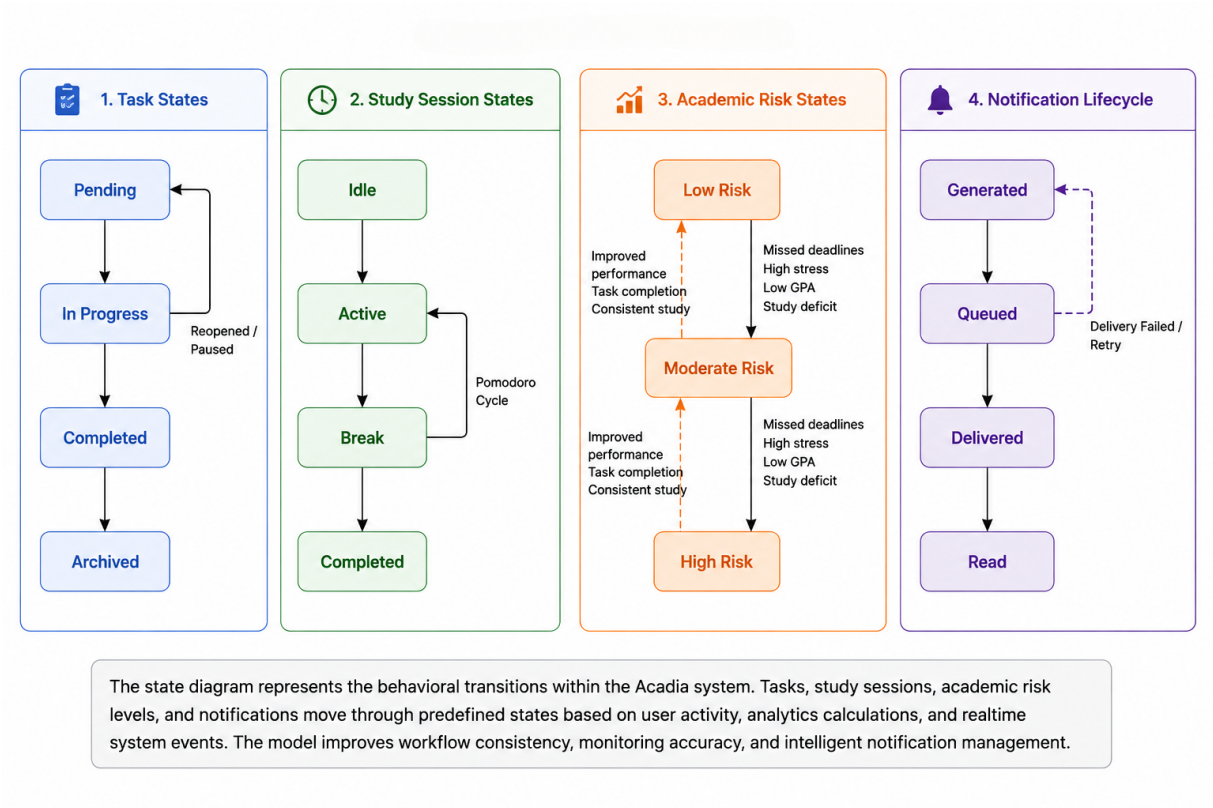
Figure 7.5. Event-driven notification model



7.3.2. State Diagram

The system state model represents transitions between academic monitoring states and study workflow conditions. Study sessions transition through idle, active, paused, completed, and synchronized states. Tasks move between pending, in-progress, overdue, and completed states. Risk evaluation also transitions between low, moderate, and high-risk conditions depending on analytics inputs.

Figure 7.6. Acadia state diagram.



The state diagram represents the behavioral transitions within the Acadia system. Tasks, study sessions, academic risk levels, and notifications move through predefined states based on user activity, analytics calculations, and realtime system events. The model improves workflow consistency, monitoring accuracy, and intelligent notification management.

8. System Implementation

8.1. Development Environment

The Acadia system was developed using a modern cross-platform software engineering environment focused on scalability, realtime synchronization, and modular architecture. The development process followed an Agile Scrum workflow supported by Git-based version control, sprint planning, and collaborative backlog management.

The frontend mobile application was implemented using Flutter and the Dart programming language to support Android, iOS, web, desktop, and responsive layouts from a unified codebase. State management was implemented using Riverpod providers, while navigation management used GoRouter for structured route handling.

The backend infrastructure was built using Supabase services, including PostgreSQL database management, authentication, realtime subscriptions, Edge Functions, and secure cloud storage. Firebase Cloud Messaging was integrated for intelligent push notification delivery and background alert handling.

Local development and testing environments were standardized using shared project configurations, GitHub repositories, environment variable management, and synchronized development workflows across team members. Sprint planning, issue tracking, and feature coordination were managed through structured project management practices defined in the Acadia Project Management Plan.

The development environment also included:

- Flutter SDK
- Android Studio
- Visual Studio Code
- Xcode for iOS builds
- Supabase CLI
- Git and GitHub
- Figma for UI/UX prototyping
- Postman for API testing

This environment enabled rapid iteration, realtime testing, modular deployment, and stable collaboration between frontend, backend, and AI development teams.

8.2. System Components

The Acadia MVP consists of four major system components working together through a layered client-server architecture.

Mobile Client Application

The primary component is the Flutter-based mobile application used by students. The mobile client provides:

- Task management
- Study session tracking
- Academic analytics
- GPA monitoring
- Mood and stress logging
- AI-assisted recommendations
- Smart notifications
- Semester management

The mobile application supports Android and iOS platforms with responsive layouts, dark/light themes, localization support, and realtime synchronization.

Backend API and Realtime Gateway

The backend infrastructure is powered by Supabase services and acts as the central communication layer between application modules. It handles:

- User authentication
- Database operations
- Realtime subscriptions
- Edge Functions
- Risk calculations
- AI request handling
- Notification processing

Realtime updates automatically synchronize changes across tasks, study sessions, courses, analytics, and notifications.

Relational Database System

The database layer uses PostgreSQL with a normalized relational schema designed for academic management, analytics computation, and behavioral tracking. Major entities include:

- Users
- Student profiles
- Semesters
- Courses
- Tasks
- Study sessions
- Mood logs
- Notifications
- Course results
- Academic deadlines

The schema uses foreign key relationships, semester isolation, realtime subscriptions, and Row Level Security (RLS) policies for secure multi-user access control.

AI and Analytics Engine

The intelligence layer combines analytics computation, academic risk scoring, decision support logic, and AI-assisted recommendations. The system evaluates:

- Study consistency
- GPA trends
- Workload density
- Burnout indicators
- Stress escalation
- Deadline pressure
- Academic risk severity

The AI assistant uses a hybrid architecture with local intent matching and cloud-based LLM fallback integration using NVIDIA NIM and Llama 3.1 models through Supabase Edge Functions.

8.3. Code Structure

The Acadia codebase follows a modular layered architecture designed for maintainability, scalability, and separation of responsibilities. The structure is organized by feature domains and infrastructure layers to simplify testing, feature expansion, and collaborative development.

The architecture follows this layered structure:

UI Layer

↓

Provider Layer

↓

Repository Layer

↓

Service Layer

↓

Backend Infrastructure

UI Layer

The UI layer contains:

- Screens
- Widgets
- Navigation flows
- Responsive layouts
- Visualization components
- User interaction logic

This layer is responsible only for presentation and user interaction handling.

Provider Layer

Riverpod providers manage:

- Application state
- Data synchronization
- Business orchestration
- Reactive UI updates
- Analytics coordination

The provider dependency graph separates infrastructure, authentication, analytics, intelligence, and dashboard systems into isolated layers.

Repository Layer

Repositories act as abstraction layers between providers and backend services.

Responsibilities include:

- JSON serialization
- Model mapping
- Data transformation
- Query abstraction
- Local caching coordination

This structure reduces direct coupling between frontend logic and database services.

Service Layer

The service layer handles low-level backend communication and infrastructure operations, including:

- Supabase queries
- Authentication handling
- Notification delivery
- Realtime subscriptions
- Edge Function communication
- Local storage operations

Feature-Based Organization

The project is separated into modular feature domains such as:

- auth
- onboarding
- semesters
- courses
- tasks
- study_sessions
- analytics
- mood_tracking
- notifications
- ai_assistant
- risk_engine

This organization improves readability, parallel team development, and feature isolation during sprint execution.

The modular architecture also simplifies future expansion for additional AI systems, predictive analytics, and university-scale deployment scenarios.

9. MVP Development

9.1. MVP Definition

The Minimum Viable Product (MVP) of Acadia was designed as the smallest fully functional academic productivity and academic risk monitoring system capable of delivering measurable value to university students through intelligent academic organization, realtime progress tracking, and behavioral analytics. The MVP focused on solving core student problems related to workload management, study inconsistency, stress monitoring, and academic risk awareness.

The MVP was developed using an incremental Agile Scrum approach with weekly sprint deliveries, continuous integration, and iterative feature validation.

The primary goals of the MVP were:

- Help students manage academic tasks and deadlines
- Improve study consistency and time management
- Monitor stress and burnout indicators
- Provide academic analytics and GPA tracking
- Generate explainable academic risk scores
- Deliver intelligent study recommendations
- Support realtime synchronization across devices

The MVP intentionally prioritized essential academic workflows before advanced predictive systems or enterprise-level integrations. The scope focused on practical usability, stability, and measurable academic support functionality.

The implemented MVP supports:

- Authentication and onboarding
- Semester and course management
- Task and deadline organization
- Study session tracking
- Mood and stress monitoring
- Academic analytics dashboards
- Risk engine calculations
- AI-assisted recommendations

- Smart notification workflows

The final MVP evolved beyond a simple prototype into a stable cross-platform academic management platform with realtime intelligence and modular backend infrastructure.

9.2. Implemented Features

The Acadia MVP includes multiple integrated systems designed around academic productivity, analytics, and behavioral monitoring. Core implemented features include:

Authentication and User Management

The system supports:

- Secure login and signup
- OTP verification
- Password reset flow
- Session persistence
- Protected routing
- Academic onboarding

Authentication was implemented using Supabase Auth with role-safe database isolation and secure session management.

Semester and Course Management

Students can:

- Create semesters
- Manage courses
- Configure ECTS values
- Track academic periods
- Switch active semesters

Each semester acts as an isolated academic workspace to improve organization and analytics separation.

Task and Deadline System

The academic organization system includes:

- Task creation and editing
- Deadline management

- Priority tracking
- Overdue monitoring
- Linked task-deadline workflows
- Progress tracking
- Sorting and filtering

Realtime synchronization updates user dashboards immediately after task changes.

Study Session and Timer System

The study module supports:

- Pomodoro mode
- Deep work sessions
- Review sessions
- Focus and break cycles
- Session persistence
- Study history
- Productivity analytics

The timer system survives app restarts, backgrounding, and lifecycle interruptions through persistent local storage recovery mechanisms.

Mood and Stress Tracking

Students can record:

- Mood levels
- Stress levels
- Energy levels
- Emotional factors
- Session-based emotional states

The analytics engine computes burnout trends, emotional consistency, and stress escalation indicators using historical mood data.

Academic Analytics Dashboard

The analytics engine calculates:

- GPA
- Study hours
- Focus score
- Burnout score
- Task completion rate
- Stress averages
- Study consistency
- Semester comparisons
- Performance summaries

The dashboard contains interactive charts, trend analysis, and realtime metric updates.

Academic Risk Engine

The risk engine evaluates:

- Missed deadlines
- GPA decline
- Study deficit
- Workload pressure
- Stress escalation
- Inconsistent study behavior

The system generates academic risk scores, severity levels, and adaptive recommendations.

AI Assistant

The AI assistant provides:

- Academic summaries
- Study recommendations
- Progress explanations
- Deadline queries
- Risk explanations

- Productivity guidance

The assistant uses hybrid execution logic combining local intent parsing and cloud-based LLM fallback systems.

Smart Notification System

The notification engine supports:

- Deadline reminders
- Risk alerts
- Stress warnings
- Study goal notifications
- Overdue task alerts
- Adaptive cooldown protection
- Background evaluation

The system reduces spam using behavioral notification intelligence and scheduling rules.

9.3. System Interfaces

The Acadia system contains multiple user interfaces optimized for academic productivity, accessibility, and realtime interaction.

Student Mobile Interface

The primary user interface is the Flutter-based student application. The design emphasizes:

- Fast navigation
- Minimal interaction friction
- Realtime updates
- Dashboard visibility
- Study workflow efficiency

The application uses a five-tab architecture consisting of:

Interface Area	Purpose
Home	Dashboard and academic overview
Tasks	Task and deadline management
Study	Study timer and session tracking
Analytics	Academic insights and trends
Profile	Settings and preferences

The interface includes:

- Glassmorphism visual design
- Dynamic gradients
- Interactive charts
- Heatmaps
- Animated transitions
- Dark/light theme support
- Responsive layouts
- Localization support

The current MVP supports English and Turkish localization.

Analytics and Dashboard Interfaces

The analytics section prioritizes fast access to academic insights and trend visibility. Users can monitor:

- GPA changes
- Study consistency
- Burnout indicators
- Stress trends
- Risk severity
- Academic workload

The dashboard integrates realtime provider updates and animated chart systems for responsive feedback.

AI Assistant Interface

The AI interface provides:

- Conversational academic interaction
- Quick action chips
- AI-generated recommendations
- Academic summaries
- Context-aware responses

The assistant interface was designed to reduce complexity while maintaining explainable academic recommendations.

9.4. System Demonstration

The Acadia MVP was demonstrated using multiple realistic academic workflow scenarios to validate system stability, usability, and realtime synchronization behavior.

The demonstration included the following representative scenarios:

Authentication and Onboarding Scenario

A new student account was created using secure authentication and OTP verification. The onboarding process demonstrated:

- Academic profile setup
- Semester creation
- Course initialization
- Study goal configuration
- Localization selection

Academic Organization Scenario

The system demonstrated creation and management of:

- Courses
- Academic tasks
- Deadlines
- Exams
- Semester schedules

Realtime updates automatically refreshed dashboards and analytics after each modification.

Study Session Scenario

The Pomodoro study system demonstration included:

- Starting focus sessions
- Timer persistence

- Break cycle transitions
- Session completion tracking
- Study analytics generation

The timer recovery system successfully preserved active sessions after simulated app backgrounding and reopening.

Mood and Burnout Monitoring Scenario

The demonstration included mood logging workflows where students recorded stress and energy levels. The analytics engine generated:

- Stress trends
- Burnout indicators
- Emotional consistency metrics
- Risk escalation signals

Academic Risk and Recommendation Scenario

The risk engine demonstration simulated:

- Overdue tasks
- Missed study goals
- GPA decline
- Heavy workload periods

The system generated adaptive risk scores, severity classifications, and intelligent study recommendations using weighted scoring logic.

AI Assistant Demonstration

The AI assistant demonstration included:

- Deadline queries
- Study summary requests
- Productivity recommendations
- Academic risk explanations
- Semester performance summaries

The hybrid AI execution model successfully routed simple requests locally and complex requests through cloud-based inference systems.

The demonstration confirmed that the MVP delivers stable academic workflow management, realtime synchronization, intelligent analytics, and adaptive recommendation capabilities within a production-ready cross-platform architecture.

10. Testing and Evaluation

10.1. Testing Strategy

The Acadia system was tested using a multi-layered testing strategy designed to validate functionality, realtime synchronization, backend stability, AI behavior, and user experience quality throughout the development lifecycle. Testing activities were distributed across multiple Scrum sprints, especially during Sprint 12 and Sprint 13, which focused on stabilization, integration testing, quality assurance, and bug fixing.

The testing process combined:

- Unit testing
- Integration testing
- Realtime synchronization testing
- Scenario-based acceptance testing
- UI/UX validation
- Multi-device testing
- AI assistant testing
- Risk engine validation
- Notification delivery testing
- Performance testing

The testing workflow was performed iteratively during sprint reviews and final QA stages.

Unit Testing

Unit testing focused on validating isolated system modules and business logic. Major tested components included:

- Authentication providers
- Analytics calculations
- GPA computation
- Risk engine formulas
- Study timer logic
- Notification cooldown logic
- Task validation rules

- Realtime provider updates

Special attention was given to academic calculations, analytics consistency, and semester isolation logic because these systems directly affect academic monitoring accuracy.

Integration Testing

Integration testing validated communication between frontend providers, repositories, Supabase backend services, realtime subscriptions, and PostgreSQL database operations.

Integrated workflows included:

- Authentication → Dashboard loading
- Task creation → Analytics update
- Study session completion → Risk recalculation
- Mood logging → Burnout analytics
- Notification generation → Firebase delivery
- AI assistant → Backend inference pipeline

The integration testing phase confirmed successful synchronization between mobile interfaces and cloud backend services.

Realtime Synchronization Testing

Realtime testing validated Supabase Realtime subscriptions and provider invalidation behavior during:

- Task updates
- Deadline changes
- Analytics refresh operations
- Notification delivery
- Study session synchronization

Testing confirmed that dashboards and analytics screens updated automatically without requiring manual refresh actions.

AI and Risk Engine Testing

The AI assistant and academic risk engine were tested using multiple academic workload scenarios. Testing verified:

- Risk score consistency
- Recommendation relevance
- Burnout detection behavior

- GPA warning generation
- Deadline pressure evaluation
- Study recommendation accuracy

The AI workflow was also tested for fallback behavior when cloud inference services were unavailable.

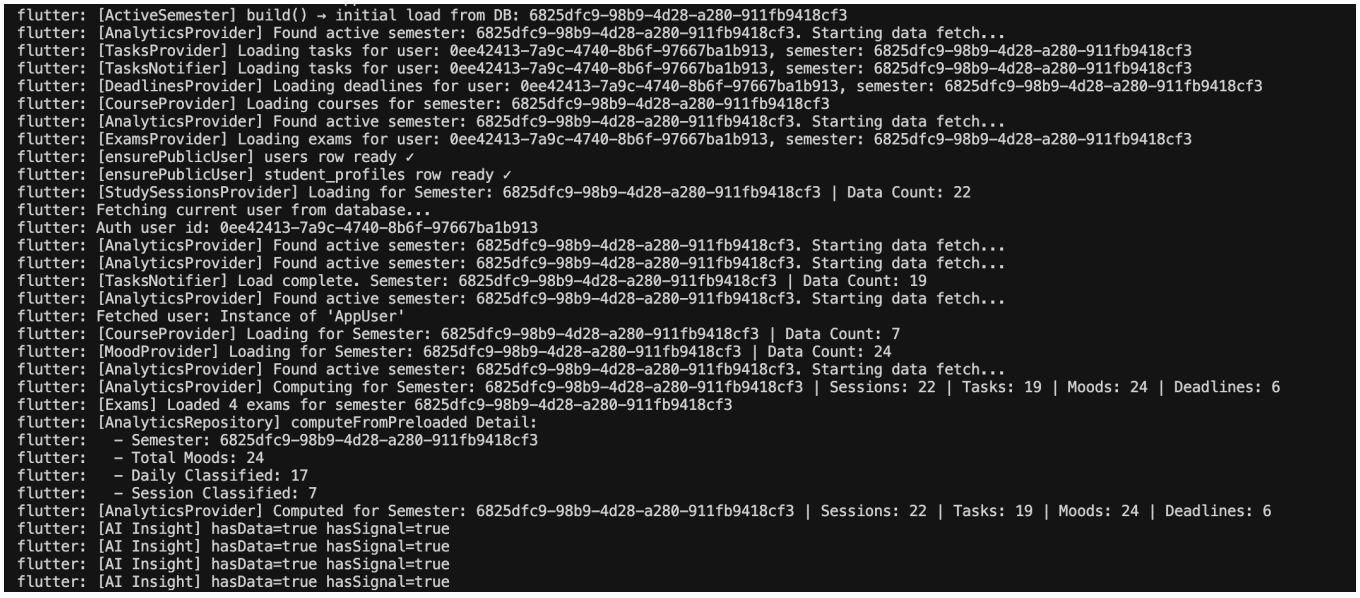
UI/UX and Cross-Platform Testing

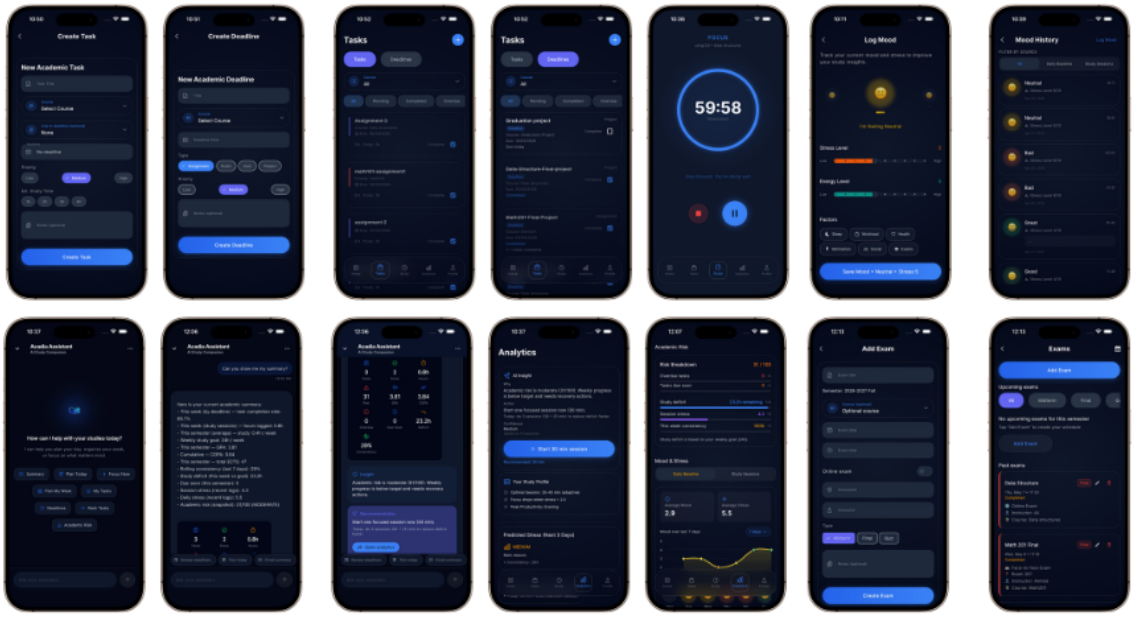
The application was tested across different screen sizes, operating systems, and device configurations to validate:

- Responsive layouts
- Theme consistency
- Navigation stability
- Animation behavior
- Accessibility visibility
- Localization rendering
- Dark/light mode consistency

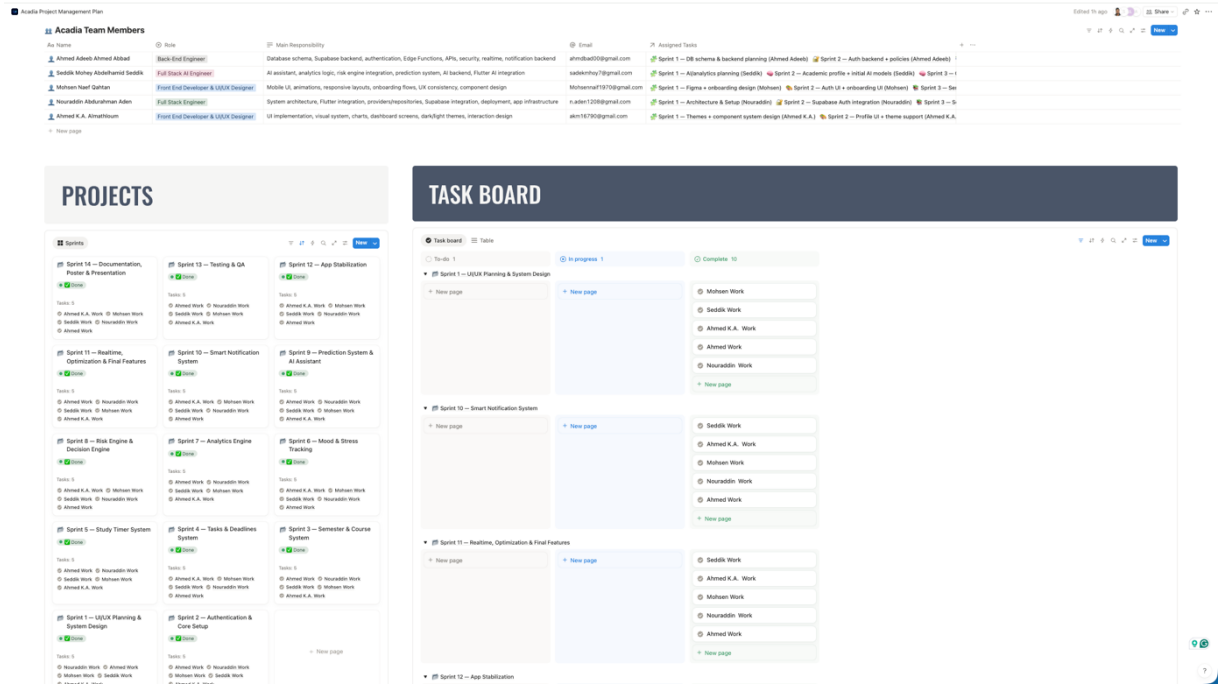
Testing was performed on Android devices, iOS simulators, and desktop environments.

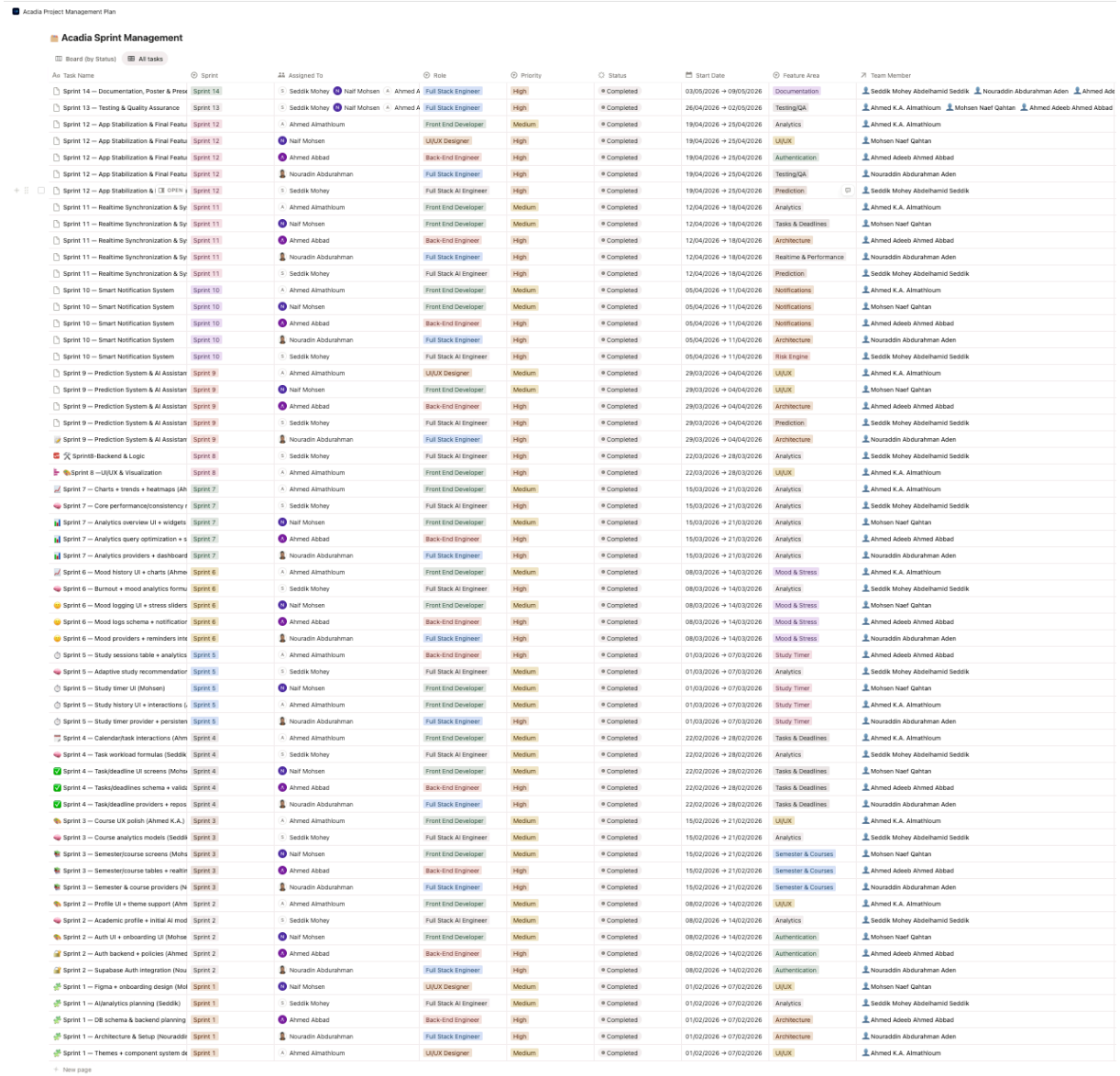
Figure 10.1. Testing and Quality Assurance Activities in Acadia





Tue 28 Apr
6:19
Test Remote Push
App is closed or backgrounded! This is real.
Sat 9 May
6:35
OMALIA Documentary
COOPSCORNER
Mood Check
How are you feeling? Take a moment to log your mood.
Do Not Disturb
Tue 5 May
5:11
Exam Now
Data structure exam Midterm is starting now
17:11 - Room 108
Turkcell
Sun 10 May · 23 Dhu'l-Q. 1447 AH
1:48
Acadia
Moderate Risk
Stay consistent to avoid falling behind.
Moderate Risk
Stay consistent to avoid falling behind.


Acadia Project Management Plan
Acadia Team Members
Name
Role
Main Responsibility
Email
Assigned Tasks
Ahmed Adeeb Ahmed Abbad
Back-End Engineer
Database schema, Supabase backend, authentication, Edge Functions, APIs, security, realtime, notification backend
ahmdbad00@gmail.com
Sprint 1 — DB schema & backend planning (Ahmed Adeeb)
Sprint 2 — Auth backend + policies (Ahmed Adeeb)
Seddik Mohey Abdelhamid Seddik
Full Stack AI Engineer
AI assistant, analytics logic, risk engine integration, prediction system, AI backend, Flutter AI integration
sadekmhoy7@gmail.com
Sprint 1 — AI/analytics planning (Seddik)
Sprint 2 — Academic profile + initial AI models (Seddik)
Mohsen Naef Qahtan
Front End Developer & UI/UX Designer
Mobile UI, animations, responsive layouts, onboarding flows, UX consistency, component design
Mohsennaif1970@gmail.com
Sprint 1 — Figma + onboarding design (Mohsen)
Sprint 2 — Auth UI + onboarding UI (Mohsen)
Nouraddin Abdurahman Aden
Full Stack Engineer
System architecture, Flutter integration, providers/repositories, Supabase integration, deployment, app infrastructure
n.aden1208@gmail.com
Sprint 1 — Architecture & Setup (Nouraddin)
Sprint 2 — Supabase Auth integration (Nouraddin)
Ahmed K.A. Almathloum
Front End Developer & UI/UX Designer
UI implementation, visual system, charts, dashboard screens, dark/light themes, interaction design
akm16790@gmail.com
Sprint 1 — Themes + component system design (Ahmed K.A.)
Sprint 2 — Profile UI + theme support (Ahmed K.A.
PROJECTS
TASK BOARD
Sprints
Task board
Table
To-do 1
In progress 1
Complete 10
Sprint 1 — UI/UX Planning & System Design
Sprint 10 — Smart Notification System
Sprint 11 — Realtime, Optimization & Final Features
Sprint 12 — App Stabilization
Mohsen Work
Seddik Work
Ahmed K.A. Work
Ahmed Work
Nouraddin Work
New page



Acadia Project Management Plan

Acadia Sprint Management

Board (by Status) | All tasks

Task Name	Sprint	Assigned To	Role	Priority	Status	Start Date	Feature Area	Team Member
Sprint 14 — Documentation, Poster & Presc	Sprint 14	Seddik Mohey, Naif Mohsen, Ahmed A	Full Stack Engineer	High	Completed	03/05/2026 → 09/05/2026	Documentation	Seddik Mohey Abdelhamid Seddik, Nouraddin Abdurahman Aden, Ahmed Ade
Sprint 13 — Testing & Quality Assurance	Sprint 13	Seddik Mohey, Naif Mohsen, Ahmed A	Full Stack Engineer	High	Completed	26/04/2026 → 02/05/2026	Testing/QA	Ahmed K.A. Almathloum, Mohsen Naef Qahtan, Ahmed Adeeb Ahmed Abbad
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Ahmed Almathloum	Front End Developer	Medium	Completed	19/04/2026 → 25/04/2026	Analytics	Ahmed K.A. Almathloum
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Naif Mohsen	UI/UX Designer	High	Completed	19/04/2026 → 25/04/2026	UI/UX	Mohsen Naef Qahtan
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Ahmed Abbad	Back-End Engineer	High	Completed	19/04/2026 → 25/04/2026	Authentication	Ahmed Adeeb Ahmed Abbad
Sprint 12 — App Stabilization & Final Featu	Sprint 12	Nouradin Abdurahman	Full Stack Engineer	High	Completed	19/04/2026 → 25/04/2026	Testing/QA	Nouraddin Abdurahman Aden
Sprint 12 — App Stabilization & I OPEN	Sprint 12	Seddik Mohey	Full Stack AI Engineer	High	Completed	19/04/2026 → 25/04/2026	Prediction	Seddik Mohey Abdelhamid Seddik
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Ahmed Almathloum	Front End Developer	Medium	Completed	12/04/2026 → 18/04/2026	Analytics	Ahmed K.A. Almathloum
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Naif Mohsen	Front End Developer	Medium	Completed	12/04/2026 → 18/04/2026	Tasks & Deadlines	Mohsen Naef Qahtan
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Ahmed Abbad	Back-End Engineer	High	Completed	12/04/2026 → 18/04/2026	Architecture	Ahmed Adeeb Ahmed Abbad
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Nouradin Abdurahman	Full Stack Engineer	High	Completed	12/04/2026 → 18/04/2026	Realtime & Performance	Nouraddin Abdurahman Aden
Sprint 11 — Realtime Synchronization & Sy	Sprint 11	Seddik Mohey	Full Stack AI Engineer	High	Completed	12/04/2026 → 18/04/2026	Prediction	Seddik Mohey Abdelhamid Seddik
Sprint 10 — Smart Notification System	Sprint 10	Ahmed Almathloum	Front End Developer	Medium	Completed	05/04/2026 → 11/04/2026	Notifications	Ahmed K.A. Almathloum
Sprint 10 — Smart Notification System	Sprint 10	Naif Mohsen	Front End Developer	Medium	Completed	05/04/2026 → 11/04/2026	Notifications	Mohsen Naef Qahtan
Sprint 10 — Smart Notification System	Sprint 10	Ahmed Abbad	Back-End Engineer	High	Completed	05/04/2026 → 11/04/2026	Notifications	Ahmed Adeeb Ahmed Abbad
Sprint 10 — Smart Notification System	Sprint 10	Nouradin Abdurahman	Full Stack Engineer	High	Completed	05/04/2026 → 11/04/2026	Architecture	Nouraddin Abdurahman Aden
Sprint 10 — Smart Notification System	Sprint 10	Seddik Mohey	Full Stack AI Engineer	High	Completed	05/04/2026 → 11/04/2026	Risk Engine	Seddik Mohey Abdelhamid Seddik
Sprint 9 — Prediction System & AI Assistan	Sprint 9	Ahmed Almathloum	UI/UX Designer	Medium	Completed	29/03/2026 → 04/04/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 9 — Prediction System & AI Assistan	Sprint 9	Naif Mohsen	Front End Developer	Medium	Completed	29/03/2026 → 04/04/2026	UI/UX	Mohsen Naef Qahtan
Sprint 9 — Prediction System & AI Assistan	Sprint 9	Ahmed Abbad	Back-End Engineer	High	Completed	29/03/2026 → 04/04/2026	Architecture	Ahmed Adeeb Ahmed Abbad
Sprint 9 — Prediction System & AI Assistan	Sprint 9	Seddik Mohey	Full Stack AI Engineer	High	Completed	29/03/2026 → 04/04/2026	Prediction	Seddik Mohey Abdelhamid Seddik
Sprint 9 — Prediction System & AI Assistan	Sprint 9	Nouradin Abdurahman	Full Stack Engineer	High	Completed	29/03/2026 → 04/04/2026	Architecture	Nouraddin Abdurahman Aden
🛠️ Sprint8-Backend & Logic	Sprint 8	Seddik Mohey	Full Stack AI Engineer	High	Completed	22/03/2026 → 28/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
🎨Sprint 8 —UI/UX & Visualization	Sprint 8	Ahmed Almathloum	Front End Developer	High	Completed	22/03/2026 → 28/03/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 7 — Charts + trends + heatmaps (Ah	Sprint 7	Ahmed Almathloum	Front End Developer	Medium	Completed	15/03/2026 → 21/03/2026	Analytics	Ahmed K.A. Almathloum
Sprint 7 — Core performance/consistency r	Sprint 7	Seddik Mohey	Full Stack AI Engineer	High	Completed	15/03/2026 → 21/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 7 — Analytics overview UI + widgets	Sprint 7	Naif Mohsen	Front End Developer	Medium	Completed	15/03/2026 → 21/03/2026	Analytics	Mohsen Naef Qahtan
Sprint 7 — Analytics query optimization + s	Sprint 7	Ahmed Abbad	Back-End Engineer	High	Completed	15/03/2026 → 21/03/2026	Analytics	Ahmed Adeeb Ahmed Abbad
Sprint 7 — Analytics providers + dashboard	Sprint 7	Nouradin Abdurahman	Full Stack Engineer	High	Completed	15/03/2026 → 21/03/2026	Analytics	Nouraddin Abdurahman Aden
Sprint 6 — Mood history UI + charts (Ahme	Sprint 6	Ahmed Almathloum	Front End Developer	Medium	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Ahmed K.A. Almathloum
Sprint 6 — Burnout + mood analytics formu	Sprint 6	Seddik Mohey	Full Stack AI Engineer	High	Completed	08/03/2026 → 14/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 6 — Mood logging UI + stress sliders	Sprint 6	Naif Mohsen	Front End Developer	Medium	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Mohsen Naef Qahtan
Sprint 6 — Mood logs schema + notificatior	Sprint 6	Ahmed Abbad	Back-End Engineer	High	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Ahmed Adeeb Ahmed Abbad
Sprint 6 — Mood providers + reminders inte	Sprint 6	Nouradin Abdurahman	Full Stack Engineer	High	Completed	08/03/2026 → 14/03/2026	Mood & Stress	Nouraddin Abdurahman Aden
Sprint 5 — Study sessions table + analytics	Sprint 5	Ahmed Almathloum	Back-End Engineer	High	Completed	01/03/2026 → 07/03/2026	Study Timer	Ahmed Adeeb Ahmed Abbad
Sprint 5 — Adaptive study recommendatior	Sprint 5	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	01/03/2026 → 07/03/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 5 — Study timer UI (Mohsen)	Sprint 5	Naif Mohsen	Front End Developer	Medium	Completed	01/03/2026 → 07/03/2026	Study Timer	Mohsen Naef Qahtan
Sprint 5 — Study history UI + interactions (	Sprint 5	Ahmed Almathloum	Front End Developer	Medium	Completed	01/03/2026 → 07/03/2026	Study Timer	Ahmed K.A. Almathloum
Sprint 5 — Study timer provider + persisten	Sprint 5	Nouradin Abdurahman	Full Stack Engineer	High	Completed	01/03/2026 → 07/03/2026	Study Timer	Nouraddin Abdurahman Aden
Sprint 4 — Calendar/task interactions (Ahm	Sprint 4	Ahmed Almathloum	Front End Developer	Medium	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Ahmed K.A. Almathloum
Sprint 4 — Task workload formulas (Seddik	Sprint 4	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	22/02/2026 → 28/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
✅ Sprint 4 — Task/deadline UI screens (Mohs	Sprint 4	Naif Mohsen	Front End Developer	Medium	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Mohsen Naef Qahtan
✅ Sprint 4 — Tasks/deadlines schema + validi	Sprint 4	Ahmed Abbad	Back-End Engineer	High	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Ahmed Adeeb Ahmed Abbad
✅ Sprint 4 — Task/deadline providers + repos	Sprint 4	Nouradin Abdurahman	Full Stack Engineer	High	Completed	22/02/2026 → 28/02/2026	Tasks & Deadlines	Nouraddin Abdurahman Aden
Sprint 3 — Course UX polish (Ahmed K.A.)	Sprint 3	Ahmed Almathloum	Front End Developer	Medium	Completed	15/02/2026 → 21/02/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 3 — Course analytics models (Seddik	Sprint 3	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	15/02/2026 → 21/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 3 — Semester/course screens (Mohs	Sprint 3	Naif Mohsen	Front End Developer	Medium	Completed	15/02/2026 → 21/02/2026	Semester & Courses	Mohsen Naef Qahtan
Sprint 3 — Semester/course tables + realtir	Sprint 3	Ahmed Abbad	Back-End Engineer	High	Completed	15/02/2026 → 21/02/2026	Semester & Courses	Ahmed Adeeb Ahmed Abbad
Sprint 3 — Semester & course providers (N	Sprint 3	Nouradin Abdurahman	Full Stack Engineer	High	Completed	15/02/2026 → 21/02/2026	Semester & Courses	Nouraddin Abdurahman Aden
Sprint 2 — Profile UI + theme support (Ahm	Sprint 2	Ahmed Almathloum	Front End Developer	Medium	Completed	08/02/2026 → 14/02/2026	UI/UX	Ahmed K.A. Almathloum
Sprint 2 — Academic profile + initial AI mod	Sprint 2	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	08/02/2026 → 14/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 2 — Auth UI + onboarding UI (Mohse	Sprint 2	Naif Mohsen	Front End Developer	Medium	Completed	08/02/2026 → 14/02/2026	Authentication	Mohsen Naef Qahtan
Sprint 2 — Auth backend + policies (Ahmed	Sprint 2	Ahmed Abbad	Back-End Engineer	High	Completed	08/02/2026 → 14/02/2026	Authentication	Ahmed Adeeb Ahmed Abbad
Sprint 2 — Supabase Auth integration (Nou	Sprint 2	Nouradin Abdurahman	Full Stack Engineer	High	Completed	08/02/2026 → 14/02/2026	Authentication	Nouraddin Abdurahman Aden
Sprint 1 — Figma + onboarding design (Mol	Sprint 1	Naif Mohsen	UI/UX Designer	Medium	Completed	01/02/2026 → 07/02/2026	UI/UX	Mohsen Naef Qahtan
Sprint 1 — AI/analytics planning (Seddik)	Sprint 1	Seddik Mohey	Full Stack AI Engineer	Medium	Completed	01/02/2026 → 07/02/2026	Analytics	Seddik Mohey Abdelhamid Seddik
Sprint 1 — DB schema & backend planning	Sprint 1	Ahmed Abbad	Back-End Engineer	High	Completed	01/02/2026 → 07/02/2026	Architecture	Ahmed Adeeb Ahmed Abbad
Sprint 1 — Architecture & Setup (Nouraddi	Sprint 1	Nouradin Abdurahman	Full Stack Engineer	High	Completed	01/02/2026 → 07/02/2026	Architecture	Nouraddin Abdurahman Aden
Sprint 1 — Themes + component system (A	Sprint 1	Ahmed Almathloum	UI/UX Designer	Medium	Completed	01/02/2026 → 07/02/2026	UI/UX	Ahmed K.A. Almathloum

New page

10.2. Test Scenarios

Several realistic academic workflow scenarios were executed to evaluate the stability, correctness, and usability of the Acadia MVP.

Table 10.1 presents representative testing scenarios used during evaluation.

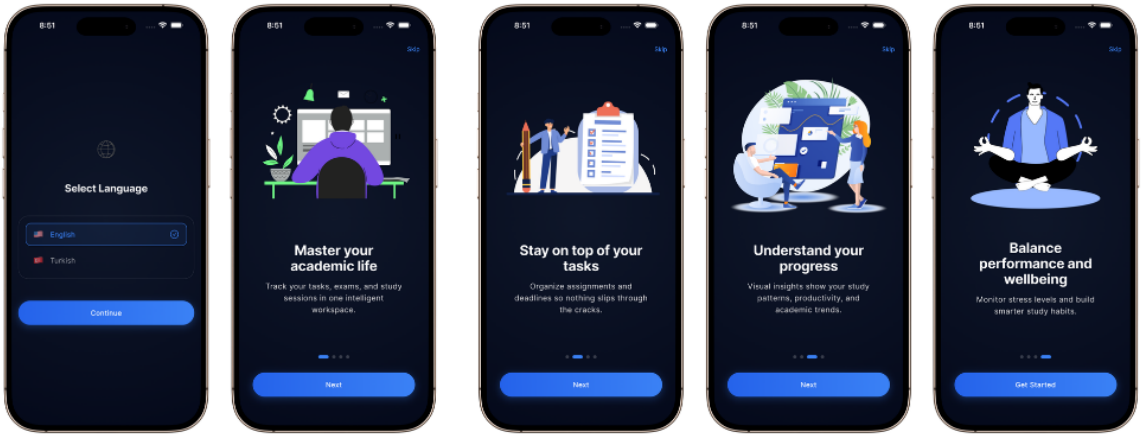
Scenario ID	Test Scenario	Expected Result
TS-1	User login with valid credentials	Successful authentication and dashboard access
TS-2	User login with invalid credentials	Error validation message displayed
TS-3	Semester creation	Semester stored and displayed correctly
TS-4	Course creation inside semester	Course linked to correct semester
TS-5	Task creation with deadline	Task appears in dashboard and analytics
TS-6	Overdue task detection	Risk engine generates overdue warning
TS-7	Pomodoro session execution	Timer tracks focus and break cycles correctly
TS-8	App restart during study session	Session recovery restores active timer
TS-9	Mood logging submission	Mood analytics updated successfully
TS-10	Analytics dashboard refresh	Charts and metrics update automatically
TS-11	Push notification delivery	Notification appears on mobile device
TS-12	AI assistant academic query	AI response generated successfully
TS-13	GPA decline simulation	Academic risk score increases
TS-14	Realtime task synchronization	UI updates without manual refresh
TS-15	Dark/light mode switching	Theme updates consistently across screens

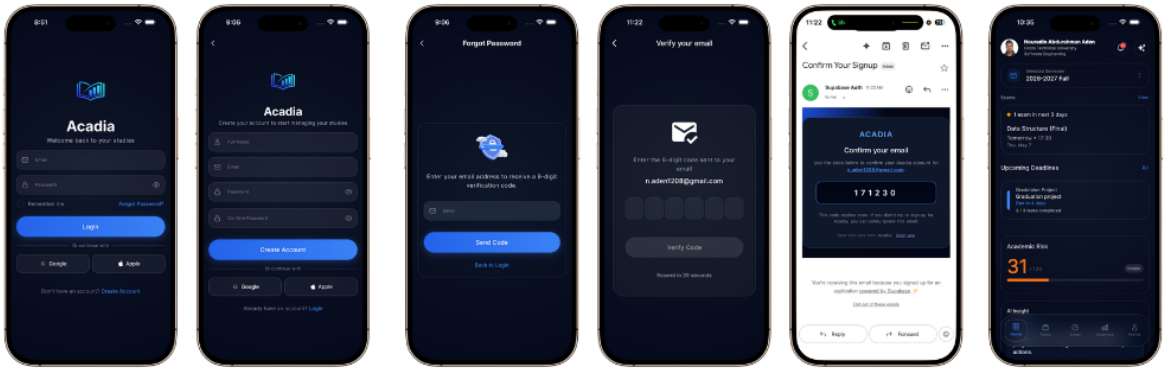
The following representative workflows were demonstrated and validated during testing:

Authentication Workflow

The authentication test validated:

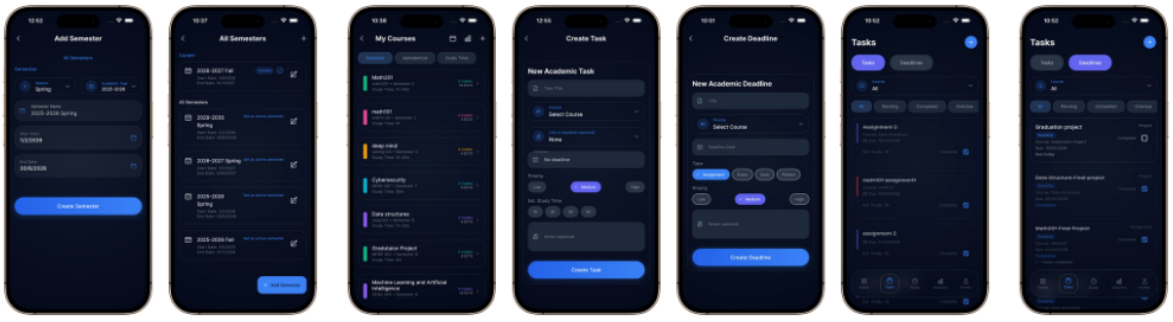
- Signup
- Login
- Password reset
- Session persistence
- Protected navigation
- User onboarding





Academic Management Workflow

This scenario tested semester organization and academic task management.



Study Session Workflow

The Pomodoro and study tracking systems were evaluated using long-duration session simulations.

Analytics and Risk Workflow

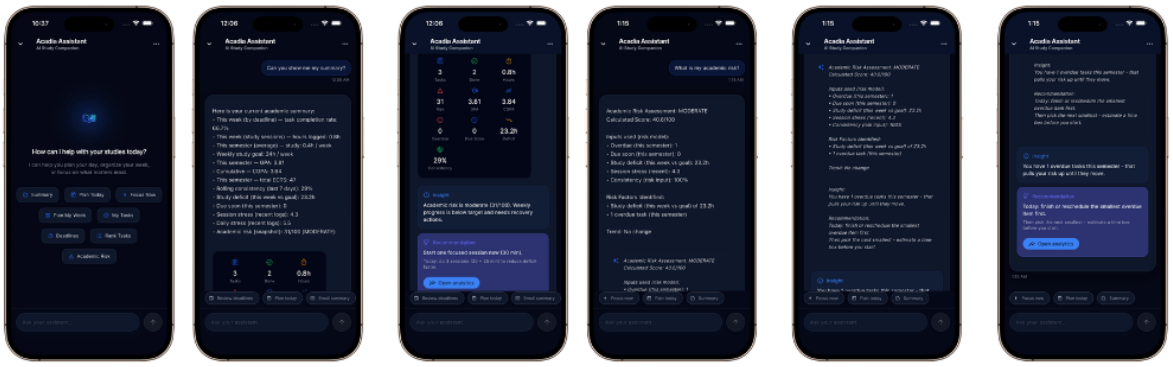
Analytics recalculation and academic risk generation were tested using simulated workload pressure and overdue tasks.
Figure 10.2.1. Analytics Dashboard After Realtime Data Synchronization

Smart Notification Workflow

Notification testing validated realtime delivery and cooldown logic.
Figure 10.2.2. Realtime Deadline Notification Received on Mobile Device

AI Assistant Workflow

The AI assistant was tested using academic recommendation and productivity-related prompts.



10.3. Test Results

The testing and evaluation process confirmed that the Acadia MVP successfully satisfies the major functional and non-functional requirements defined during the analysis and sprint planning phases.

Most critical user stories passed validation successfully during pilot testing and integration evaluation.

The following major results were observed:

Functional Validation Results

The implemented features operated successfully under normal testing conditions, including:

- Authentication workflows
- Semester management
- Task organization
- Study session tracking
- Mood logging
- Analytics generation
- Academic risk evaluation
- AI recommendation delivery
- Notification synchronization

Realtime synchronization behaved consistently across multiple workflows without requiring manual interface refreshes.

Performance Results

The application maintained stable performance during:

- Concurrent analytics updates
- Realtime event processing
- Notification generation
- Long-duration study sessions
- Dashboard rendering
- Background synchronization

Database queries and provider invalidation operations were optimized during Sprint 11 and Sprint 12 to improve responsiveness and reduce unnecessary recalculations.

AI and Risk Engine Results

The academic risk engine successfully identified:

- Overdue tasks
- Heavy workload periods
- GPA decline patterns
- Stress escalation trends
- Study inconsistency behavior

The recommendation engine generated adaptive suggestions based on workload density, study history, and emotional tracking data.

UI/UX Evaluation Results

The interface evaluation confirmed:

- Responsive mobile layouts
- Stable navigation flows
- Consistent dark/light themes
- Smooth animations
- Clear dashboard visibility
- Usable academic workflows

Minor improvements were identified during final QA testing, including:

- Better accessibility contrast in some chart views
- Improved validation wording
- Small UI spacing refinements
- Notification message clarity improvements

These issues were corrected during final stabilization sprints before deployment preparation.

Final Evaluation Summary

The final evaluation demonstrates that Acadia successfully delivers a stable and functional academic productivity platform capable of combining:

- academic organization
- realtime synchronization
- intelligent analytics
- academic risk monitoring
- AI-assisted recommendations
- behavioral notification systems

The testing results also confirm that the modular architecture and Scrum-based development process supported reliable feature integration, maintainability, and scalable future expansion.

11. Results and Discussion

The development and evaluation of Acadia demonstrate that a Scrum-driven software engineering approach can successfully produce a functional and intelligent academic productivity platform within a limited university project timeline. Through iterative sprint development, modular architecture design, and continuous testing, the project evolved from an initial concept into a stable cross-platform MVP capable of supporting academic organization, behavioral monitoring, analytics processing, and AI-assisted recommendations.

The final system successfully integrated multiple complex subsystems into a unified academic management platform, including:

- semester and course management
- task and deadline tracking
- Pomodoro and deep-work study sessions
- mood and stress monitoring
- realtime analytics dashboards
- academic risk evaluation
- AI assistant integration
- smart notification delivery

The Scrum-based workflow played a major role in maintaining development organization and feature stability throughout the project lifecycle. The use of structured weekly sprints, backlog management, sprint reviews, and integration testing allowed the team to progressively implement and validate each subsystem without disrupting existing functionality.

One of the most important outcomes of the project was the successful implementation of realtime synchronization between frontend interfaces and backend services. Changes in tasks, study sessions, notifications, and analytics were reflected instantly through Supabase Realtime subscriptions and Riverpod provider invalidation workflows. This significantly improved responsiveness and overall user experience.

The analytics and academic risk systems also produced promising results during testing and demonstration scenarios. The platform successfully identified:

- overdue tasks
- study inconsistency patterns
- elevated stress conditions
- workload pressure
- GPA decline indicators
- burnout trends

The risk engine generated adaptive recommendations and intelligent notifications based on user behavior and academic data. These features demonstrated how academic monitoring systems can provide proactive decision support rather than simple task organization.

The AI assistant subsystem added an additional layer of interaction and usability to the platform. During evaluation scenarios, the assistant successfully generated:

- academic summaries
- study recommendations
- deadline reminders
- productivity suggestions
- semester insights
- risk explanations

The hybrid AI architecture combining local intent handling and cloud-based inference improved flexibility and reduced unnecessary external API requests.

From a UI/UX perspective, the project achieved a modern and responsive user experience across multiple platforms. The interface successfully integrated:

- responsive layouts
- animated transitions
- dashboard visualizations
- dark/light themes
- interactive charts
- notification interfaces
- accessibility-aware navigation

Testing results showed stable navigation flows and good usability during academic workflow simulations. The dashboard and analytics screens especially improved visibility of academic progress and workload trends.

The notification system produced strong practical results during realtime testing scenarios. Push notifications for deadlines, overdue tasks, academic risk warnings, and study reminders were successfully delivered even when the application was inactive. This demonstrated the reliability of the Firebase Cloud Messaging integration and the backend notification engine.

Figure 11.1. Realtime Smart Notifications Delivered by the Acadia Notification Engine

The project also highlighted several technical and organizational challenges during development. The most significant challenges included:

- realtime synchronization complexity
- provider dependency management
- AI integration stability
- notification cooldown optimization
- analytics recalculation efficiency
- cross-platform UI consistency
- timer persistence across app lifecycle states

These issues required multiple architecture refinements and optimization phases, especially during Sprint 11 and Sprint 12 stabilization stages.

Despite the successful implementation, the project still contains several limitations. The evaluation process was conducted within a relatively limited pilot scope and did not include long-term production-scale academic data from large university populations. Some AI recommendation behaviors and academic prediction mechanisms also require additional training data and broader testing scenarios to improve accuracy and personalization quality.

Another limitation is that certain advanced features were intentionally postponed to future development phases, including:

- collaborative study groups
- calendar synchronization with external platforms
- advanced predictive GPA forecasting
- multi-user academic comparison systems
- institutional administrator dashboards
- wearable device integration
- long-term academic behavior prediction

Even with these limitations, the project demonstrates that intelligent academic monitoring systems can significantly improve academic organization, productivity awareness, and behavioral insight generation for university students.

The final MVP successfully achieved the primary project objectives and established a scalable foundation for future AI-driven academic support systems.

12. Conclusion and Future Work

12.1. Conclusion

This graduation project successfully designed, implemented, and evaluated Acadia, a cross-platform academic productivity and academic risk monitoring system developed using Agile Scrum methodology and modern cloud-based technologies. The project covered the complete software engineering lifecycle, beginning with problem analysis and requirements gathering, followed by system modeling, architecture design, implementation, testing, evaluation, and MVP validation.

The final system demonstrates that intelligent academic monitoring platforms can provide practical support for university students by combining:

- academic organization
- realtime synchronization
- analytics processing
- stress and burnout monitoring
- AI-assisted recommendations
- smart notification systems

The project successfully integrated multiple complex subsystems into a unified and scalable architecture using Flutter, Riverpod, Supabase, PostgreSQL, Firebase Cloud Messaging, and AI-assisted backend services.

The implemented MVP provides students with tools for:

- semester management
- course tracking
- task and deadline organization
- study session monitoring
- GPA and analytics tracking
- academic risk evaluation
- mood and stress logging
- intelligent productivity recommendations

The realtime backend infrastructure, provider-based frontend architecture, and modular feature organization significantly improved maintainability, scalability, and feature integration throughout development. The project also demonstrated successful use of Scrum-based sprint

management for organizing collaborative software engineering workflows under constrained academic timelines.

Testing and evaluation results confirmed that the major functional and non-functional requirements were successfully satisfied. The system maintained stable performance during realtime synchronization, analytics recalculation, notification delivery, and AI-assisted interactions. The project also demonstrated successful cross-platform deployment support for Android, iOS, web, and desktop environments.

Another important contribution of the project is the implementation of explainable academic intelligence systems rather than relying entirely on opaque black-box AI models. The academic risk engine, recommendation engine, and notification workflows were designed using transparent scoring logic and structured analytics formulas to improve reliability and interpretability.

Overall, Acadia evolved beyond a basic university prototype into a production-oriented academic productivity platform with scalable architecture, intelligent monitoring capabilities, and extensible backend infrastructure. The project establishes a strong technical foundation for future research and advanced educational technology systems.

12.2. Future Work

Although the current MVP successfully achieves the primary objectives of the project, several improvements and advanced features can be implemented in future development phases.

One of the most important future improvements is the enhancement of predictive analytics and adaptive recommendation systems. The current academic risk engine already evaluates workload pressure, overdue tasks, stress escalation, and study consistency. Future versions could integrate more advanced machine learning models for:

- GPA prediction
- course failure prediction
- burnout forecasting
- adaptive workload balancing
- personalized semester planning

Additional AI improvements may include:

- conversational academic coaching
- long-term study behavior analysis
- adaptive scheduling systems
- intelligent exam preparation planning

- personalized productivity optimization

The notification engine can also be expanded with more advanced behavioral intelligence features such as:

- adaptive notification timing
- learning-based notification prioritization
- context-aware productivity recommendations
- calendar-aware academic reminders

Future versions of Acadia may also support collaborative and institutional features including:

- shared study groups
- collaborative task systems
- peer productivity comparison
- instructor dashboards
- university administrator analytics
- department-level academic monitoring systems

Several technical enhancements are also planned for future releases, including:

- offline-first synchronization
- advanced caching systems
- improved realtime scaling
- cloud function optimization
- improved battery-efficient background processing
- advanced analytics visualization systems

Cross-platform improvements may include:

- smartwatch integration
- tablet-optimized layouts
- desktop productivity workspaces
- wearable stress tracking integration

Another important future goal is broader deployment and evaluation using larger student populations across different universities and academic environments. Large-scale deployment would allow more accurate validation of:

- academic risk calculations
- burnout prediction accuracy
- notification effectiveness
- AI recommendation quality
- long-term productivity improvements

Future research may also compare academic outcomes between students using intelligent productivity systems and traditional academic planning methods.

The modular architecture developed in this project provides a strong foundation for future expansion and long-term system evolution. Acadia can continue developing into a comprehensive educational intelligence platform capable of supporting advanced academic analytics, institutional insights, and personalized student success systems.

REFERENCES

[1] K. Schwaber and J. Sutherland, “The Scrum Guide: The Definitive Guide to Scrum,” Scrum.org, 2020.

[2] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

[3] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner’s Approach, 9th ed. New York, NY, USA: McGraw-Hill, 2019.

[4] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.

[6] Google, “Flutter Documentation,” 2026. [Online]. Available: https://docs.flutter.dev

[7] Google, “Dart Documentation,” 2026. [Online]. Available: https://dart.dev

[8] Supabase Inc., “Supabase Documentation,” 2026. [Online]. Available: https://supabase.com/docs

[9] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2026. [Online]. Available: https://www.postgresql.org/docs

[10] Google, “Firebase Cloud Messaging Documentation,” 2026. [Online]. Available: https://firebase.google.com/docs/cloud-messaging

[11] Riverpod Contributors, “Riverpod Documentation,” 2026. [Online]. Available: https://riverpod.dev

[12] Flutter Community, “GoRouter Documentation,” 2026. [Online]. Available: https://pub.dev/packages/go_router

[13] Hive Contributors, “Hive Documentation,” 2026. [Online]. Available: https://docs.hivedb.dev

[14] GitHub Inc., “GitHub Documentation,” 2026. [Online]. Available: https://docs.github.com

[15] Notion Labs Inc., “Notion Documentation,” 2026. [Online]. Available: https://www.notion.so/help

[16] Google, “Android Developer Documentation,” 2026. [Online]. Available: https://developer.android.com/docs

[17] Apple Inc., “Apple Developer Documentation,” 2026. [Online]. Available: https://developer.apple.com/documentation

[18] M. L. Cohn, Agile Estimating and Planning. Upper Saddle River, NJ, USA: Pearson Education, 2005.

[19] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[20] OpenAI, “Artificial Intelligence and Natural Language Processing Technologies,” 2026. [Online]. Available: https://openai.com

[21] NVIDIA Corporation, “NVIDIA NIM Documentation,” 2026. [Online]. Available: https://docs.nvidia.com/nim

[22] Meta Platforms Inc., “Llama 3.1 Documentation,” 2026. [Online]. Available: https://ai.meta.com

[23] IEEE Standards Association, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998.

[24] ISO/IEC/IEEE 12207:2017, Systems and Software Engineering — Software Life Cycle Processes.

[25] Acadia Development Team, “Acadia Academic Productivity and Academic Risk Monitoring System,” Graduation Project Repository, OSTİM Teknik University, 2026.

[26] Acadia Development Team, “Acadia GitHub Repository,” GitHub, 2026. [Online]. Available: https://github.com/NouradinAbdurahman/Acadia

[27] Acadia Development Team, “Acadia Scrum Sprint Planning and Project Management Workspace,” Notion, 2026. [Online]. Available: https://steep-part-a5a.notion.site/Acadia-Project-Management-Plan-32a9d63b261182c98d1401ea99495daa

[28] Acadia Development Team, “Acadia Official Website,” 2026. [Online]. Available: https://acadia-landing.vercel.app

[29] Acadia Development Team, “Acadia Study Planner,” Apple App Store, 2026. [Online]. Available: https://apps.apple.com/tr/app/acadia-study-planner/id6761367583

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name: NOURADDIN ABDURAHMAN ADEN

Nationality: Somali

Place and Date of Birth: Mogadishu-12/08/2001

E-mail: n.aden1208@gmail.com

EDUCATION

Degree	Institution	Graduation Year
High School:	Omar bin Abdul Aziz Secondary Independent School for Boys, Doha, Qatar	Jun 2021
Undergraduate:	OSTİM Teknik Üniversitesi	Jun 2026

PROFESSIONAL EXPERIENCE

Institution	Year
Full-Stack Developer DAKAEi AI: Acadia (Graduation Project)	04/2025 - Present 2026

AREA OF EXPERTISE

Field
Software Development
Team Collaboration

FOREIGN LANGUAGES

Language	Proficiency Level
English:	Fluent
Arabic:	Native
Somali:	Native

CERTIFICATES

Certificate Area	Issuing Institution	Year
Associate AI Engineer for Data Scientists	DataCamp	Jan 2025
Associate Data Engineer in SQL	DataCamp	Jan 2025

OTHER RELEVANT QUALIFICATIONS

PUBLICATIONS

Publication 1 None

Publication 2 None

PRESENTATIONS

Presentation 1 None

Presentation 2 None

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name: AHMED ADEEB AHMED ABBAD

Nationality: YEMEN

Place and Date of Birth: RIYADH - 11/10/2002

E-mail: 210208817@ostimteknk.edu.tr

EDUCATION

Degree	Institution	Graduation Year
High School:	KA'AB BIN RABIAH	2020
Undergraduate:	OSTİM Teknik Üniversitesi	2026

PROFESSIONAL EXPERIENCE

Institution	Year
DAKAEi AI:	2026
Acadia (Graduation Project)	2026

AREA OF EXPERTISE

Field
Software Development
Team Collaboration

FOREIGN LANGUAGES

Language	Proficiency Level
English:	Advanced
Arabic:	Native

CERTIFICATES

Certificate Area	Issuing Institution	Year
Python Essentials 1	Cisco	2025
Associate Data Analyst in SQL	DataCamp	2024
Data Scientist in Python	DataCamp	2024

OTHER RELEVANT QUALIFICATIONS

PUBLICATIONS

Publication 1 None

Publication 2 None

PRESENTATIONS

Presentation 1 None

Presentation 2 None

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name: Seddik Mohey
Nationality: Egypt
Place and Date of Birth: Egypt-20/08/2003
E-mail: sadekmhoy7@gmail.com

EDUCATION

Degree	Institution	Graduation Year
High School:	Bani Ahmad	2021
Undergraduate:	OSTİM Teknik Üniversitesi	2026

PROFESSIONAL EXPERIENCE

Institution	Year
BLC	2026
Acadia (Graduation Project)	2026

AREA OF EXPERTISE

Field
Ai Engineering
Team Collaboration

FOREIGN LANGUAGES

Language	Proficiency Level
English:	Advanced
Arabic:	Native

CERTIFICATES

Certificate Area	Issuing Institution	Year
Machine Learning Specialization	Coursera	2026
Aspire Leaders Program	Aspire Institute	2025
Introduction to Analytic Thinking, Data Science, and Data Mining	Coursera	2025

OTHER RELEVANT QUALIFICATIONS

PUBLICATIONS

Publication 1 None
Publication 2 None

PRESENTATIONS

Presentation 1 None
Presentation 2 None

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name: AHMED ALMATHLOUM
Nationality: Palestinian
Place and Date of Birth: Saudi Arabia 19/07/2002
E-mail: akm16790@gmail.com

EDUCATION

Degree	Institution	Graduation Year
High School:	Kaab bin rabiaa	2020
Undergraduate:	OSTİM Teknik Üniversitesi	2026

PROFESSIONAL EXPERIENCE

Institution	Year
Quiz master (personal project)	2026
Acadia (Graduation Project)	2026

AREA OF EXPERTISE

Field
Full stack development
Team Collaboration

FOREIGN LANGUAGES

Language	Proficiency Level
English:	Advanced
Arabic:	Native

CERTIFICATES

Certificate Area	Issuing Institution	Year
Associate Data Analyst in SQL	Data Camp	2024
Cyber security	Cisco	2025

OTHER RELEVANT QUALIFICATIONS

PUBLICATIONS

Publication 1 None
Publication 2 None

PRESENTATIONS

Presentation 1 None
Presentation 2 None

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name: MOHSEN NAEF QAHTAN
Nationality: YEMEN
Place and Date of Birth: UAE, Ajman – 2002
E-mail: 210208968@ostimteknik.edu.tr

EDUCATION

Degree	Institution	Graduation Year
High School:	IBN KHALDUN HIGH SCHOOL	2019
Undergraduate:	OSTİM Teknik Üniversitesi	2026

PROFESSIONAL EXPERIENCE

Institution	Year
Full-Stack E-Commerce Platform	2025
Acadia (Graduation Project)	2026

AREA OF EXPERTISE

Field
Full stack development
Team Collaboration

FOREIGN LANGUAGES

Language	Proficiency Level
English:	Advanced
Arabic:	Native

CERTIFICATES

Certificate Area	Issuing Institution	Year
Associate Data Analyst in SQL	Data Camp	2024
Cyber security	Cisco	2025

OTHER RELEVANT QUALIFICATIONS

PUBLICATIONS

Publication 1 None
Publication 2 None

PRESENTATIONS

Presentation 1 None
Presentation 2 None

13. Appendices

13.1. Appendix A: Source Code

This appendix contains representative source code excerpts and implementation evidence from the Acadia system. The included code samples demonstrate the architecture, backend integration, realtime synchronization, analytics processing, authentication workflow, and intelligent notification systems implemented during development.

The appendix should include selected code examples from the following major modules:

Authentication System

This appendix presents representative source code from the Acadia application's authentication module. The selected excerpts demonstrate the layered architecture, Supabase backend integration, session management, protected routing, and security mechanisms implemented during development.

A.1 — Supabase Auth Service

File: lib/services/supabase/auth_service.dart

Purpose: The AuthService class forms the lowest layer of the authentication stack. It acts as a direct adapter to the Supabase Auth SDK, exposing only the operations needed by upper layers. All Supabase calls are isolated here — no other layer imports the SDK directly.

```dart
class AuthService {
  final SupabaseClient _supabase;

  AuthService(this._supabase);

  // Exposes the currently authenticated user from the Supabase session
  User? get currentUser => _supabase.auth.currentUser;

  // Reactive stream of auth state changes — used by Riverpod providers
  Stream<AuthState> get authStateChanges =>
_supabase.auth.onAuthStateChange;
  Future<AuthResponse> login(String email, String password) async {
    return await _supabase.auth.signInWithPassword(
      email: email,
      password: password,
    );
  }

  Future<AuthResponse> signUp(String email, String password) async {
    return await _supabase.auth.signUp(email: email, password: password);
  }

  Future<void> logout() async {
    await _supabase.auth.signOut();
  }

  Future<void> sendPasswordResetOTP(String email) async {
    await _supabase.auth.resetPasswordForEmail(email);
  }

  Future<AuthResponse> verifyPasswordResetOTP({
    required String email,
    required String code,
  }) async {
    return await _supabase.auth.verifyOTP(
      email: email,
      token: code,
      type: OtpType.recovery,
    );
  }

  Future<UserResponse> updatePasswordAfterOTP({
    required String email,
    String? sessionToken,
    required String newPassword,
  }) async {
    return await _supabase.auth.updateUser(
      UserAttributes(password: newPassword),
    );
  }

  Future<AuthResponse> verifyOTP(
      String email, String token, OtpType type) async {
    return await _supabase.auth.verifyOTP(
      email: email,
      token: token,
      type: type,
    );
  }

  Future<void> resendOTP(String email,
      {OtpType type = OtpType.signup}) async {
    await _supabase.auth.resend(type: type, email: email);
  }

  Future<Map<String, dynamic>?> getUserProfile(String userId) async {
    return await _supabase
        .from('users')
        .select()
        .eq('id', userId)
        .maybeSingle();
  }
}
```

A.2 — Authentication Repository

File: lib/repositories/auth_repository.dart

Purpose: The AuthRepository sits above AuthService and adds application-level concerns: rate limiting with exponential backoff, OAuth provider support, and database consistency guarantees. It is the only layer that the UI and providers communicate with for authentication.

```
Rate Limiting with Exponential Backoff
To prevent abuse and handle Supabase email-send limits, the repository tracks failed attempts and
enforces progressively longer cooldowns. Timestamps are persisted to Hive so limits survive app
restarts.
/// Marks a rate-limit event and applies exponential backoff (60s → 300s max).
void _markRateLimit({required bool isEmail}) {
  final now = DateTime.now();
  if (isEmail) {
    _emailRateLimitCount++;
    final cooldownSecs = (60 * _emailRateLimitCount).clamp(60, 300);
    _lastEmailRateLimit = now.add(Duration(seconds: cooldownSecs));
  } else {
    _authRateLimitCount++;
    final cooldownSecs = (60 * _authRateLimitCount).clamp(60, 300);
    _lastAuthRateLimit = now.add(Duration(seconds: cooldownSecs));
  }
  _saveLimits();
}
Login with Rate Limit Enforcement
Before forwarding to Supabase, the repository checks whether the user is currently rate-limited. On
success, counters are reset. On failure, the appropriate rate-limit type is recorded.
Future<AuthResponse> login(String email, String password) async {
  if (isAuthRateLimited()) {
    throw AuthApiException(
      'Please wait before trying to login again.',
      code: 'too_many_requests',
    );
  }

  try {
    final response = await _supabase.auth
        .signInWithPassword(email: email, password: password)
        .timeout(const Duration(seconds: 45));

    // Reset counters on successful login
    _authRateLimitCount = 0;
    _lastAuthRateLimit = null;
    _saveLimits();

    return response;
  } on AuthApiException catch (e) {
    final msg = e.message.toLowerCase();
    if (e.code == 'over_email_send_rate_limit' || msg.contains('rate limit')) {
      _markRateLimit(isEmail: true);
    } else if (e.code == 'too_many_requests' || msg.contains('too many requests')) {
      _markRateLimit(isEmail: false);
    }
    rethrow;
  }
}
Sign-Up with OTP Verification
After the OTP is confirmed, the repository creates both a users record (used for routing guards) and a
student_profiles record (used for academic identity). This ensures database consistency before the user
enters the app.
Future<AuthResponse> verifyOTP(
    String email, String token, String fullName) async {
  final response = await _supabase.auth
      .verifyOTP(email: email, token: token, type: OtpType.signup)
      .timeout(const Duration(seconds: 20));

  if (response.user != null) {
    // Create the app-level user record used by routing guards
    await _supabase.from('users').upsert({
      'id': response.user!.id,
      'academic_profile_completed': false,
    }, onConflict: 'id');

    // Create the academic profile record via the service layer
    await _studentProfileService.upsertInitialProfile(
      userId: response.user!.id,
      fullName: fullName,
    );
  }

  return response;
}
OAuth Sign-In (Google and Apple)
OAuth is handled via Supabase's PKCE flow. The app opens an external browser to avoid in-app webview
issues and listens for the redirect URL via authStateChanges.
Future<bool> signInWithOAuth(OAuthProvider provider) async {
  return _supabase.auth.signInWithOAuth(
    provider,
    redirectTo: oauthRedirectForCurrentPlatform(),
    authScreenLaunchMode: LaunchMode.externalApplication,
    queryParams: const {'access_type': 'offline', 'prompt': 'consent'},
  );
}
Ensuring Database Row Consistency After Sign-In
This method is called on every AuthChangeEvent.signedIn. It guarantees that both the users and
student_profiles rows exist, and backfills empty display names from OAuth metadata when necessary.
Future<void> ensurePublicUserRowIfMissing() async {
  final user = _supabase.auth.currentUser;
  if (user == null) return;

  final oauthName = _extractDisplayName(user);

  // Step 1: Ensure users row exists
  final existing = await _supabase
      .from('users')
      .select('id, name')
      .eq('id', user.id)
      .maybeSingle();

  if (existing == null) {
    await _supabase.from('users').insert({
      'id' : user.id,
      'email': user.email,
      'name': oauthName.isNotEmpty ? oauthName : user.email,
      'academic_profile_completed': false,
    });
  } else if ((existing['name'] as String? ?? '').trim().isEmpty &&
      oauthName.isNotEmpty) {
    // Backfill empty name from OAuth provider metadata
    await _supabase
        .from('users')
        .update({'name': oauthName})
        .eq('id', user.id);
  }

  // Step 2: Ensure student_profiles row exists
  final existingProfile = await _supabase
      .from('student_profiles')
      .select('user_id')
      .eq('user_id', user.id)
      .maybeSingle();

  if (existingProfile == null) {
    await _studentProfileService.upsertInitialProfile(
      userId: user.id,
      fullName: oauthName,
    );
  }
}
```

A.3 — Riverpod Authentication Providers

File: lib/providers/app_providers.dart

Purpose: Riverpod providers form the bridge between the data layer and the UI. The auth providers follow a layered dependency chain: SupabaseClient → AuthService → AuthRepository → reactive state streams → app-level user model.

```dart
// Layer 1: Infrastructure — wraps the Supabase singleton
final supabaseProvider = Provider<SupabaseClient>(
  (ref) => Supabase.instance.client,
);

// Layer 2: Service — thin Supabase SDK adapter
final authServiceProvider = Provider<AuthService>(
  (ref) => AuthService(ref.watch(supabaseProvider)),
);

// Layer 3: Repository — business logic, rate limiting, consistency
final authRepositoryProvider = Provider<AuthRepository>(
  (ref) => AuthRepository(ref.watch(authServiceProvider)),
);

// Reactive stream of Supabase auth events (login, logout, token refresh)
final authStateProvider = StreamProvider<AuthState>((ref) {
  return ref.watch(authRepositoryProvider).authStateChanges;
});

// Current authenticated Supabase user (falls back to cached session)
final currentUserProvider = Provider<User?>((ref) {
  final authState = ref.watch(authStateProvider);
  return authState.value?.session?.user ??
      Supabase.instance.client.auth.currentUser;
});

// App-level user model loaded from the Supabase `users` table.
// Used by the router to enforce the academic-profile completion guard.
final currentAppUserProvider = FutureProvider<AppUser?>((ref) async {
  final user = ref.watch(currentUserProvider);
  if (user == null) return null;

  final authRepo = ref.watch(authRepositoryProvider);
  return await authRepo.getUserProfile(user.id).timeout(
    const Duration(seconds: 25),
    // Returns null on timeout so the splash screen does not hang indefinitely
    onTimeout: () => null,
  );
});
```

A.4 — Protected Routing with Auth Guards

File: lib/main.dart — routerProvider

Purpose: All navigation protection is centralised in a single redirect function inside routerProvider. It evaluates six ordered guards on every route change, covering unauthenticated access, incomplete onboarding, missing academic profile, and missing active semester. The router re-evaluates automatically whenever the Supabase auth state changes.

```dart
// Router re-evaluates on every Supabase auth state change
refreshListenable: Listenable.merge([
  GoRouterRefreshStream(Supabase.instance.client.auth.onAuthStateChange),
]),

redirect: (context, state) {
  final loggedIn = Supabase.instance.client.auth.currentSession != null;

  // Guard 1: Wait for Hive-backed settings before making routing decisions
  if (!localeLoaded || !onboardingLoaded || !themeLoaded) {
    return isSplashScreen ? null : '/splash';
  }

  // Guard 2: Unauthenticated — enforce Language → Onboarding → Login flow
  if (!loggedIn) {
    if (!languageSelected && !isLanguageScreen) return '/language';
    if (languageSelected && !onboardingCompleted && !isOnboardingScreen)
      return '/onboarding';
    if (languageSelected && onboardingCompleted && !isAuthScreen)
      return '/login';
    return null;
  }

  // Guard 3: Authenticated but app-user profile still loading from Supabase
  if (appUserAsync.isLoading) return isSplashScreen ? null : '/splash';

  final appUser = appUserAsync.asData?.value;

  // Guard 4: Academic profile not yet completed — force onboarding wizard
  if (appUser == null || !appUser.academicProfileCompleted) {
    if (isAcademicOnboardingScreen) return null;
    return '/academic-onboarding';
  }

  // Guard 5: No active semester — force first semester creation
  if (activeSemesterAsync.asData?.value == null &&
      !isOnboardingSemesterScreen) {
    return '/onboarding-semester';
  }

  // Guard 6: All guards passed — redirect auth/splash/onboarding screens to home
  if (isSplashScreen || isAuthScreen || isLanguageScreen || isOnboardingScreen) {
    return '/home';
  }

  return null;
},
```

A.5 — Login Screen

File: lib/screens/auth/login_screen.dart

Purpose: The login screen handles credential validation, error classification, remember-me persistence, and OAuth sign-in. It consumes authRepositoryProvider via Riverpod and navigates using GoRouter. The live countdown timer in the button label gives users real-time feedback when a rate limit is active.

```
Future<void> _login() async {
  if (_isLoading) return;
  if (!_formKey.currentState!.validate()) return;

  setState(() { _isLoading = true; _errorMessage = null; });

  try {
    final authRepo = ref.read(authRepositoryProvider);
    final response = await authRepo.login(
      _emailController.text.trim(),
      _passwordController.text.trim(),
    );

    // Redirect to email verification if account is unconfirmed
    if (response.user != null && response.user!.emailConfirmedAt == null) {
      context.go(
        '/verify-email?email=${Uri.encodeComponent(_emailController.text.trim())}',
      );
      return;
    }

    // Persist remember-me credentials securely in Hive local storage
    final box = await Hive.openBox('settings');
    if (_rememberMe) {
      await box.put('remembered_email', _emailController.text.trim());
      await box.put('remembered_password', _passwordController.text.trim());
    } else {
      await box.delete('remembered_email');
      await box.delete('remembered_password');
    }
  } on AuthApiException catch (e) {
    final lower = e.message.toLowerCase();

    // Unconfirmed email — redirect silently rather than showing an error
    if (e.code == 'email_not_confirmed') {
      context.go(
        '/verify-email?email=${Uri.encodeComponent(_emailController.text.trim())}',
      );
      return;
    }

    // Classify the Supabase error code into a user-facing l10n message
    if (e.code == 'invalid_credentials' ||
        lower.contains('invalid login credentials')) {
      message = l10n?.invalidCredentials ?? '';
    } else if (e.code == 'over_email_send_rate_limit') {
      message = l10n?.emailRateLimit ?? '';
    } else if (e.code == 'too_many_requests' ||
        lower.contains('too many requests')) {
      message = l10n?.tooManyRequests ?? '';
    } else {
      message = l10n?.networkError ?? '';
    }

    setState(() => _errorMessage = message);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}
```

A.6 — Sign-Up Screen

File: lib/screens/auth/signup_screen.dart

Purpose: The sign-up screen collects the user's name, email, and password. It enforces a client-side signup cooldown in addition to server-side rate limiting, detects existing-email registrations via Supabase's identity enumeration protection, and routes the user to OTP verification on success.

```dart
Future<void> _signUp() async {
  final authRepo = ref.read(authRepositoryProvider);

  // Check client-side rate limit before making a network request
  if (authRepo.isBlockedForEmail()) {
    final remaining = authRepo.remainingEmailRateLimitTime();
    final timeStr =
        '${remaining.inMinutes}:${(remaining.inSeconds % 60).toString().padLeft(2, "0")}';
    setState(() => _errorMessage = l10n?.emailRateLimitWithTimer(timeStr) ?? '');
    return;
  }

  if (!authRepo.canSignup()) {
    setState(() => _errorMessage = l10n?.pleaseWait ?? '');
    return;
  }

  if (!_formKey.currentState!.validate()) return;

  setState(() { _isLoading = true; _errorMessage = null; });

  try {
    final response = await authRepo.signUp(
      _emailController.text.trim(),
      _passwordController.text.trim(),
      _nameController.text.trim(),
    );

    // Supabase returns an empty identities list for duplicate emails
    // when enumeration protection is enabled — detect it explicitly
    final user = response.user;
    if (user != null && user.identities != null && user.identities!.isEmpty) {
      setState(() => _errorMessage = l10n?.emailAlreadyInUse ?? '');
      return;
    }

    // Navigate to OTP verification, passing name for profile creation
    context.push(
      '/verify-email'
      '?email=${Uri.encodeComponent(_emailController.text.trim())}'
      '&fullName=${Uri.encodeComponent(_nameController.text.trim())}',
    );
  } on AuthApiException catch (e) {
    final lower = e.message.toLowerCase();
    if (e.code == 'over_email_send_rate_limit' || lower.contains('rate limit')) {
      msg = l10n?.emailRateLimit ?? '';
    } else if (lower.contains('already registered') ||
        e.code == 'user_already_exists') {
      msg = l10n?.emailAlreadyInUse ?? '';
    } else if (e.code == 'weak_password') {
      msg = l10n?.passwordTooShort ?? '';
    }
    setState(() => _errorMessage = msg);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}
```

A.7 — OTP Email Verification

File: lib/screens/auth/verify_email_screen.dart

Purpose: After sign-up, the user must verify their email using a six-digit OTP sent by Supabase. This screen renders a PIN input field and auto-submits when the sixth digit is entered. On success, the repository creates the user's database records and the router redirects to onboarding.

```dart
Future<void> _verifyOtp(String code) async {
  if (code.length != 6) return;

  setState(() { _isLoading = true; _errorMessage = null; });

  try {
    final authRepo = ref.read(authRepositoryProvider);
    // Repository verifies OTP, creates users + student_profiles rows on success
    await authRepo.verifyOTP(widget.email, code, widget.fullName);

    if (mounted) context.go('/onboarding');
  } catch (e) {
    if (e is AuthApiException) {
      if (e.message.toLowerCase().contains('invalid') ||
          e.code == 'invalid_otp') {
        message = l10n?.invalidOtp ?? 'Invalid code, please try again.';
      } else if (e.code == 'otp_expired' ||
          e.message.toLowerCase().contains('expired')) {
        message = l10n?.otpExpired ?? 'Code expired. Please request a new one.';
      } else if (e.code == 'over_email_send_rate_limit') {
        message = l10n?.emailRateLimit ?? 'Too many attempts. Please wait.';
      }
    }
    setState(() => _errorMessage = message);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}

Future<void> _resendCode() async {
  // Enforce client-side rate limit before resending
  if (authRepo.isBlockedForEmail()) {
    final remaining = authRepo.remainingEmailRateLimitTime();
    setState(() {
      _errorMessage = l10n?.emailRateLimitWithTimer(
        '${remaining.inMinutes}:${(remaining.inSeconds % 60).toString().padLeft(2, "0")}',
      );
    });
    return;
  }

  await authRepo.resendOTP(widget.email);

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(l10n?.codeResent ?? 'Code sent! Check your inbox.')),
  );
}
```

A.8 — Password Reset Flow (Three Steps)

Files: forgot_password_otp_screen.dart, forgot_password_code_screen.dart, reset_password_screen.dart

Purpose: Password reset is implemented as a three-step OTP flow using Supabase's recovery token mechanism. The flow is: (1) user enters email and receives a one-time code, (2) code is verified and a session token is returned, (3) user sets a new password using that session.

```
Step 1 — Request OTP:
Future<void> _sendOTP() async {
  if (authRepo.isBlockedForEmail()) {
    final timeStr = '${remaining.inMinutes}:${(remaining.inSeconds % 60).toString().padLeft(2, "0")}';
    setState(() => _error = l10n?.emailRateLimitWithTimer(timeStr));
    return;
  }

  final email = _emailController.text.trim();
  await authRepo.sendPasswordResetOTP(email);

  // Proceed to the OTP code entry screen
  context.push('/forgot-password-code?email=${Uri.encodeComponent(email)}');
}


Step 2 — Verify Recovery OTP:
Future<Map<String, dynamic>> verifyPasswordResetOTP({
  required String email,
  required String code,
}) async {
  final response = await _supabase.auth.verifyOTP(
    email: email,
    token: code,
    type: OtpType.recovery,  // Recovery type establishes a password-reset session
  );
  return {
    'success': response.session != null,
    'session_token': response.session?.accessToken,
  };
}
Step 3 — Set New Password:
Future<void> _updatePassword() async {
  if (!_formKey.currentState!.validate()) return;

  setState(() { _isLoading = true; _errorMessage = null; });

  try {
    final result = await authRepo.updatePasswordAfterOTP(
      email: widget.email,
      sessionToken: widget.sessionToken,
      newPassword: _passwordController.text.trim(),
    );

    if (result['success'] == true) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(l10n?.resetPassword ?? '')),
      );
      context.go('/login');
    }
  } on AuthApiException catch (e) {
    if (e.code == 'same_password') {
      message = l10n?.passwordSameAsOld ?? '';
    } else if (e.code == 'weak_password') {
      message = l10n?.passwordTooShort ?? '';
    } else {
      message = l10n?.networkError ?? '';
    }
    setState(() => _errorMessage = message);
  } finally {
    if (mounted) setState(() => _isLoading = false);
  }
}
```

A.9 — Password Validation Logic

File: lib/screens/auth/password_validation.dart

Purpose: Password validation is extracted into a standalone utility to ensure consistent rules are applied on both the sign-up and password-reset screens. The strength indicator provides real-time visual feedback during typing.

```dart
/// Returns a validation error if the password is under 8 characters.
String? validatePasswordForSignup(String? value, AppLocalizations? l10n) {
  if (value == null || value.length < 8) return l10n?.passwordTooShort ?? '';
  return null;
}

/// Returns a validation error if the confirmation does not match.
String? validateConfirmPasswordForSignup(
  String? value,
  String password,
  AppLocalizations? l10n,
) {
  if (value != password) return l10n?.passwordsDoNotMatch ?? '';
  return null;
}

/// Returns a localised strength label based on password length.
String passwordStrengthLabel(String password, AppLocalizations? l10n) {
  if (password.length >= 8) return l10n?.passwordStrong ?? '';
  if (password.length >= 5) return l10n?.passwordFair ?? '';
  return l10n?.passwordWeak ?? '';
}

/// Returns a colour token corresponding to the strength level.
Color passwordStrengthColor(ThemeData theme, String password) {
  if (password.length >= 8) return theme.colorScheme.primary; // Strong — green
  if (password.length >= 5) return Colors.orange;             // Fair   — orange
  return theme.colorScheme.error;                             // Weak   — red
}
```

Figure A.1. Supabase Authentication Integration

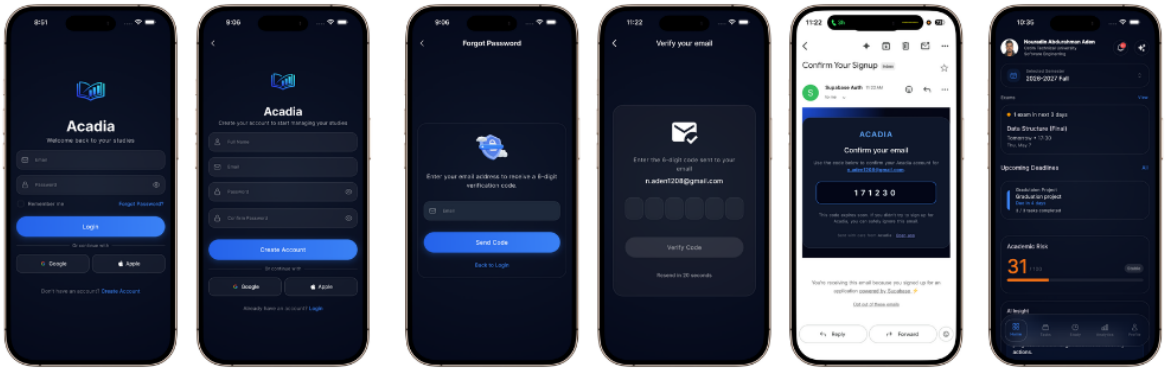
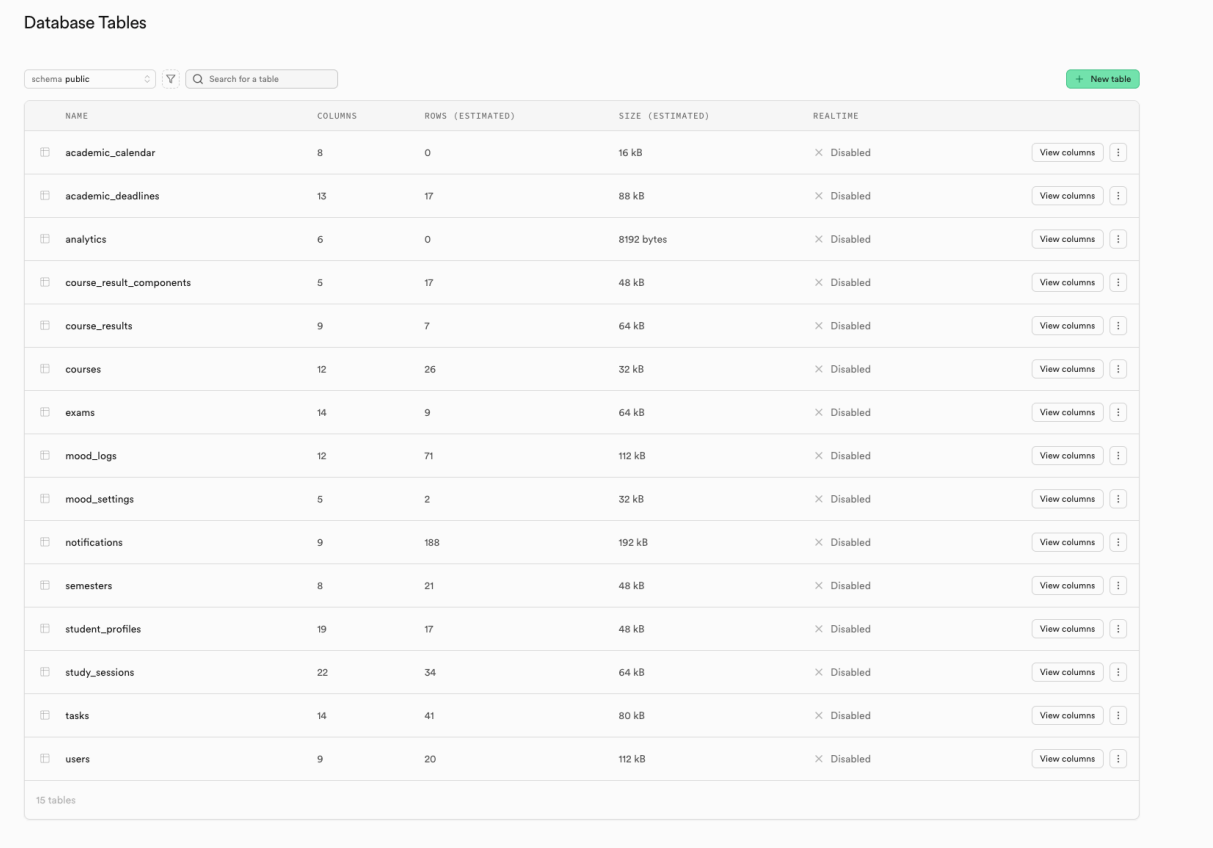
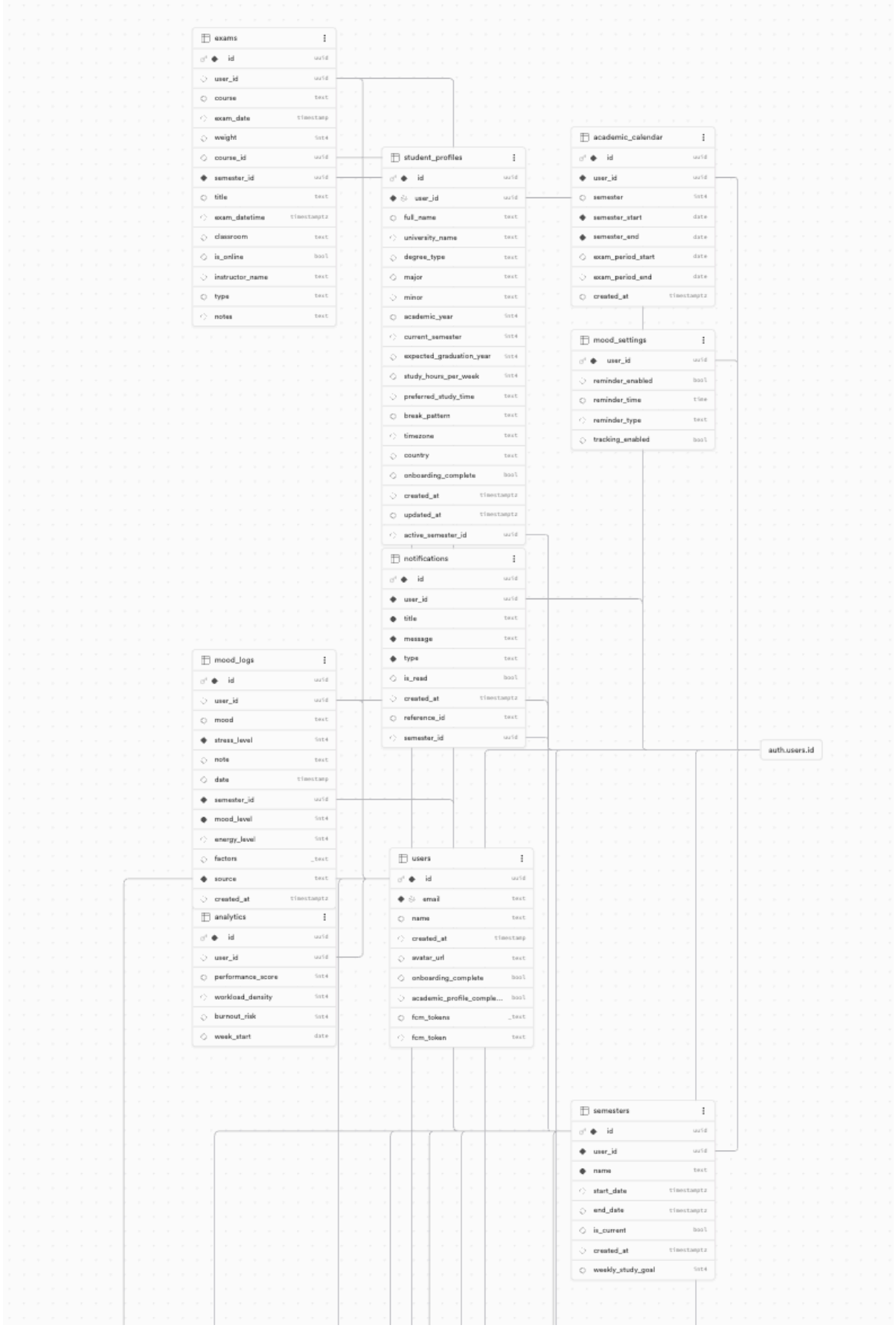
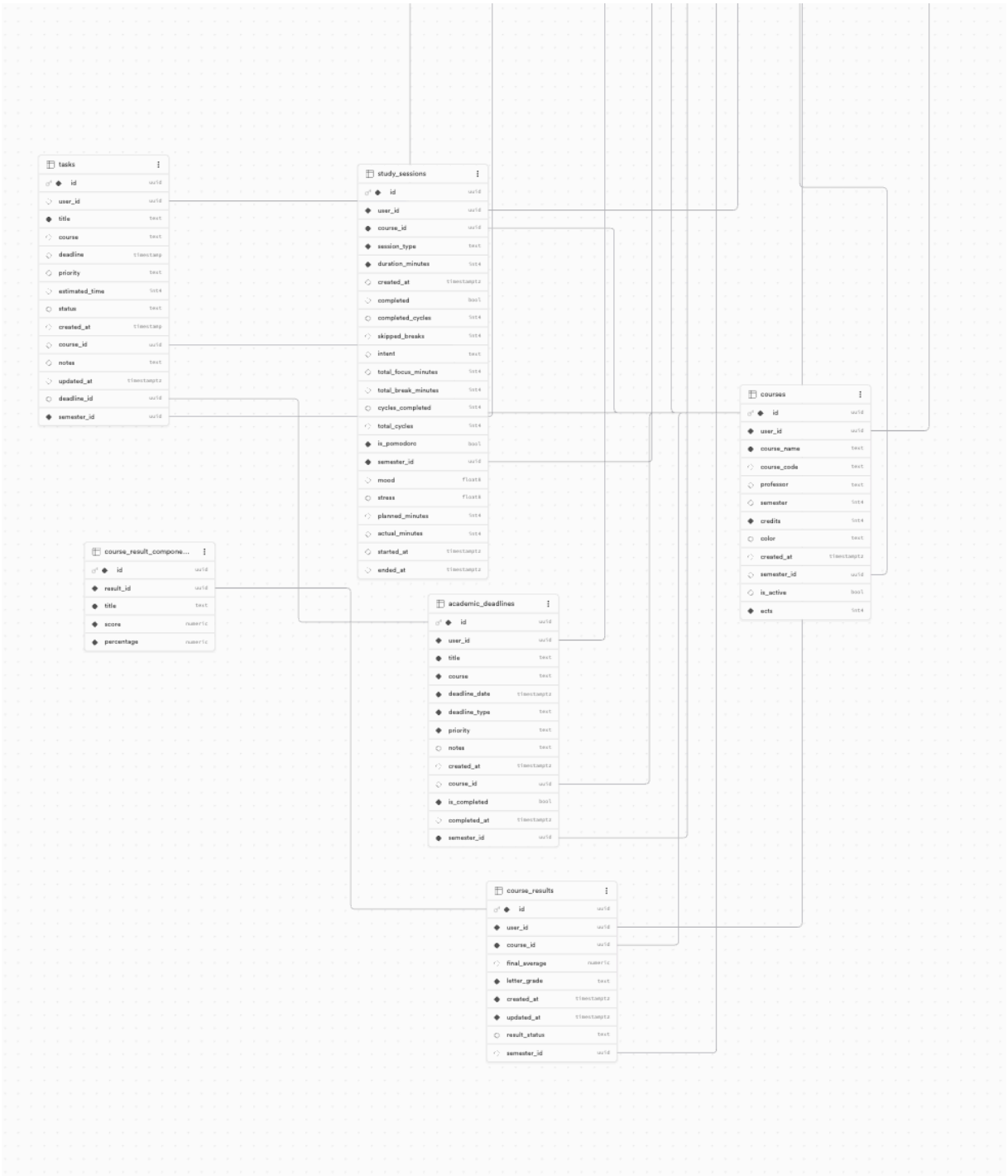


Figure A.2. Database Schema and Supabase Tables




exams
id uuid
user_id uuid
course text
exam_date timestamp
weight int4
course_id uuid
semester_id uuid
title text
exam_datetime timestamptz
classroom text
is_online bool
instructor_name text
type text
notes text
student_profiles
id uuid
user_id uuid
full_name text
university_name text
degree_type text
major text
minor text
academic_year int4
current_semester int4
expected_graduation_year int4
study_hours_per_week int4
preferred_study_time text
break_pattern text
timezone text
country text
onboarding_complete bool
created_at timestamptz
updated_at timestamptz
active_semester_id uuid
academic_calendar
id uuid
user_id uuid
semester int4
semester_start date
semester_end date
exam_period_start date
exam_period_end date
created_at timestamptz
mood_settings
user_id uuid
reminder_enabled bool
reminder_time time
reminder_type text
tracking_enabled bool
notifications
id uuid
user_id uuid
title text
message text
type text
is_read bool
created_at timestamptz
reference_id text
semester_id uuid
mood_logs
id uuid
user_id uuid
mood text
stress_level int4
note text
date timestamp
semester_id uuid
mood_level int4
energy_level int4
factors _text
source text
created_at timestamptz
analytics
id uuid
user_id uuid
performance_score int4
workload_density int4
burnout_risk int4
week_start date
users
id uuid
email text
name text
created_at timestamp
avatar_url text
onboarding_complete bool
academic_profile_comple... bool
fcm_tokens _text
fcm_token text
auth.users.id
semesters
id uuid
user_id uuid
name text
start_date timestamptz
end_date timestamptz
is_current bool
created_at timestamptz
weekly_study_goal int4


tasks
id uuid
user_id uuid
title text
course text
deadline timestamp
priority text
estimated_time int4
status text
created_at timestamp
course_id uuid
notes text
updated_at timestamptz
deadline_id uuid
semester_id uuid
study_sessions
id uuid
user_id uuid
course_id uuid
session_type text
duration_minutes int4
created_at timestamptz
completed bool
completed_cycles int4
skipped_breaks int4
intent text
total_focus_minutes int4
total_break_minutes int4
cycles_completed int4
total_cycles int4
is_pomodoro bool
semester_id uuid
mood float8
stress float8
planned_minutes int4
actual_minutes int4
started_at timestamptz
ended_at timestamptz
courses
id uuid
user_id uuid
course_name text
course_code text
professor text
semester int4
credits int4
color text
created_at timestamptz
semester_id uuid
is_active bool
ects int4
course_result_compone...
id uuid
result_id uuid
title text
score numeric
percentage numeric
academic_deadlines
id uuid
user_id uuid
title text
course text
deadline_date timestamptz
deadline_type text
priority text
notes text
created_at timestamptz
course_id uuid
is_completed bool
completed_at timestamptz
semester_id uuid
course_results
id uuid
user_id uuid
course_id uuid
final_average numeric
letter_grade text
created_at timestamptz
updated_at timestamptz
result_status text
semester_id uuid

Academic Management System

This appendix presents representative source code from Acadia's academic management module. The excerpts cover semester lifecycle management, course CRUD operations, task and deadline systems, and the Supabase Realtime synchronisation layer that keeps all data in sync across sessions.

B.1 — Semester Domain Model

File: lib/models/semester.dart

Purpose: The Semester model is the foundational data structure of the entire application. Every piece of academic data — courses, tasks, deadlines, study sessions — is scoped to a semester ID. The model implements fromJson / toJson for clean serialisation to and from Supabase's Postgres tables.

```dart
class Semester {
  final String id;
  final String userId;
  final String name;
  final DateTime? startDate;
  final DateTime? endDate;
  final int? weeklyStudyGoal;  // Minutes per week the student aims to study
  final bool isCurrent;
  final DateTime createdAt;

  Semester({
    required this.id,
    required this.userId,
    required this.name,
    this.startDate,
    this.endDate,
    this.weeklyStudyGoal,
    this.isCurrent = false,
    required this.createdAt,
  });

  factory Semester.fromJson(Map<String, dynamic> json) {
    return Semester(
      id: json['id'],
      userId: json['user_id'],
      name: json['name'],
      startDate: json['start_date'] != null
          ? DateTime.parse(json['start_date'])
          : null,
      endDate: json['end_date'] != null
          ? DateTime.parse(json['end_date'])
          : null,
      weeklyStudyGoal: json['weekly_study_goal'],
      isCurrent: json['is_current'] ?? false,
      createdAt: DateTime.parse(json['created_at']),
    );
  }

  Map<String, dynamic> toJson() {
    final data = {
      'user_id': userId,
      'name': name,
      'start_date': startDate?.toIso8601String(),
      'end_date': endDate?.toIso8601String(),
      'weekly_study_goal': weeklyStudyGoal,
      'is_current': isCurrent,
      'created_at': createdAt.toIso8601String(),
    };
    // Exclude id on creation — Postgres generates it via UUID default
    if (id.isNotEmpty) data['id'] = id;
    return data;
  }

  Semester copyWith({ String? id, String? name, DateTime? startDate,
      DateTime? endDate, int? weeklyStudyGoal, bool? isCurrent, ... }) {
    return Semester(
      id: id ?? this.id,
      name: name ?? this.name,
      startDate: startDate ?? this.startDate,
      endDate: endDate ?? this.endDate,
      weeklyStudyGoal: weeklyStudyGoal ?? this.weeklyStudyGoal,
      isCurrent: isCurrent ?? this.isCurrent,
      ...
    );
  }
}
```

B.2 — Semester Service and Repository

Files: lib/services/supabase/semester_service.dart · lib/repositories/semester_repository.dart

Purpose: The SemesterService handles all raw Supabase queries for the semesters table. The SemesterRepository sits above it, converting raw JSON maps into typed Semester domain models before they reach the provider layer. No other layer interacts with raw maps directly.

```
SemesterService — Supabase CRUD:
class SemesterService {
  final SupabaseClient _supabase;

  SemesterService(this._supabase);

  Future<List<Map<String, dynamic>>> getSemesters() async {
    return await _supabase
        .from('semesters')
        .select()
        .order('start_date', ascending: false);  // Most recent first
  }

  Future<Map<String, dynamic>> createSemester(
      Map<String, dynamic> data) async {
    return await _supabase.from('semesters').insert(data).select().single();
  }

  Future<Map<String, dynamic>> updateSemester(
      String id, Map<String, dynamic> data) async {
    return await _supabase
        .from('semesters')
        .update(data)
        .eq('id', id)
        .select()
        .single();
  }

  Future<void> deleteSemester(String id) async {
    await _supabase.from('semesters').delete().eq('id', id);
  }
}
SemesterRepository — JSON to typed model conversion:
class SemesterRepository {
  final SemesterService _service;

  SemesterRepository(this._service);

  Future<List<Semester>> getSemesters() async {
    final data = await _service.getSemesters();
    return data.map((json) => Semester.fromJson(json)).toList();
  }

  Future<Semester> createSemester(Semester semester) async {
    final data = await _service.createSemester(semester.toJson());
    return Semester.fromJson(data);
  }

  Future<Semester> updateSemester(Semester semester) async {
    final data = await _service.updateSemester(semester.id, semester.toJson());
    return Semester.fromJson(data);
  }

  Future<void> deleteSemester(String id) async {
    await _service.deleteSemester(id);
  }
}
```

B.3 — Semester Provider and Active Semester Management

File: lib/providers/semester_provider.dart

Purpose: The semester provider layer has two distinct responsibilities. SemesterNotifier manages the full list of semesters the user has created. ActiveSemesterNotifier manages the currently selected semester — the keystone of the app's data isolation system. Every data provider (courses, tasks, deadlines, sessions) reads the active semester ID before fetching anything.

```
SemesterNotifier — List management:
class SemesterNotifier extends StateNotifier<AsyncValue<List<Semester>>> {
  final SemesterRepository _repository;

  SemesterNotifier(this._repository, Ref _)
      : super(const AsyncValue.loading()) {
    loadSemesters();
  }

  Future<void> loadSemesters() async {
    // Preserve previous data during reload to avoid blank UI flickers
    state = const AsyncValue<List<Semester>>.loading().copyWithPrevious(state);
    try {
      final semesters = await _repository.getSemesters();
      if (!mounted) return;
      state = AsyncValue.data(semesters);
    } catch (e, st) {
      if (!mounted) return;
      state = AsyncValue.error(e, st);
    }
  }

  Future<Semester?> addSemester(Semester semester) async {
    try {
      final created = await _repository.createSemester(semester);
      await loadSemesters();
      return created;
    } catch (e) {
      return null;
    }
  }

  Future<void> updateSemester(Semester semester) async {
    await _repository.updateSemester(semester);
    await loadSemesters();
  }

  Future<void> deleteSemester(String id) async {
    await _repository.deleteSemester(id);
    await loadSemesters();
  }
}
ActiveSemesterNotifier — Keystone selection with persistence:
The active semester ID is stored in student_profiles.active_semester_id in Supabase. An
_explicitSelection static field prevents the selection from reverting during provider rebuilds caused
by token refreshes.
class ActiveSemesterNotifier extends AsyncNotifier<String?> {
  // Survives provider rebuilds — only cleared on logout
  static String? _explicitSelection;

  @override
  FutureOr<String?> build() async {
    final user = ref.watch(currentUserProvider);
    if (user == null) {
      _explicitSelection = null;  // Clear on logout
      return null;
    }

    // Return the explicit in-memory selection if the user already chose one
    if (_explicitSelection != null) return _explicitSelection;

    // Otherwise, load from Supabase student_profiles
    return await _service.getActiveSemesterId()
        .timeout(const Duration(seconds: 25));
  }

  Future<void> setActiveSemester(String semesterId) async {
    // Guard: prevent concurrent semester switches
    if (ref.read(isSemesterSwitchingProvider)) return;
    if (state.valueOrNull == semesterId) return;

    ref.read(isSemesterSwitchingProvider.notifier).state = true;
    try {
      final success = await _service.updateActiveSemester(semesterId);
      if (!success) return;

      // Persist in memory to survive provider rebuilds
      _explicitSelection = semesterId;
      state = AsyncValue.data(semesterId);
    } finally {
      ref.read(isSemesterSwitchingProvider.notifier).state = false;
    }
  }
}
Display ordering — active semester always appears first:
List<Semester> orderSemestersForDisplay(
  List<Semester> semesters, {
  String? activeSemesterId,
}) {
  final sorted = List<Semester>.from(semesters)
    ..sort((a, b) {
      final aDate = a.startDate ?? a.createdAt;
      final bDate = b.startDate ?? b.createdAt;
      return bDate.compareTo(aDate);  // Most recent first
    });

  if (activeSemesterId == null) return sorted;

  // Promote the active semester to the top of the list
  final activeIndex = sorted.indexWhere((s) => s.id == activeSemesterId);
  if (activeIndex > 0) {
    final active = sorted.removeAt(activeIndex);
    sorted.insert(0, active);
  }
  return sorted;
}
```

B.4 — Course Service, Repository, and Provider

Files: lib/services/supabase/course_service.dart · lib/repositories/course_repository.dart · lib/providers/course_provider.dart

Purpose: Courses are always fetched scoped to the active semester ID. The CourseNotifier includes a migration safety step that backfills any legacy course rows that were created before the semester isolation constraint was enforced.

```dart
// CourseService — Semester-scoped Supabase queries:
class CourseService {
  final SupabaseClient _supabase;

  CourseService(this._supabase);

  Future<List<Map<String, dynamic>>> getCourses({String? semesterId}) async {
    // Return empty immediately — prevents stale cross-semester data
    if (semesterId == null) return [];

    return await _supabase
        .from('courses')
        .select()
        .eq('semester_id', semesterId)  // Always scoped to active semester
        .order('course_name', ascending: true);
  }

  Future<Map<String, dynamic>> createCourse(Map<String, dynamic> data) async {
    return await _supabase.from('courses').insert(data).select().single();
  }

  Future<Map<String, dynamic>> updateCourse(
      String id, Map<String, dynamic> data) async {
    return await _supabase
        .from('courses').update(data).eq('id', id).select().single();
  }

  Future<void> deleteCourse(String id) async {
    await _supabase.from('courses').delete().eq('id', id);
  }
}
// CourseRepository — typed model conversion:
class CourseRepository {
  final CourseService _service;

  CourseRepository(this._service);

  Future<List<Course>> getCourses({String? semesterId}) async {
    final data = await _service.getCourses(semesterId: semesterId);
    return data.map((json) => Course.fromJson(json)).toList();
  }

  Future<Course> createCourse(Course course) async {
    final data = await _service.createCourse(course.toJson());
    return Course.fromJson(data);
  }

  Future<Course> updateCourse(Course course) async {
    final data = await _service.updateCourse(course.id, course.toJson());
    return Course.fromJson(data);
  }

  Future<void> deleteCourse(String id) async {
    await _service.deleteCourse(id);
  }
}
// CourseNotifier — semester-aware loading with migration safety:
class CourseNotifier extends StateNotifier<AsyncValue<List<Course>>> {
  final CourseRepository _repository;
  String? _lastLoadedSemesterId;

  Future<void> loadCourses({String? semesterId, bool force = false}) async {
    // Skip reload if the same semester is already loaded and data is valid
    if (semesterId != null &&
        _lastLoadedSemesterId == semesterId &&
        state.hasValue &&
        !force) return;

    state = const AsyncValue<List<Course>>.loading().copyWithPrevious(state);
    try {
      final courses = await _repository.getCourses(semesterId: semesterId);

      // Migration safety: backfill legacy rows missing a semester_id
      if (semesterId != null) {
        for (final course in courses.where((c) => c.semesterId == null)) {
          await _repository.updateCourse(
              course.copyWith(semesterId: semesterId));
        }
      }

      // Normalize and filter to ensure strict semester isolation
      final normalized = semesterId == null
          ? courses
          : courses
              .map((c) => c.copyWith(semesterId: c.semesterId ?? semesterId))
              .where((c) => c.semesterId == semesterId)
              .toList();

      if (!mounted) return;
      _lastLoadedSemesterId = semesterId;
      state = AsyncValue.data(normalized);
    } catch (e, st) {
      if (!mounted) return;
      state = AsyncValue.error(e, st);
    }
  }

  Future<void> addCourse(Course course) async {
    final semesterId = ref?.read(activeSemesterProvider).value;
    if (semesterId == null) return;
    // Inject active semester before persisting
    await _repository.createCourse(course.copyWith(semesterId: semesterId));
    await loadCourses(semesterId: semesterId, force: true);
  }

  Future<void> deleteCourse(String id) async {
    final semesterId = ref?.read(activeSemesterProvider).value;
    await _repository.deleteCourse(id);
    if (semesterId != null) await loadCourses(semesterId: semesterId, force: true);
  }
}

// Derived provider — only active (non-archived) courses for UI dropdowns
final activeCoursesProvider = Provider<List<Course>>((ref) {
  final courses = ref.watch(courseProvider).value ?? [];
  return courses.where((c) => c.isActive).toList();
});
```

B.5 — Task Service and Repository

Files: lib/services/supabase/task_service.dart · lib/repositories/task_repository.dart

Purpose: All task queries are doubly scoped — by user_id (enforced by RLS) and by semester_id (enforced by the application layer). The completeAllTasksForDeadline operation ensures progress tracking stays consistent when a parent deadline is marked as done.

```dart
TaskService — Supabase operations:
class TaskService {
  final SupabaseClient _supabase;

  TaskService(this._supabase);

  Future<List<Map<String, dynamic>>> getTasks({
    required String userId,
    String? semesterId,
  }) async {
    if (semesterId == null) return [];

    return await _supabase
        .from('tasks')
        .select()
        .eq('user_id', userId)
        .eq('semester_id', semesterId)   // Strict semester isolation
        .order('created_at', ascending: false);
  }

  Future<Map<String, dynamic>> createTask(Map<String, dynamic> data) async {
    return await _supabase.from('tasks').insert(data).select().single();
  }

  Future<Map<String, dynamic>> updateTask(
      String id, Map<String, dynamic> data) async {
    final query = _supabase.from('tasks').update(data).eq('id', id);
    // Additional safety: scope update to the authenticated user's rows
    if (data.containsKey('user_id')) query.eq('user_id', data['user_id']);
    return await query.select().single();
  }

  Future<void> deleteTask(String id) async {
    await _supabase.from('tasks').delete().eq('id', id);
  }

  /// Bulk-completes all tasks linked to a deadline when the parent deadline
  /// is marked done, keeping task and deadline progress counts aligned.
  Future<void> completeAllTasksForDeadline({
    required String userId,
    required String semesterId,
    required String deadlineId,
  }) async {
    await _supabase
        .from('tasks')
        .update({
          'status': 'completed',
          'updated_at': DateTime.now().toIso8601String(),
        })
        .eq('user_id', userId)
        .eq('semester_id', semesterId)
        .eq('deadline_id', deadlineId)
        .neq('status', 'completed');  // Only update non-completed tasks
  }
}
TaskRepository — typed model mapping:
class TaskRepository {
  final TaskService _taskService;

  TaskRepository(this._taskService);

  Future<List<Task>> getTasks(
      {required String userId, String? semesterId}) async {
    final data = await _taskService.getTasks(
        userId: userId, semesterId: semesterId);
    return data.map((json) => Task.fromJson(json)).toList();
  }

  Future<Task> createTask(Task task) async {
    final data = task.toJson();
    data.remove('id');          // Let Postgres generate UUID
    data.remove('created_at'); // Let Postgres set the timestamp
    final response = await _taskService.createTask(data);
    return Task.fromJson(response);
  }

  Future<Task> updateTask(Task task) async {
    final data = Map<String, dynamic>.from(task.toJson());
    data.remove('id');
    data.remove('created_at');
    data['deadline_id'] = task.deadlineId;  // Explicit null is valid here
    final response = await _taskService.updateTask(task.id, data);
    return Task.fromJson(response);
  }

  Future<void> deleteTask(String taskId) async {
    await _taskService.deleteTask(taskId);
  }

  Future<void> completeAllTasksForDeadline({
    required String userId,
    required String semesterId,
    required String deadlineId,
  }) => _taskService.completeAllTasksForDeadline(
        userId: userId,
        semesterId: semesterId,
        deadlineId: deadlineId,
      );
}
```

B.6 — Task Management Provider

File: lib/providers/tasks_provider.dart

Purpose: TasksNotifier manages the full task lifecycle — loading, creating, updating, deleting, and status toggling. It applies an optimistic update pattern: state is updated immediately in memory, and rolled back to its previous value if the Supabase write fails. Smart notifications are triggered after every successful mutation without blocking the UI.

```dart
class TasksNotifier extends StateNotifier<AsyncValue<List<Task>>> {
  final Ref _ref;
  final dynamic taskRepository;
  final SupabaseClient supabaseClient;
  String? _lastLoadedSemesterId;

  Future<void> createTask({
    required String title,
    String? courseId,
    DateTime? deadline,
    String? deadlineId,
    String? priority,
    ...
  }) async {
    final userId = supabaseClient.auth.currentUser?.id;
    if (userId == null) return;

    final semesterId = _ref.read(activeSemesterProvider).value;
    if (semesterId == null) {
      throw Exception('No active semester set. Cannot create task.');
    }

    // Validate that the linked deadline belongs to the same course
    final deadlinesList = _ref.read(deadlinesProvider).valueOrNull ?? [];
    final safeDeadlineId = validatedDeadlineIdForTask(
      userId: userId,
      taskCourseId: courseId,
      deadlineId: deadlineId,
      deadlines: deadlinesList,
    );

    final newTask = Task(
      id: '',
      userId: userId,
      semesterId: semesterId,  // Always scoped to active semester
      title: title,
      courseId: courseId,
      deadline: deadline,
      deadlineId: safeDeadlineId,
      priority: priority,
      status: 'pending',
      createdAt: DateTime.now(),
    );

    final previous = state;
    try {
      final created = await taskRepository.createTask(newTask);
      // Optimistic: prepend to list before any network confirmation
      state = state.whenData((tasks) => [created, ...tasks]);
      _runSmartNotifications(state.valueOrNull ?? []);
    } catch (e, st) {
      state = previous;  // Rollback on failure
      Error.throwWithStackTrace(e, st);
    }
  }

  Future<void> updateTask(Task task) async {
    final previous = state;
    try {
      final updated = await taskRepository.updateTask(task);
      state = state.whenData((tasks) =>
          tasks.map<Task>((t) => t.id == updated.id ? updated : t).toList());
      _runSmartNotifications(state.valueOrNull ?? []);
    } catch (e, st) {
      state = previous;  // Rollback on failure
      Error.throwWithStackTrace(e, st);
    }
  }

  Future<void> updateTaskStatus(String taskId, String status) async {
    final current = state.value;
    if (current == null) return;
    final existing = current.firstWhere((t) => t.id == taskId);
    await updateTask(existing.copyWith(
      status: status,
      updatedAt: DateTime.now(),
    ));
  }

  Future<void> deleteTask(String taskId) async {
    final previous = state;
    try {
      await taskRepository.deleteTask(taskId);
      state = state.whenData(
          (tasks) => tasks.where((t) => t.id != taskId).toList());
      _runSmartNotifications(state.valueOrNull ?? []);
    } catch (e, st) {
      state = previous;  // Rollback on failure
      Error.throwWithStackTrace(e, st);
    }
  }

  /// Evaluates smart notifications after any task mutation.
  /// Runs fire-and-forget — failures must not break task operations.
  Future<void> _runSmartNotifications(List<Task> tasks) async {
    if (!mounted) return;
    try {
      await SmartNotificationEngine.evaluateTaskChanges(tasks);
    } catch (_) {}
  }
}
Filtered and sorted task view:
final filteredTasksProvider = Provider<AsyncValue<List<Task>>>((ref) {
  final tasksAsync = ref.watch(tasksProvider);
  final filter = ref.watch(taskFilterProvider);

  // Sort: overdue first, then by deadline ascending, completed last
  int compareTasks(Task a, Task b) {
    if (a.status == 'completed' && b.status != 'completed') return 1;
    if (a.status != 'completed' && b.status == 'completed') return -1;

    final aIsOverdue = a.deadline != null &&
        a.deadline!.isBefore(DateTime.now()) && a.status != 'completed';
    final bIsOverdue = b.deadline != null &&
        b.deadline!.isBefore(DateTime.now()) && b.status != 'completed';

    if (aIsOverdue && !bIsOverdue) return -1;
    if (!aIsOverdue && bIsOverdue) return 1;

    if (a.deadline == null) return 1;
    if (b.deadline == null) return -1;
    return a.deadline!.compareTo(b.deadline!);
  }

  return tasksAsync.whenData((tasks) {
    final filtered = switch (filter) {
      'pending'   => tasks.where((t) => t.status == 'pending').toList(),
      'completed' => tasks.where((t) => t.status == 'completed').toList(),
      'overdue'   => tasks.where((t) =>
          t.deadline != null &&
          t.deadline!.isBefore(DateTime.now()) &&
          t.status != 'completed').toList(),
      _           => tasks,
    };
    return List<Task>.from(filtered)..sort(compareTasks);
  });
});
```

B.7 — Deadline Provider

File: lib/providers/deadlines_provider.dart

Purpose: The DeadlinesNotifier manages academic deadlines and enforces cascading completion — when a deadline is marked done, all tasks linked to it are automatically completed. This maintains consistent progress counts across the task and deadline views. The rollback pattern mirrors the task provider to ensure UI consistency on failure.

```dart
class DeadlinesNotifier extends StateNotifier<AsyncValue<List<Deadline>>> {

  Future<void> createDeadline(Deadline deadline) async {
    final semesterId = _ref.read(activeSemesterProvider).value;
    if (semesterId == null) {
      throw Exception('No active semester set. Cannot create deadline.');
    }
    // Inject semester scope before persisting
    final scopedDeadline = deadline.copyWith(semesterId: semesterId);
    final previous = state;
    try {
      final created = await deadlineRepository.createDeadline(scopedDeadline);
      state = state.whenData((items) => [created, ...items]);
    } catch (e, st) {
      state = previous;
      Error.throwWithStackTrace(e, st);
    }
  }

  Future<void> toggleDeadlineCompletion(String id) async {
    final existing = state.value?.firstWhere((d) => d.id == id);
    if (existing == null) return;

    final previous = state;
    try {
      final willComplete = !existing.isCompleted;
      final updated = await deadlineRepository.markDeadlineCompleted(
        id: id,
        isCompleted: !existing.isCompleted,
      );
      state = state.whenData(
          (items) => items.map((d) => d.id == id ? updated : d).toList());

      // Cascade: complete all linked tasks when the deadline itself is done
      if (willComplete) await _completeLinkedTasksWhenDeadlineMarkedDone(id);
    } catch (e, st) {
      state = previous;
      Error.throwWithStackTrace(e, st);
    }
  }

  /// Completing a deadline marks the whole milestone as handled.
  /// Un-completing a deadline intentionally does NOT revert tasks —
  /// per-task completions represent honest individual progress records.
  Future<void> _completeLinkedTasksWhenDeadlineMarkedDone(
      String deadlineId) async {
    final userId = supabaseClient.auth.currentUser?.id;
    final semesterId = _ref.read(activeSemesterProvider).value;
    if (userId == null || semesterId == null) return;

    await _ref.read(taskRepositoryProvider).completeAllTasksForDeadline(
      userId: userId,
      semesterId: semesterId,
      deadlineId: deadlineId,
    );
    // Refresh tasks so the UI reflects the cascaded completions
    await _ref.read(tasksProvider.notifier).refresh();
  }
}
```

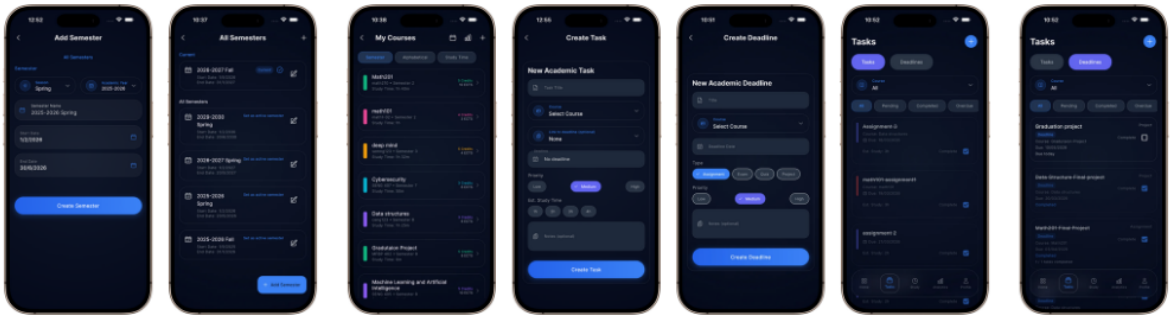
B.8 — Realtime Synchronisation

File: lib/providers/realtime_provider.dart

Purpose: The realtimeProvider establishes a single persistent Supabase Realtime channel per authenticated user. It subscribes to Postgres change events on five core tables and uses 250ms debounced invalidation to batch rapid successive changes into a single provider refresh. This prevents unnecessary rebuilds while keeping the UI in sync with the database in near real-time.

```dart
final realtimeProvider = Provider.autoDispose<void>((ref) {
  ref.keepAlive();  // Persist even after the last subscriber disposes
  final supabase = ref.read(supabaseProvider);
  final user = ref.watch(currentUserProvider);
  if (user == null) return;

  // Separate debounce timers per table — independent refresh cadences
  Timer? taskDebounce;
  Timer? studyDebounce;
  Timer? courseResultsDebounce;
  Timer? coursesDebounce;
  Timer? deadlinesDebounce;

  /// RLS provides server-side protection, but user-ID checks here prevent
  /// acting on broadcast events from other users on the same channel.
  bool payloadBelongsToUser(PostgresChangePayload payload) {
    final newUserId = payload.newRecord['user_id']?.toString();
    final oldUserId = payload.oldRecord['user_id']?.toString();
    return newUserId == user.id || oldUserId == user.id;
  }

  /// Only refresh course results if the change belongs to the active semester.
  bool shouldRefreshSelectedSemesterResults(PostgresChangePayload payload) {
    final activeSemesterId = ref.read(activeSemesterProvider).value;
    if (activeSemesterId == null) return false;
    final newSemesterId = payload.newRecord['semester_id']?.toString();
    final oldSemesterId = payload.oldRecord['semester_id']?.toString();
    return newSemesterId == activeSemesterId || oldSemesterId == activeSemesterId;
  }

  /// Only invalidate risk scores when ECTS credits change — not on every course edit.
  bool didCourseChangeAffectRisk(PostgresChangePayload payload) {
    if (payload.oldRecord.isEmpty || payload.newRecord.isEmpty) return true;
    return payload.oldRecord['ects']?.toString() !=
        payload.newRecord['ects']?.toString();
  }

  // Debounce helpers — cancel pending timer and restart 250ms window
  void scheduleTaskRefresh() {
    taskDebounce?.cancel();
    taskDebounce = Timer(const Duration(milliseconds: 250),
        () => ref.invalidate(tasksProvider));
  }

  void scheduleStudyRefresh() {
    studyDebounce?.cancel();
    studyDebounce = Timer(const Duration(milliseconds: 250),
        () => ref.invalidate(studySessionsProvider));
  }

  void scheduleDeadlinesRefresh() {
    deadlinesDebounce?.cancel();
    deadlinesDebounce = Timer(const Duration(milliseconds: 250),
        () => ref.invalidate(deadlinesProvider));
  }

  void scheduleCoursesRefresh() {
    coursesDebounce?.cancel();
    coursesDebounce = Timer(const Duration(milliseconds: 250),
        () => ref.invalidate(courseProvider));
  }

  void scheduleCourseResultsRefresh(String eventSemesterId) {
    courseResultsDebounce?.cancel();
    courseResultsDebounce = Timer(const Duration(milliseconds: 250), () async {
      final activeSemesterId = ref.read(activeSemesterProvider).value;
      if (activeSemesterId != null && activeSemesterId == eventSemesterId) {
        await ref.read(courseResultsProvider.notifier)
            .loadForSemester(activeSemesterId, forceRefresh: true);
      }
      ref.invalidate(riskProvider);  // Risk score depends on grade data
    });
  }

  // Single channel for the authenticated user — subscribes to all five tables
  final channel = supabase
      .channel('core-live-${user.id}')
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'tasks',
        callback: (payload) {
          if (!payloadBelongsToUser(payload)) return;
          scheduleTaskRefresh();
        },
      )
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'academic_deadlines',
        callback: (payload) {
          if (!payloadBelongsToUser(payload)) return;
          scheduleDeadlinesRefresh();
        },
      )
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'study_sessions',
        callback: (payload) {
          if (!payloadBelongsToUser(payload)) return;
          scheduleStudyRefresh();
        },
      )
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'course_results',
        callback: (payload) {
          if (!payloadBelongsToUser(payload)) return;
          if (!shouldRefreshSelectedSemesterResults(payload)) return;
          final eventSemesterId =
              payload.newRecord['semester_id']?.toString() ??
              payload.oldRecord['semester_id']?.toString();
          if (eventSemesterId == null) return;
          scheduleCourseResultsRefresh(eventSemesterId);
        },
      )
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'courses',
        callback: (payload) {
          if (!payloadBelongsToUser(payload)) return;
          scheduleCoursesRefresh();
          // Selectively invalidate risk when ECTS credits change
          if (didCourseChangeAffectRisk(payload)) ref.invalidate(riskProvider);
        },
      )
      .subscribe();

  // Clean up all timers and close the WebSocket channel on provider disposal
  ref.onDispose(() async {
    taskDebounce?.cancel();
    studyDebounce?.cancel();
    courseResultsDebounce?.cancel();
    coursesDebounce?.cancel();
    deadlinesDebounce?.cancel();
    await supabase.removeChannel(channel);
  });
});
```

Figure A.3. Semester and Course Management System



Study Session and Pomodoro System

This appendix presents representative source code from Acadia's study session and Pomodoro timer module. The excerpts demonstrate the timer state machine, focus/break cycle management, Hive-backed persistence across app restarts, background recovery, structured session saving, and study analytics generation.

C.1 — Study Session Domain Model

File: lib/models/study_session.dart

Purpose: The StudySession model represents a completed or in-progress study record. It carries two sets of fields: a base set shared by all session types, and a Pomodoro-specific set (totalFocusMinutes, cyclesCompleted, totalBreakMinutes) that is only populated when isPomodoro is true. Deserialisation handles the joined courses table response from Supabase.

```dart
class StudySession {
  final String id;
  final String userId;
  final String courseId;
  final String semesterId;
  final String sessionType;   // pomodoro | deep_work | review | practice
  final int durationMinutes;  // For Pomodoro: mirrors totalFfocusMinutes
  final DateTime createdAt;
  final String? intent;       // What the student planned to study
  final bool completed;

  // ── Pomodoro-specific fields ──
  final bool isPomodoro;
  final int? totalFocusMinutes;
  final int? totalBreakMinutes;
  final int? cyclesCompleted;
  final int? totalCycles;

  // ── Post-session mood attachment ──
  final double? mood;
  final double? stress;

  // ── Accuracy tracking ──
  final int? plannedMinutes;
  final int? actualMinutes;
  final DateTime? startedAt;
  final DateTime? endedAt;

  factory StudySession.fromJson(Map<String, dynamic> json) {
    final isPomodoro = json['is_pomodoro'] as bool? ?? false;
    final totalFocusMinutes = json['total_focus_minutes'] as int?;

    // Handle Supabase joined courses: can arrive as a List or a Map
    final dynamic courseJoinRaw = json['courses'];
    final Map<String, dynamic>? courseJoin = courseJoinRaw is List
        ? (courseJoinRaw.isNotEmpty ? Map<String, dynamic>.from(courseJoinRaw.first) : null)
        : (courseJoinRaw is Map ? Map<String, dynamic>.from(courseJoinRaw) : null);

    // For Pomodoro sessions, durationMinutes mirrors totalFocusMinutes
    // to keep backwards-compatible analytics queries working correctly
    final durationMinutes = isPomodoro
        ? (totalFocusMinutes ?? (json['duration_minutes'] as num).toInt())
        : (json['duration_minutes'] as num).toInt();

    return StudySession(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      courseId: json['course_id'] as String,
      semesterId: json['semester_id'] as String? ?? '',
      sessionType: json['session_type'] as String,
      durationMinutes: durationMinutes,
      createdAt: DateTime.parse(json['created_at'] as String),
      course: courseJoin != null ? Course.fromJson(courseJoin) : null,
      intent: json['intent'] as String?,
      completed: json['completed'] as bool? ?? true,
      isPomodoro: isPomodoro,
      totalFocusMinutes: totalFocusMinutes,
      totalBreakMinutes: json['total_break_minutes'] as int?,
      cyclesCompleted: json['cycles_completed'] as int?,
      totalCycles: json['total_cycles'] as int?,
      mood: (json['mood'] as num?)?.toDouble(),
      stress: (json['stress'] as num?)?.toDouble(),
      plannedMinutes: (json['planned_minutes'] as num?)?.toInt(),
      actualMinutes: (json['actual_minutes'] as num?)?.toInt() ?? durationMinutes,
      startedAt: json['started_at'] != null
          ? DateTime.tryParse(json['started_at'] as String)
          : DateTime.parse(json['created_at'] as String),
      endedAt: json['ended_at'] != null
          ? DateTime.tryParse(json['ended_at'] as String)
          : DateTime.parse(json['created_at'] as String)
              .add(Duration(minutes: durationMinutes)),
    );
  }

  Map<String, dynamic> toJson() => {
    if (id.isNotEmpty) 'id': id,
    'user_id': userId,
    'course_id': courseId,
    'semester_id': semesterId,
    'session_type': sessionType,
    'duration_minutes': durationMinutes,
    'created_at': createdAt.toIso8601String(),
    if (intent != null) 'intent': intent,
    'completed': completed,
    'is_pomodoro': isPomodoro,
    if (totalFocusMinutes != null) 'total_focus_minutes': totalFocusMinutes,
    if (totalBreakMinutes != null) 'total_break_minutes': totalBreakMinutes,
    if (cyclesCompleted != null)   'cycles_completed': cyclesCompleted,
    if (totalCycles != null)       'total_cycles': totalCycles,
    if (mood != null)              'mood': mood,
    if (stress != null)            'stress': stress,
    if (plannedMinutes != null)    'planned_minutes': plannedMinutes,
    if (actualMinutes != null)     'actual_minutes': actualMinutes,
    if (startedAt != null)         'started_at': startedAt!.toIso8601String(),
    if (endedAt != null)           'ended_at': endedAt!.toIso8601String(),
  };
}
```

C.2 — Timer State Model

File: lib/providers/study_sessions_provider.dart

Purpose: StudyTimerState is a fully immutable value object representing every aspect of the timer at a given instant. Immutability ensures that each tick produces a new state, making the state history debuggable and the UI rebuild predictable.

```dart
enum StudyType { pomodoro, deepWork, review }

int getDefaultDuration(StudyType type) {
  switch (type) {
    case StudyType.pomodoro: return 25;  // minutes
    case StudyType.deepWork: return 45;
    case StudyType.review:   return 20;
  }
}

class StudyTimerState {
  final Duration elapsed;
  final bool isRunning;
  final bool isPaused;
  final DateTime? startTime;
  final DateTime? endTime;
  final DateTime? pausedAt;
  final Duration pausedDuration;     // Cumulative time spent paused
  final StudyType sessionType;
  final String? courseId;
  final String? intent;

  // ── Pomodoro cycle tracking ──
  final int currentRound;            // 1 to totalRounds
  final int totalRounds;             // Always 4 for Pomodoro
  final bool isBreak;                // True during 5-minute break phases
  final Duration targetDuration;
  final bool isCompleted;
  final int completedCycles;
  final int totalFocusAccumulated;   // Minutes of focus across all
ro  final int totalBreakAccumulated;   // Minutes of break across all
rounds
  const StudyTimerState({
    this.elapsed = Duration.zero,
    this.isRunning = false,
    this.isPaused = false,
    this.startTime,
    this.pausedDuration = Duration.zero,
    this.sessionType = StudyType.deepWork,
    this.currentRound = 1,
    this.totalRounds = 4,
    this.isBreak = false,
    this.targetDuration = const Duration(minutes: 45),
    this.isCompleted = false,
    this.completedCycles = 0,
    this.totalFocusAccumulated = 0,
    this.totalBreakAccumulated = 0,
    ...
  });
}
```

C.3 — Timer State Machine

File: lib/providers/study_sessions_provider.dart — StudyTimerNotifier

Purpose: StudyTimerNotifier implements the complete timer state machine. It tracks wall-clock time rather than tick counts, making it immune to frame drops and accurate after background recovery. The notifier registers itself as a WidgetsBindingObserver to detect when the app is foregrounded and recalculate elapsed time based on actual timestamps.

```
IDLE → RUNNING ↔ PAUSED → COMPLETED
          ↕ (Pomodoro: Focus → Break → Focus → ... × 4 rounds)
Active elapsed time calculation:
/// Computes true elapsed time using wall-clock arithmetic.
/// Subtracts all accumulated pause time so neither pausing nor
/// backgrounding inflates the elapsed duration.
Duration _elapsedAt(DateTime now) {
  if (state.startTime == null) return Duration.zero;

  final totalSinceStart = now.difference(state.startTime!);

  // Subtract all previously accumulated pause time
  var activeElapsed = totalSinceStart - state.pausedDuration;

  // Also subtract time elapsed since the current pause began
  if (state.isPaused && state.pausedAt != null) {
    activeElapsed -= now.difference(state.pausedAt!);
  }

  return activeElapsed.isNegative ? Duration.zero : activeElapsed;
}
Start:
void start() {
  if (state.isRunning) return;
  _timer?.cancel();

  // Reset accumulation counters when starting fresh after a completed session
  if (state.isCompleted || state.startTime == null) {
    state = state.copyWith(
      currentRound: 1,
      isBreak: false,
      totalFocusAccumulated: 0,
      totalBreakAccumulated: 0,
    );
    ref.read(studySessionsProvider.notifier).resetPomodoroSaveFlag();
  }
  _handleStart();
}

void _handleStart() {
  final now = DateTime.now();
  final isPomodoro = state.sessionType == StudyType.pomodoro;

  state = state.copyWith(
    elapsed: Duration.zero,
    isRunning: true,
    isPaused: false,
    startTime: now,
    pausedAt: null,
    pausedDuration: Duration.zero,
    isCompleted: false,
    totalRounds: isPomodoro ? 4 : 1,
    targetDuration: isPomodoro && !state.isBreak
        ? const Duration(minutes: 25)
        : state.targetDuration,
  );

  NotificationService.scheduleCompletion(state.targetDuration);
  _startTimerTargetCheck();
  _persist();
}
Pause and Resume:
void pause() {
  if (!state.isRunning) return;
  _timer?.cancel();
  final now = DateTime.now();
  state = state.copyWith(
    elapsed: _elapsedAt(now),
    isRunning: false,
    isPaused: true,
    pausedAt: now,   // Record the exact moment pausing began
  );
  NotificationService.cancelAll();
  _persist();
}

void resume() {
  if (!state.isPaused || state.startTime == null) return;
  final now = DateTime.now();
  // Accumulate the duration of this pause into the running total
  final extraPause = state.pausedAt == null
      ? Duration.zero
      : now.difference(state.pausedAt!);

  state = state.copyWith(
    elapsed: _elapsedAt(now),
    isRunning: true,
    isPaused: false,
    pausedAt: null,
    pausedDuration: state.pausedDuration + extraPause,
  );
  NotificationService.scheduleCompletion(state.targetDuration -
state.elapsed);
  _startTimerTargetCheck();
  _persist();
}
Per-second tick with target detection:
void _startTimerTargetCheck() {
  _timer?.cancel();
  _timer = Timer.periodic(const Duration(seconds: 1), (_) {
    final elapsed = _elapsedAt(DateTime.now());
    if (elapsed >= state.targetDuration) {
      _timer?.cancel();
      state = state.copyWith(elapsed: state.targetDuration);
      _onPhaseComplete();   // Transition to next phase or end session
    } else {
      state = state.copyWith(elapsed: elapsed);
    }
    _persist();  // Write state to Hive on every tick
  });
}
```

C.4 — Focus/Break Cycle Management

File: lib/providers/study_sessions_provider.dart — _onPhaseComplete()

Purpose: This method drives the Pomodoro cycle transitions. After each focus round, it either starts a 5-minute break (if rounds remain) or marks the session fully complete (after round 4). After each break, it automatically starts the next focus round. Audio feedback and haptic vibration are triggered at each transition.

```dart
Future<void> _onPhaseComplete() async {
  _timer?.cancel();
  NotificationService.showCompletionNotification();

  // Audio and haptic feedback on every phase completion
  await _player.play(AssetSource('sounds/finish.mp3'));
  Haptics.vibrate(HapticsType.success);

  final isPomodoro = state.sessionType == StudyType.pomodoro;

  // Accumulate the minutes from the phase that just finished
  if (!state.isBreak) {
    state = state.copyWith(
      totalFocusAccumulated:
          state.totalFocusAccumulated + state.elapsed.inSeconds ~/ 60,
    );
  } else {
    state = state.copyWith(
      totalBreakAccumulated:
          state.totalBreakAccumulated + state.elapsed.inSeconds ~/ 60,
    );
  }

  // Non-Pomodoro sessions end here
  if (!isPomodoro) {
    state = state.copyWith(isCompleted: true, isRunning: false);
    if (state.courseId != null) {
      ref.read(studySessionsProvider.notifier).saveFromTimer(state);
    }
    _persist();
    return;
  }

  // ── Pomodoro: Focus phase completed ──
  if (!state.isBreak) {
    final isLastRound = state.currentRound == state.totalRounds;

    if (isLastRound) {
      // All 4 rounds done — mark fully complete and save
      state = state.copyWith(
        isCompleted: true,
        isRunning: false,
        completedCycles: state.completedCycles + 1,
      );
      if (state.courseId != null) {
        ref.read(studySessionsProvider.notifier)
            .savePomodoroSession(state, isFullyCompleted: true);
      }
    } else {
      // Start the 5-minute break before the next focus round
      state = state.copyWith(
        isBreak: true,
        isCompleted: false,
        targetDuration: const Duration(minutes: 5),
        elapsed: Duration.zero,
        completedCycles: state.completedCycles + 1,
      );
      Future.delayed(const Duration(milliseconds: 50), _handleStart);
    }

  // ── Pomodoro: Break phase completed ──
  } else {
    if (state.currentRound < state.totalRounds) {
      // Advance to the next focus round
      state = state.copyWith(
        isBreak: false,
        isCompleted: false,
        currentRound: state.currentRound + 1,
        targetDuration: const Duration(minutes: 25),
        elapsed: Duration.zero,
      );
      Future.delayed(const Duration(milliseconds: 50), _handleStart);
    }
  }
  _persist();
}
```

C.5 — Hive Persistence Logic

File: lib/providers/study_sessions_provider.dart — _initPersistence() and _persist()

Purpose: Timer state is written to a Hive box on every tick and every state transition. On app launch, the persisted state is read back and the elapsed time is recomputed from the stored startTime and lastUpdate timestamps. If the timer expired while the app was closed, the completion handler is triggered immediately rather than leaving the session in an ambiguous state.

```
Writing state to Hive (called on every tick):
Future<void> _persist() async {
  final box = await Hive.openBox('timer_state');

  // If no active session, clear the box to avoid phantom recovery
  if (state.startTime == null && !state.isRunning && !state.isPaused) {
    await box.clear();
    return;
  }

  await box.put('startTime',              state.startTime?.toIso8601String());
  await box.put('lastUpdate',             DateTime.now().toIso8601String());
  await box.put('isRunning',              state.isRunning);
  await box.put('isPaused',               state.isPaused);
  await box.put('pausedAt',               state.pausedAt?.toIso8601String());
  await box.put('pausedDurationMs',       state.pausedDuration.inMilliseconds);
  await box.put('sessionTypeIndex',       state.sessionType.index);
  await box.put('courseId',               state.courseId);
  await box.put('intent',                 state.intent);
  await box.put('currentRound',           state.currentRound);
  await box.put('totalFocusAccumulated',  state.totalFocusAccumulated);
  await box.put('totalBreakAccumulated',  state.totalBreakAccumulated);
  await box.put('targetDurationSeconds',  state.targetDuration.inSeconds);
}
Restoring state on cold start:
Future<void> _initPersistence() async {
  final box = await Hive.openBox('timer_state');

  final startTimeStr    = box.get('startTime') as String?;
  final lastUpdateStr   = box.get('lastUpdate') as String?;
  final isRunning       = box.get('isRunning', defaultValue: false) as bool;
  final isPaused        = box.get('isPaused', defaultValue: false) as bool;
  final pausedAtStr     = box.get('pausedAt') as String?;
  final pausedDurationMs = box.get('pausedDurationMs', defaultValue: 0) as int;
  final sessionTypeIndex = box.get('sessionTypeIndex', defaultValue: 0) as int;
  final courseId        = box.get('courseId') as String?;
  final currentRound    = box.get('currentRound', defaultValue: 1) as int;
  final totalFocusAcc   = box.get('totalFocusAccumulated', defaultValue: 0) as
infinal totalBreakAcc   = box.get('totalBreakAccumulated', defaultValue: 0) as
infinal targetSecs      = box.get('targetDurationSeconds') as int?;

  if (startTimeStr == null) return; // No persisted session

  final startTime    = DateTime.parse(startTimeStr);
  final sessionType  = StudyType.values[sessionTypeIndex];
  final targetDuration = targetSecs != null
      ? Duration(seconds: targetSecs)
      : const Duration(minutes: 25);

  // Restore the full timer state before computing elapsed
  state = state.copyWith(
    startTime: startTime,
    isRunning: isRunning,
    isPaused: isPaused,
    pausedDuration: Duration(milliseconds: pausedDurationMs),
    pausedAt: pausedAtStr != null ? DateTime.parse(pausedAtStr) : null,
    sessionType: sessionType,
    courseId: courseId,
    currentRound: currentRound,
    totalFocusAccumulated: totalFocusAcc,
    totalBreakAccumulated: totalBreakAcc,
    targetDuration: targetDuration,
  );

  final now = DateTime.now();
  if (isRunning && lastUpdateStr != null) {
    final elapsed = _elapsedAt(now);

    if (elapsed >= targetDuration) {
      // Timer expired while app was closed — trigger completion immediately
      state = state.copyWith(elapsed: targetDuration);
      _onPhaseComplete();
    } else {
      // Resume from correct position and reschedule the completion notification
      state = state.copyWith(elapsed: elapsed);
      NotificationService.scheduleCompletion(targetDuration - elapsed);
      _startTimerTargetCheck();
    }
  } else {
    state = state.copyWith(elapsed: _elapsedAt(now));
  }
}
```

C.6 — Background Recovery

File: lib/providers/study_sessions_provider.dart — didChangeAppLifecycleState()

Purpose: The notifier implements WidgetsBindingObserver to intercept lifecycle events. When the app returns from background, elapsed time is recalculated from startTime using wall-clock arithmetic — correcting for any time that passed while the app was suspended. This prevents the timer from falling behind after a background period.

```dart
@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.resumed && this.state.isRunning) {
    final now = DateTime.now();
    final elapsed = _elapsedAt(now);  // Recompute from real timestamps

    if (elapsed >= this.state.targetDuration) {
      // Phase ended while app was in background — trigger completion now
      this.state = this.state.copyWith(elapsed:
this.state.targetDuration);
      _completePhase();
    } else {
      // Advance elapsed to the correct position
      this.state = this.state.copyWith(elapsed: elapsed);
    }
    _persist();
  }
}
```

C.7 — Session Persistence to Supabase

File: lib/providers/study_sessions_provider.dart — StudySessionsNotifier

Purpose: StudySessionsNotifier handles the transition from in-memory timer state to a persisted database record. Two separate save paths exist: one for standard sessions, and one for Pomodoro sessions which carry rich cycle metadata. A duplicate-save guard (_pomodoroSaved) prevents the same session being written twice when both _onPhaseComplete and stop() are called in quick succession.

```
Standard session save (Deep Work / Review):
Future<void> saveFromTimer(StudyTimerState timerState, {
  String? sessionTypeOverride,
}) async {
  final minutes = timerState.elapsed.inMinutes;

  // Business rule: a valid session must be completed and at least 10 minutes
  final isValidSession = timerState.isCompleted && minutes >= 10;
  if (!isValidSession || timerState.isBreak || _isSaving) return;

  final semesterId = ref.read(activeSemesterProvider).value;
  if (semesterId == null) throw Exception('No active semester. Cannot save.');

  _isSaving = true;
  try {
    final newSession = StudySession(
      id: '',
      userId: supabaseClient.auth.currentUser!.id,
      courseId: timerState.courseId!,
      semesterId: semesterId,
      sessionType: sessionTypeOverride ?? _mapType(timerState.sessionType),
      durationMinutes: minutes,
      createdAt: DateTime.now(),
      plannedMinutes: timerState.targetDuration.inMinutes,
      actualMinutes: minutes,
      startedAt: timerState.startTime,
      endedAt: DateTime.now(),
      intent: timerState.intent,
      completed: timerState.isCompleted,
    );

    final previous = state;
    try {
      final created = await studyRepository.createSession(newSession);
      if (!mounted) return;
      state = state.whenData((sessions) => [created, ...sessions]);

      // Refresh analytics so the dashboard reflects the new session immediately
      ref.read(analyticsProvider.notifier).refresh();

      // Prompt mood logging if enabled; show info snackbar if tracking is off
      if (ref.read(moodSettingsProvider.notifier).shouldShowAfterStudy) {
        ref.read(moodActionsProvider).queueSessionMoodPrompt();
      } else if (!ref.read(moodSettingsProvider).trackingEnabled) {
        ref.read(moodTrackingOffSnackbarProvider.notifier).state = true;
      }
    } catch (e, st) {
      state = previous;
      Error.throwWithStackTrace(e, st);
    }
  } finally {
    _isSaving = false;
  }
}
Pomodoro structured session save:
Future<void> savePomodoroSession(StudyTimerState timerState, {
  bool isFullyCompleted = false,
}) async {
  final totalFocus = timerState.totalFocusAccumulated;

  if (totalFocus < 10) return;  // Minimum meaningful focus time

  // Duplicate-save guard — prevents double write from _onPhaseComplete + stop()
  if (_pomodoroSaved) return;
  _pomodoroSaved = true;

  if (_isSaving) return;
  _isSaving = true;

  final semesterId = ref.read(activeSemesterProvider).value;
  if (semesterId == null) throw Exception('No active semester. Cannot save.');

  try {
    final session = StudySession(
      id: '',
      userId: supabaseClient.auth.currentUser!.id,
      courseId: timerState.courseId!,
      semesterId: semesterId,
      sessionType: 'pomodoro',
      durationMinutes: totalFocus,    // Analytics queries use this field
      createdAt: DateTime.now(),
      plannedMinutes: timerState.targetDuration.inMinutes,
      actualMinutes: totalFocus,
      startedAt: timerState.startTime,
      endedAt: DateTime.now(),
      intent: timerState.intent,
      completed: isFullyCompleted,

      // Pomodoro-specific rich metadata
      isPomodoro: true,
      totalFocusMinutes: totalFocus,
      totalBreakMinutes: timerState.totalBreakAccumulated,
      cyclesCompleted: timerState.completedCycles,
      totalCycles: timerState.totalRounds,
    );

    final previous = state;
    try {
      final created = await studyRepository.createSession(session);
      if (!mounted) return;
      state = state.whenData((sessions) => [created, ...sessions]);
      ref.read(analyticsProvider.notifier).refresh();
    } catch (e, st) {
      state = previous;
      Error.throwWithStackTrace(e, st);
    }
  } finally {
    _isSaving = false;
  }
}
```

C.8 — Study Service and Repository

Files: lib/services/supabase/study_service.dart · lib/repositories/study_repository.dart

Purpose: The service performs a relational SELECT that joins the linked course record in a single query — avoiding a separate round-trip. The repository converts raw JSON to typed StudySession models before they reach the provider layer.

```
StudyService — joined Supabase query:
class StudyService {
  final SupabaseClient _supabase;

  StudyService(this._supabase);

  Future<List<Map<String, dynamic>>> getSessions({
    required String userId,
    String? semesterId,
  }) async {
    if (semesterId == null) return [];

    return await _supabase
        .from('study_sessions')
        .select('''
          id, user_id, course_id, semester_id,
          session_type, duration_minutes, created_at,
          intent, completed, completed_cycles, skipped_breaks,
          is_pomodoro, total_focus_minutes, total_break_minutes,
          cycles_completed, total_cycles,
          mood, stress, planned_minutes, actual_minutes,
          started_at, ended_at,

          courses:course_id (
            id, course_name, color, semester_id
          )
        ''')
        .eq('user_id', userId)
        .eq('semester_id', semesterId)
        .order('created_at', ascending: false)
        .limit(50);
  }

  Future<Map<String, dynamic>> createSession(Map<String, dynamic> data) async {
    return _supabase.from('study_sessions').insert(data).select('''
      id, user_id, course_id, semester_id,
      session_type, duration_minutes, created_at,
      is_pomodoro, total_focus_minutes, total_break_minutes,
      cycles_completed, total_cycles, mood, stress,
      planned_minutes, actual_minutes, started_at, ended_at,
      courses:course_id ( id, course_name, color, semester_id )
    ''').single();
  }

  Future<Map<String, dynamic>> updateSession(
      String id, Map<String, dynamic> data) async {
    return _supabase.from('study_sessions').update(data).eq('id', id)
        .select('id, mood, stress, ...').single();
  }
}
StudyRepository — typed conversion:
class StudyRepository {
  final StudyService _studyService;

  StudyRepository(this._studyService);

  Future<List<StudySession>> getSessions(
      {required String userId, String? semesterId}) async {
    final data = await _studyService.getSessions(
        userId: userId, semesterId: semesterId);
    return data.map((json) => StudySession.fromJson(json)).toList();
  }

  Future<StudySession> createSession(StudySession session) async {
    final data = session.toJson()..remove('id');  // Let Postgres generate UUID
    final response = await _studyService.createSession(data);
    return StudySession.fromJson(response);
  }

  Future<StudySession> updateSession(StudySession session) async {
    final data = session.toJson()..remove('id');
    final response = await _studyService.updateSession(session.id, data);
    return StudySession.fromJson(response);
  }
}
```

C.9 — Session Analytics Generation: Study Heatmap

File: lib/providers/study_heatmap_provider.dart

Purpose: The heatmap provider computes a full year of daily study activity scores from the session list, without any additional database query. Each day's score combines total minutes and session count, giving higher weight to consistency (multiple sessions) alongside volume. The provider uses copyWithPrevious to prevent the heatmap from disappearing during data refreshes.

```dart
final studyHeatmapProvider = Provider<AsyncValue<HeatmapData>>((ref) {
  final sessionsAsync = ref.watch(studySessionsProvider);

  // Keep showing previous heatmap during refresh — avoids layout flicker
  if (sessionsAsync.isLoading && !sessionsAsync.hasValue) {
    return const AsyncValue.loading();
  }
  final sessions = sessionsAsync.valueOrNull ?? [];

  final today = _normalize(DateTime.now());

  // Build a full year aligned to Monday boundaries
  final firstDay = today.subtract(const Duration(days: 364));
  final paddingStart = firstDay.weekday - 1;
  final startDate = _normalize(firstDay.subtract(Duration(days:
paddingStart)));= _normalize(today.add(Duration(days: 7 - today.weekday)));

  // Initialise every day in the range with zero stats
  final Map<DateTime, List<int>> dailyStats = {};
  final totalDays = endDate.difference(startDate).inDays + 1;
  for (int i = 0; i < totalDays; i++) {
    dailyStats[_normalize(startDate.add(Duration(days: i)))] = [0, 0];
  }

  int maxScore = 1;

  // Aggregate sessions into daily buckets
  for (final session in sessions) {
    final dayKey = _normalize(session.createdAt);
    if (dailyStats.containsKey(dayKey)) {
      dailyStats[dayKey]![0] += session.durationMinutes;  // Total minutes
      dailyStats[dayKey]![1] += 1;                        // Session count

      // Score = minutes + (sessions × 5) — rewards consistency alongside volume
      final score = dailyStats[dayKey]![0] + (dailyStats[dayKey]![1] * 5);
      if (score > maxScore) maxScore = score;
    }
  }

  // Build the final list of HeatmapDay objects
  final heatmapDays = List.generate(totalDays, (i) {
    final date = _normalize(startDate.add(Duration(days: i)));
    final stats = dailyStats[date] ?? [0, 0];
    return HeatmapDay(
      date: date,
      totalMinutes: stats[0],
      sessionsCount: stats[1],
      score: stats[0] + (stats[1] * 5),
    );
  });

  final result = HeatmapData(days: heatmapDays, maxScore: maxScore);

  // Propagate loading state with previous data so UI does not blank
  if (sessionsAsync.isLoading) {
    return AsyncValue<HeatmapData>.loading()
        .copyWithPrevious(AsyncValue.data(result));
  }
  if (sessionsAsync.hasError) {
    return AsyncValue.error(
        sessionsAsync.error!, sessionsAsync.stackTrace!);
  }
  return AsyncValue.data(result);
});
```

Figure A.4. Study Session Persistence System

Analytics and Risk Engine

This appendix documents the analytical subsystem responsible for evaluating a student's academic health at runtime. The system is composed of five cooperating layers: a pure data model (**RiskInput**) that aggregates behavioural signals, a mathematical scoring engine (RiskEngine) that converts those signals into a bounded risk score, a stateless calculation adapter (**RiskCalculator**) that bridges the two, a live Riverpod provider (**riskProvider**) that feeds real-time data into the pipeline and tracks week-over-week trends, and a decision engine (**DecisionEngine**) that translates quantitative scores into prioritised, context-aware recommendations. GPA calculation and burnout scoring are computed inside AnalyticsRepository, which also manages caching and avoids redundant database queries by reusing model objects already loaded by other providers.

D.1 — Risk Input Data Model

File: lib/domain/risk/risk_input.dart

Purpose: RiskInput is an immutable value object that carries every signal required for a risk calculation. Separating the data contract from the computation logic means the scoring formula can be unit-tested in isolation without touching Riverpod, Supabase, or Flutter. Two guard properties — isEmpty and hasInsufficientData — allow upstream providers to abort calculation cleanly when the student has not yet generated any behavioural data.

```dart
class RiskInput {
  final int overdueTasks;
  final int upcomingTasks;
  final int totalECTS;
  final double studyHours;
  final double studyGoal;
  final double studyDeficit;
  final double gradeRiskAdjustment;
  final double avgDailyStress;
  final double avgSessionStress;
  final bool isStressRising;
  final double consistency;
  final bool hasStressData;
  final bool hasAcademicContent;

  const RiskInput({ ... });

  // Sentinel to skip computation when the semester has no content yet.
  factory RiskInput.empty() => const RiskInput(
        overdueTasks: 0, upcomingTasks: 0,
        totalECTS: 0, studyHours: 0, studyGoal: 0.0,
      );

  bool get isEmpty =>
      overdueTasks == 0 && upcomingTasks == 0 &&
      studyHours == 0 && totalECTS == 0;

  // True when no study sessions OR mood logs exist — prevents false-positive risk
  // scores being shown to a user who has just enrolled.
  bool get hasInsufficientData => studyHours == 0 && !hasStressData;
}
```

D.2 — Risk Scoring Engine

File: lib/core/services/risk_engine.dart

Purpose: RiskEngine is the single authoritative implementation of the academic risk formula. It is a purely static, side-effect-free class — it receives primitive inputs and returns a RiskResult with a bounded score, level classification, per-factor breakdown, and human-readable reason list. Every factor contributes an independently capped sub-score; the five caps sum to exactly 100, ensuring no single dimension can dominate the output.

```dart
class RiskEngine {
  static RiskResult calculate({
    required int overdueTasks,
    required int dueSoonTasks,
    required double studyDeficitHours,
    required double stressLevel,
    required double consistency,
    bool hasStressData = false,
  }) {
    // Factor 1 — Overdue tasks (0–30 pts)
    // 10 pts per task; three or more overdue tasks reach the ceiling.
    final overdueScore = (overdueTasks * 10.0).clamp(0.0, 30.0);

    // Factor 2 — Imminent deadlines (0–15 pts)
    // 5 pts per task due within three days; caps at three tasks.
    final dueSoonScore = (dueSoonTasks * 5.0).clamp(0.0, 15.0);

    // Factor 3 — Study deficit (0–25 pts)
    // Normalised against 24 hours — a full day of missed study fills the cap.
    final deficitScore = ((studyDeficitHours / 24.0) * 25.0).clamp(0.0, 25.0);

    // Factor 4 — Session stress (0–15 pts)
    // Only applied when the student has logged mood data; avoids penalising
    // new users who have not yet tracked a session mood.
    double stressScore = 0.0;
    if (hasStressData) {
      stressScore = ((stressLevel / 10.0) * 15.0).clamp(0.0, 15.0);
    }

    // Factor 5 — Consistency penalty (0–15 pts)
    // Perfect consistency (1.0) contributes 0; zero consistency contributes
    final consistencyScore = ((1.0 - consistency) * 15.0).clamp(0.0, 15.0);

    // Sum is strictly additive: 30 + 15 + 25 + 15 + 15 = 100.
    final finalScore = (overdueScore + dueSoonScore + deficitScore +
        stressScore + consistencyScore).clamp(0.0, 100.0);

    final level = finalScore >= 60 ? RiskLevel.high
                : finalScore >= 30 ? RiskLevel.moderate
                : RiskLevel.low;

    return RiskResult(
      score: finalScore,
      level: level,
      reasons: [...],
      breakdown: {
        'overdue': overdueScore, 'dueSoon': dueSoonScore,
        'deficit': deficitScore, 'stress': stressScore,
        'consistency': consistencyScore,
      },
    );
  }
}
```

D.3 — Risk Calculator (Domain Adapter)

File: lib/domain/risk/risk_calculator.dart

Purpose: RiskCalculator is a stateless adapter that translates a RiskInput domain object into the primitive parameters expected by RiskEngine, then maps the engine's raw breakdown back into the typed RiskResult used by the UI. This indirection means the core scoring formula can be replaced or recalibrated without touching provider or UI code.

```dart
class RiskCalculator {
  static RiskResult calculate(RiskInput input) {
    // Delegate to the core engine — all scoring logic lives there.
    final coreResult = core.RiskEngine.calculate(
      overdueTasks:      input.overdueTasks,
      dueSoonTasks:      input.upcomingTasks,
      studyDeficitHours: input.studyDeficit,
      // avgSessionStress drives this factor, not daily stress,
      // because it reflects stress during actual study behaviour.
      stressLevel:       input.avgSessionStress,
      consistency:       input.consistency,
      hasStressData:     input.hasStressData,
    );

    return RiskResult(
      overdueContribution:      coreResult.breakdown['overdue']      ?? 0.0,
      upcomingContribution:     coreResult.breakdown['dueSoon']      ?? 0.0,
      ectsContribution:         0.0, // Reserved for future ECTS-weighted model
      studyContribution:        coreResult.breakdown['deficit']      ?? 0.0,
      sessionStressContribution: coreResult.breakdown['stress']      ?? 0.0,
      consistencyContribution:  coreResult.breakdown['consistency']  ?? 0.0,
      totalScore: coreResult.score,
      riskLevel:  _level(coreResult.score),
    );
  }

  static String _level(double score) {
    if (score < 30) return 'low';
    if (score < 60) return 'medium';
    return 'high';
  }
}
```

D.4 — Live Risk Provider (Data Aggregation and Trend)

File: lib/providers/risk_provider.dart

Purpose: riskInputProvider watches every live Riverpod data source simultaneously and blocks calculation until all are resolved — preventing a partial dataset from producing a misleadingly low score. Beyond the core five factors, it applies grade-based adjustments (penalty for failing grades, relief for recently completed deadlines and linked tasks) to modulate the raw score with academic context. riskProvider then runs the calculator twice — once for the current week and once for the previous week — to derive a trend delta that the dashboard displays as a directional indicator.

```dart
final riskInputProvider = Provider<RiskInput?>((ref) {
  // Guard: return null until every async source has finished loading.
  // A partial dataset would understate risk and mislead the student.
  if (tasksAsync.isLoading || sessionsAsync.isLoading ||
      moodsAsync.isLoading || deadlinesAsync.isLoading ||
      coursesAsync.isLoading || resultsAsync.isLoading) {
    return null;
  }

  // Overdue/upcoming counts span both tasks AND academic_deadlines
  // to maintain parity with the analytics and chat subsystems.
  final overdue   = overdueFromTasks + overdueFromDeadlines;
  final upcoming  = upcomingFromTasks + upcomingFromDeadlines;

  // Consistency is measured as task completion rate, not study-day frequency,
  // because it better reflects whether the student is meeting their
commitments.
  final consistency = filteredTasks.isEmpty ? 1.0
      : (filteredTasks.where((t) => t.status == 'completed').length /
         filteredTasks.length).clamp(0.0, 1.0);

  // Grade adjustment — failing (FF/DD) grades add pressure;
  // high grades (BA/AA) provide a small relief.
  final adjustment = _normalizeGradeRiskAdjustment(
    _gradeAdjustment(results) +
    _overdueDeadlinesPenalty(filteredDeadlines, startOfToday) +
    _linkedTaskCompletionRelief(filteredTasks, filteredDeadlines) +
    _completedDeadlineAdjustment(filteredDeadlines)
  );

  return RiskInput(
    overdueTasks: overdue, upcomingTasks: upcoming,
    studyDeficit: deficit, consistency: consistency,
    avgSessionStress: avgSessionStress,
    hasStressData: sessionMoods.isNotEmpty,
    gradeRiskAdjustment: adjustment,
    ...
  );
});

final riskProvider = Provider<RiskState>((ref) {
  final currentResult = RiskCalculator.calculate(input);

  // Reconstruct last week's input using prior-week session hours only,
  // keeping all other factors constant to isolate the study-behaviour change.
  final previousResult = RiskCalculator.calculate(previousInput);

  // Suppress trend delta for accounts younger than 7 days — there is no
  // meaningful baseline to compare against.
  final trendDelta = user.createdAt.isAfter(sevenDaysAgo)
      ? null
      : (currentResult.totalScore - previousResult.totalScore).toInt();

  return RiskState(
    input: input, result: currentResult,
    hasData: input.hasAcademicContent, trendDelta: trendDelta,
  );
});

// Grade-based risk modifier — letter grades map to signed pressure scores.
double _gradeAdjustment(List<CourseResult> results) {
  double score = 0;
  for (final result in results) {
    final code = result.letterGrade.trim().toUpperCase().split(' ').first;
    if (code == 'FF' || code == 'DD' || code == 'DC') score += 10;
    else if (code == 'CC' || code == 'CB' || code == 'BB') score += 5;
    else if (code == 'BA') score -= 3;
    else if (code == 'AA') score -= 5;
  }
  return score;
}

// Completing ≥60 % of tasks linked to an open deadline reduces risk slightly —
// partial progress on a deadline is academically meaningful.
double _linkedTaskCompletionRelief(List<Task> tasks, List<Deadline> deadlines) {
  final rate = done / linked;
  if (rate >= 0.6) return -5.0;
  return 0.0;
}
```

D.5 — GPA Calculation and Burnout Scoring

File: lib/repositories/analytics_repository.dart

Purpose: _calculateGpa implements credit-weighted GPA following standard university policy: for any course attempted multiple times, only the most recent result is counted, preventing retake grades from being averaged with earlier failures. _computeBurnoutScore combines session stress, study deficit, and schedule consistency into a 0–100 burnout index that is independent of the task-based risk score and feeds the DecisionEngine's burnout guard rule.

```dart
double _calculateGpa({
  required List<Map<String, dynamic>> results,
  required List<Map<String, dynamic>> courses,
  Map<String, int>? gradeDistributionOut,
}) {
  // Build credit lookup from courses table.
  final courseCreditById = <String, int>{};
  for (final course in courses) {
    courseCreditById[course['id']] = course['credits'] ?? 0;
  }

  // University policy: for retaken courses, only the latest attempt counts.
  // Iterating all results and keeping the most-recent timestamp per course_id
  // avoids a dedicated "latest attempt" query.
  final latestResultByCourse = <String, Map<String, dynamic>>{};
  for (final result in results) {
    final courseId = result['course_id'] ?? '';
    final existing = latestResultByCourse[courseId];
    if (existing == null || _resultTimestamp(result).isAfter(_resultTimestamp(existing))) {
      latestResultByCourse[courseId] = result;
    }
  }

  double weightedGradePoints = 0.0;
  int totalWeightedCredits = 0;

  for (final result in latestResultByCourse.values) {
    final code    = _normalizeGradeCode(result['letter_grade']);
    final gradeValue = _gradeToValue(code!); // AA=4.0 … FF=0.0
    final credits = courseCreditById[result['course_id']] ?? 0;

    // Weight each grade by its ECTS credit count before summing.
    weightedGradePoints  += gradeValue! * credits;
    totalWeightedCredits += credits;
  }

  // Fall back to unweighted average when ECTS data is unavailable.
  if (totalWeightedCredits > 0) return weightedGradePoints / totalWeightedCredits;
  return unweightedGradePoints / unweightedCount;
}

double _computeBurnoutScore({
  required double avgStress,
  required double studyDeficit,
  required double consistency,
}) {
  // Each factor is normalised to [0, 1] before applying its weight.
  // Stress and deficit each account for 40 %; schedule irregularity for 20 %.
  // The asymmetric weighting reflects that active stress and missed study
  // hours are stronger burnout predictors than inconsistency alone.
  final stressFactor      = (avgStress   / 10.0).clamp(0.0, 1.0);
  final deficitFactor     = (studyDeficit / 24.0).clamp(0.0, 1.0);
  final consistencyFactor = (1.0 - consistency).clamp(0.0, 1.0);

  return ((stressFactor * 0.4 + deficitFactor * 0.4 + consistencyFactor * 0.2) * 100.0)
      .clamp(0.0, 100.0);
}
```

D.6 — Analytics Repository — Metrics Computation

File: lib/repositories/analytics_repository.dart

Purpose: computeFromPreloaded is the primary entry point when the analytics dashboard is opened. Rather than issuing a full set of database queries, it accepts the typed model objects already held in memory by the study session, task, and mood providers, and fetches only the data not available elsewhere (courses, results, semester metadata). This design avoids redundant Supabase round-trips while keeping the analytics screen consistent with the live state shown elsewhere in the application.

```dart
Future<AnalyticsData> computeFromPreloaded({
  required String userId,
  required String semesterId,
  required List<StudySession> sessions,
  required List<Task> tasks,
  required List<MoodLog> moodLogs,
  List<Deadline> deadlines = const [],
}) async {
  // Bust the per-semester cache so the dashboard always reflects live state.
  _cache.remove(semesterId);

  // Fetch only what isn't held by other providers — avoids duplicate DB queries.
  final results = await _analyticsService.getCourseResults(...);
  final courses = await _analyticsService.getCourses(semesterId: semesterId);
  final semData = await _analyticsService.getSemester(semesterId);
  final studyGoal = (semData?['weekly_study_goal'] ?? 15).toDouble();

  // Split mood logs by source so the UI can display daily vs. session
  // averages on separate tabs without a second query.
  bool isSession(MoodLog m) => m.source.trim().toLowerCase() == 'session';
  final dailyMoods   = moodLogs.where((m) => !isSession(m)).toList();
  final sessionMoods = moodLogs.where((m) =>  isSession(m)).toList();

  // Consistency = distinct study days in the last 7 days / 7.
  // Day-based (not session-count-based) to reward daily engagement.
  final sessionDays = sessions
      .where((s) => s.createdAt.isAfter(now.subtract(const Duration(days: 7))))
      .map((s) { final d = s.createdAt.toLocal(); return '${d.year}-${d.month}-${d.day}'; })
      .toSet();
  final consistency = (sessionDays.length / 7.0).clamp(0.0, 1.0);

  // Weekly average study hours = total semester hours / semester weeks.
  // Provides a long-run baseline more stable than the current-week figure.
  final weeks = (end.difference(start).inDays / 7.0).clamp(1.0, double.infinity);
  weeklyAvg = totalHistoricalHours / weeks;

  // Semester GPA and cumulative GPA are computed separately so the dashboard
  // can display both without an additional query.
  final semesterGpa   = _calculateGpa(results: results, courses: courses, ...);
  final cumulativeGpa = _calculateGpa(results: allResults, courses: allCourses);

  // Risk and burnout are computed here for the analytics snapshot; riskProvider
  // computes the live version independently from typed Riverpod state.
  final riskScore    = _calculateSnapshotRisk(...);
  final burnoutScore = _computeBurnoutScore(
    avgStress:     avgSessionStressVal,
    studyDeficit:  studyDeficitHours,
    consistency:   consistency,
  );

  return AnalyticsData(
    averageGPA: semesterGpa, cumulativeGPA: cumulativeGpa,
    riskScore: riskScore, burnoutScore: burnoutScore,
    consistency: consistency, weeklyAverageStudyHours: weeklyAvg,
    avgDailyMood: avgDailyMoodVal, avgSessionStress: avgSessionStressVal,
    isStressRising: stressTrend > 0.5,
    ...
  );
}
```

D.7 — Decision Engine (Recommendation Generator)

File: lib/domain/decision/decision_engine.dart

Purpose: DecisionEngine.evaluate() converts quantitative analytics signals into a prioritised list of natural-language recommendations. Rules are evaluated in strict descending priority: welfare concerns (high stress, burnout) always appear before productivity guidance (deficit, deadlines, time-of-day). Confidence is calibrated against session count and consistency so the system acknowledges data scarcity rather than generating high-confidence recommendations from a thin behavioural record.

Figure A.5. Analytics Dashboard Implementation

Smart Notification System

This appendix documents the notification subsystem, which is responsible for generating, filtering, delivering, and persisting academic alerts across the full application lifecycle — including when the application is suspended or terminated. The system is structured in four cooperating layers: a core delivery service (**NotificationService**) that manages Firebase Cloud Messaging integration, per-type cooldown enforcement, and local scheduling; a rule-based evaluation engine (**SmartNotificationEngine**) that decides which notifications are warranted based on live analytics; a platform-specific background worker (**BackgroundNotificationService**) that re-runs the same evaluation logic on a 30-minute schedule without any foreground UI; and a Riverpod provider (**NotificationsNotifier**) that maintains the in-app notification inbox with Realtime synchronisation and idempotent deduplication.

E.1 — Notification Service (FCM Integration and Delivery)

File: lib/services/notification_service.dart

Purpose: NotificationService is the single delivery gateway for all notification output. It initialises Firebase Cloud Messaging and the local notification plugin, manages FCM token persistence (including token rotation), enforces a per-type cooldown and a hard daily cap of five notifications, and routes notification taps to the correct screen using a type|referenceId payload convention. Night-time suppression (22:00–07:00) prevents non-critical alerts from interrupting sleep; the inbox row is still written so the student sees the notification when they next open the app.

E.2 — Smart Notification Engine (Contextual Alert Generation)

File: lib/services/smart_notification_engine.dart

Purpose: SmartNotificationEngine applies academic context rules to decide which notifications are appropriate at any given moment. It consolidates multiple overdue or upcoming items into a single summary notification rather than sending one per task — preventing notification fatigue. Risk and stress signals are evaluated independently against threshold values aligned with the RiskEngine bands (≥70 = high, ≥50 = moderate; stress ≥7 = high). All output is routed through enqueueSmartNotification, which applies an additional daily-cap and night-time gate before delivery.

```
class NotificationService {
  // In-memory cooldown mirror — also persisted to SharedPreferences so the
  // cooldown window survives app restarts and background worker invocations.
  static final Map<String, DateTime> _lastSent = {};
  static const int _dailyLimit = 5;

  static Future<void> init({
    bool requestPermissions = true,
    bool attachForegroundListeners = true,
  }) async {
    await _loadPersistedCooldowns(); // Restore cooldown state from disk on boot.

    // FCM token refresh: rotated tokens must be re-persisted immediately
    // so the Edge Function always has a valid delivery address.
    FirebaseMessaging.instance.onTokenRefresh.listen((newToken) async {
      final user = Supabase.instance.client.auth.currentUser;
      if (user != null) {
        await Supabase.instance.client
            .from('users').update({'fcm_token': newToken}).eq('id', user.id);
      }
    });

    // Handle push received while app is in the foreground — FCM does not
    // show a system tray notification in this case, so we display it locally.
    FirebaseMessaging.onMessage.listen((RemoteMessage message) {
      _plugin.show(
        message.notification.hashCode,
        message.notification!.title,
        message.notification!.body,
        _notificationDetailsForType(message.data['type'] ?? 'system'),
        payload: '${message.data['type']}|${message.data['id'] ?? ''}',
      );
    });

    // Handle tap on a push received while the app was backgrounded (not terminated).
    FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
      _onNotificationTap('${message.data['type']}|${message.data['id'] ?? ''}');
    });

    // Handle cold-start tap: app was terminated when the user tapped the push.
    await _handleInitialMessage();
  }

  // Two-gate cooldown check: per-type cooldown window AND global daily cap.
  static Future<bool> _canSend(String key, {Duration? cooldown}) async {
    final now = DateTime.now().toUtc();
    final last = _lastSent[key];
    final resolvedCooldown = cooldown ?? NotificationTypeConfig.resolve(key).cooldown;

    // Gate 1: Has enough time passed since the last notification of this type?
    if (last != null && now.difference(last) <= resolvedCooldown) return false;

    // Gate 2: Have we already sent the maximum allowed notifications today?
    final dailyCount = await _readCounter(_storageKey(_dailyCountPrefix, _dailyBucketKey(now)));
    if (dailyCount >= _dailyLimit) return false;

    // Record the send — both in memory and on disk for background-worker parity.
    _lastSent[key] = now;
    await _prefs!.setString(_storageKey(_lastSentPrefix, key), now.toIso8601String());
    await _writeCounter(_storageKey(_dailyCountPrefix, _dailyBucketKey(now)), dailyCount + 1);
    return true;
  }

  // Deliver via FCM Edge Function (system tray) AND local plugin (immediate display).
  // Using both channels ensures delivery even when the FCM network path is slow.
  static Future<void> _deliverPushAndLocal({
    required String title, required String body,
    required String type, String? referenceId,
  }) async {
    final token = userData?['fcm_token'];
    if (token != null) {
      await supabase.functions.invoke('send-push', body: {
        'token': token, 'title': title, 'body': body,
        'type': type, 'referenceId': referenceId ?? '',
        'priority': _edgePriorityForType(effectivePriority),
      });
    }

    // Local plugin ensures the notification appears even if the Edge Function
    // is delayed or the device has no network at the moment of delivery.
    await _plugin.show(
      DateTime.now().millisecondsSinceEpoch % 100000,
      title, body,
      _notificationDetailsForType(type),
      payload: '$type|${referenceId ?? ""}',
    );
  }

  // Tap handler: parse the type|referenceId payload and navigate to the
  // appropriate screen. If GoRouter context is not yet ready (cold start),
  // the payload is buffered and flushed once the context becomes available.
  static void _onNotificationTap(String payload) {
    final context = rootNavigatorKey.currentContext;
    if (context == null) {
      _pendingTapPayload = payload;
      Future.delayed(const Duration(milliseconds: 800), _flushPendingTapIfAny);
      return;
    }
    final [type, rawId] = payload.split('|');
    switch (type) {
      case 'due_today':
      case 'overdue':      context.push('/task-detail?id=${Uri.encodeComponent(taskId!)}'); break;
      case 'deadline':     context.push('/deadline-detail?id=${Uri.encodeComponent(deadlineId!)}');
break;
      case 'risk':
      case 'stress':
      case 'academic_summary': context.go('/home'); break;
      default:             context.go('/home');
    }
  }

  // Schedule an exact-time local notification for a study session end.
  // Uses zonedSchedule so the trigger is timezone-aware and survives device sleep.
  static Future<void> scheduleCompletion(Duration remaining) async {
    final scheduledDate = tz.TZDateTime.now(tz.local).add(remaining);
    await _plugin.zonedSchedule(0, 'Session Complete', 'Your study session has finished.',
      scheduledDate, _notificationDetails,
      androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,
      uiLocalNotificationDateInterpretation: UILocalNotificationDateInterpretation.absoluteTime,
      payload: 'study_goal|',
    );
  }
}
```

E.3 — Background Workers (Platform-Specific Scheduling)

File: lib/services/background_notification_service_mobile.dart

Purpose: BackgroundNotificationService registers platform-specific background execution mechanisms — Workmanager on Android and BackgroundFetch on iOS — and routes both into the same runEvaluation entry point. The evaluation bootstraps its own Supabase and NotificationService instances without a Flutter UI context, fetches fresh analytics and task data, then hands off to NotificationIntelligenceService. This design ensures that academic risk and overdue alerts are delivered even when the student has not opened the application for hours.

```dart
// Android: Workmanager periodic task — runs approximately every 30 minutes
// when the device has network, surviving reboots via registerPeriodicTask.
@pragma('vm:entry-point')
void callbackDispatcher() {
  wm.Workmanager().executeTask((task, inputData) async {
    WidgetsFlutterBinding.ensureInitialized();
    await BackgroundNotificationService.runEvaluation(source: 'workmanager');
    return true;
  });
}

// iOS: BackgroundFetch headless task — called by the OS when the app is
// not in the foreground; task.timeout signals the OS-imposed deadline.
@pragma('vm:entry-point')
void backgroundFetchHeadlessTask(bf.HeadlessTask task) async {
  if (task.timeout) { bf.BackgroundFetch.finish(task.taskId); return; }
  await BackgroundNotificationService.runEvaluation(source: 'background_fetch');
  bf.BackgroundFetch.finish(task.taskId);
}

class BackgroundNotificationService {
  static Future<void> initialize({
    required String supabaseUrl, required String supabaseAnonKey,
  }) async {
    // Persist credentials to SharedPreferences so the isolated background
    // process can bootstrap Supabase without access to the .env asset.
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_supabaseUrlKey, supabaseUrl);
    await prefs.setString(_supabaseAnonKeyKey, supabaseAnonKey);

    if (Platform.isAndroid) {
      await wm.Workmanager().initialize(callbackDispatcher, isInDebugMode: false);
      await wm.Workmanager().registerPeriodicTask(
        _workmanagerTaskName, _workmanagerTaskName,
        frequency: const Duration(minutes: 30),
        existingWorkPolicy: wm.ExistingWorkPolicy.keep, // Prevent duplicate schedules.
        constraints: wm.Constraints(networkType: wm.NetworkType.connected),
      );
    }

    if (Platform.isIOS) {
      await bf.BackgroundFetch.configure(
        bf.BackgroundFetchConfig(
          minimumFetchInterval: 30, stopOnTerminate: false,
          enableHeadless: true, startOnBoot: true,
        ),
        (taskId) async { await runEvaluation(source: 'background_fetch');
bf.BackgroundFetch.finish(taskId); },
        (taskId) async { bf.BackgroundFetch.finish(taskId); },
      );
      bf.BackgroundFetch.registerHeadlessTask(backgroundFetchHeadlessTask);
    }
  }

  static Future<void> runEvaluation({String source = 'manual'}) async {
    // Restore Supabase from persisted credentials — background isolates do
    // not inherit the foreground app's initialized state.
    final supabaseUrl    = prefs.getString(_supabaseUrlKey);
    final supabaseAnonKey = prefs.getString(_supabaseAnonKeyKey);
    await Supabase.initialize(url: supabaseUrl!, anonKey: supabaseAnonKey!);

    // Initialize NotificationService without requesting permissions or
    // attaching foreground listeners — the background has no UI context.
    await NotificationService.init(
      requestPermissions: false,
      attachForegroundListeners: false,
    );

    final user = Supabase.instance.client.auth.currentUser;
    if (user == null) return;

    final semesterId = await SemesterStateService().getActiveSemesterId();
    if (semesterId == null) return;

    // Fetch fresh data independently — the background has no access to
    // Riverpod providers or cached foreground state.
    final analytics = await analyticsRepository.getAnalyticsData(user.id, semesterId: semesterId);
    final tasks     = await taskRepository.getTasks(userId: user.id, semesterId: semesterId);

    await NotificationIntelligenceService.evaluateBackground(
      analytics: analytics, tasks: tasks,
    );
  }
}
```

E.4 — Notification Intelligence Service (Filtering Layer)

File: lib/services/notification_intelligence_service.dart

Purpose: NotificationIntelligenceService is a stateless façade that unifies foreground and background evaluation paths. Both call through the same SmartNotificationEngine, ensuring identical decision logic regardless of execution context. It also exposes per-type configuration lookups and a shouldDeferDelivery() check that upstream callers can query before enqueuing, keeping night-time suppression logic in one place.

```dart
class NotificationIntelligenceService {
  // Foreground path: called after every analytics refresh or task mutation.
  static Future<void> evaluateForeground({
    required AnalyticsData analytics,
    List<Task>? tasks,
    TaskFetcher? fetchTasks,
  }) {
    return SmartNotificationEngine.evaluate(
      analytics: analytics, tasks: tasks, fetchTasks: fetchTasks,
    );
  }

  // Background path: called from Workmanager / BackgroundFetch workers.
  // Same decision logic — no divergence between foreground and background.
  static Future<void> evaluateBackground({
    required AnalyticsData analytics,
    List<Task>? tasks,
    TaskFetcher? fetchTasks,
  }) {
    return SmartNotificationEngine.evaluate(
      analytics: analytics, tasks: tasks, fetchTasks: fetchTasks,
    );
  }

  // Returns the per-type cooldown window and criticality flag — used by
  // upstream callers to decide whether to skip enqueuing entirely.
  static NotificationConfig configFor(String type) =>
      NotificationTypeConfig.resolve(type);

  // True between 22:00 and 07:00 — non-critical notifications should be
  // deferred to inbox-only during this window.
  static bool shouldDeferDelivery() => NotificationService.isNightTime();

  static bool isCritical(String type) => NotificationService.isCriticalType(type);
  static bool isSilent(String type)   => NotificationService.isSilentType(type);
}
```

E.5 — Notifications Provider (Inbox Sync and Realtime)

File: lib/providers/notifications_provider.dart

Purpose: NotificationsNotifier maintains the in-app notification inbox as a Riverpod StateNotifier. On every load it first runs _syncTriggeredNotifications, which derives notification rows directly from the current state of tasks, deadlines, and study sessions and writes them idempotently using _ensureDailyNotification (upsert-on-match, skip if identical). A Supabase Realtime channel monitors four tables and triggers a 350 ms debounced reload on any change, keeping the badge count accurate without polling. A deduplication pass on load removes stale or redundant rows using a composite referenceId|type key.

```dart
class NotificationsNotifier extends StateNotifier<AsyncValue<List<NotificationModel>>> {

  // Single Realtime channel subscribed to four tables — Postgres changes on
  // any of them may affect which notification rows are valid.
  Future<void> _ensureRealtimeSubscription(String userId) async {
    _realtimeChannel = supabaseClient
        .channel('notifications-live-$userId')
        .onPostgresChanges(
          event: PostgresChangeEvent.all, table: 'notifications',
          callback: (p) { if (_payloadBelongsToUser(p, userId)) _scheduleReload(withSync: false); },
        )
        .onPostgresChanges(
          event: PostgresChangeEvent.all, table: 'tasks',
          callback: (p) { if (_payloadBelongsToUser(p, userId)) _scheduleReload(withSync: true); },
        )
        .onPostgresChanges(
          event: PostgresChangeEvent.all, table: 'academic_deadlines',
          callback: (p) { if (_payloadBelongsToUser(p, userId)) _scheduleReload(withSync: true); },
        )
        .subscribe();
  }

  // Idempotent upsert: if a matching notification (same referenceId + type) already
  // exists, update title/message if they have changed; otherwise insert.
  // This prevents the inbox from growing duplicate rows on every app open.
  Future<void> _ensureDailyNotification({...}) async {
    final existing = await supabaseClient.from('notifications')
        .select('id,title,message')
        .eq('reference_id', referenceId).eq('type', type).maybeSingle();

    if (existing != null) {
      if (existing['title'] != title || existing['message'] != message) {
        await supabaseClient.from('notifications')
            .update({'title': title, 'message': message}).eq('id', existing['id']);
      }
      return; // Already present and up to date — no insert needed.
    }
    await notificationRepository.createNotification(NotificationModel(...));
  }

  // Derive notification seeds from current database state — tasks and deadlines
  // that are overdue, due today, or due tomorrow each produce one seed.
  // Deadlines that share an identity key (title + course + date) with a task
  // are suppressed to prevent duplicate entries for the same work item.
  Future<List<_NotificationSeed>> _buildTaskAndDeadlineSeeds({...}) async {
    for (final row in deadlineRows) {
      final dedupKey = _itemIdentityKey(title, courseName, dueDate);
      if (taskIdentityKeys.contains(dedupKey)) continue; // Task row takes precedence.
      seeds.add(_NotificationSeed(referenceId: 'deadline:$deadlineId', ...));
    }
    return seeds;
  }

  // Weekly study goal check — fires only on Sundays when actual minutes
  // fall short of the semester-configured weekly goal.
  Future<void> _syncTriggeredNotifications({...}) async {
    if (now.weekday == DateTime.sunday) {
      final weekMinutes = weekSessions.fold<int>(0,
          (sum, row) => sum + ((row['duration_minutes'] as num?)?.toInt() ?? 0));
      if (weekMinutes < weeklyGoalMinutes) {
        await _ensureDailyNotification(
          referenceId: 'study_goal:$weekKey',
          type: 'study_goal',
          title: l10n.weeklyStudyGoalMissed,
          message: l10n.weeklyStudyGoalMissedMessage(
            (weekMinutes / 60).toStringAsFixed(1),
            (weeklyGoalMinutes / 60).toStringAsFixed(1),
          ),
        );
      }
    }
  }

  // Mark-as-read applies optimistically in memory before writing to Supabase
  // so the badge clears immediately on tap without waiting for a round-trip.
  Future<void> markAsRead(String id) async {
    state = AsyncValue.data(state.valueOrNull!
        .map((n) => n.id == id ? n.copyWith(isRead: true) : n).toList());
    try {
      final updated = await notificationRepository.markAsRead(id) as NotificationModel;
      state = state.whenData(
          (items) => items.map((n) => n.id == updated.id ? updated : n).toList());
    } catch (_) {
      loadNotifications(); // Roll back on server failure.
    }
  }
}
```

Figure A.6. Smart Notification Delivery System

AI Assistant System

This appendix documents the AI assistant subsystem, which provides students with a conversational academic interface supporting intent-driven local data resolution and cloud LLM fallback. The system is composed of six cooperating layers: an intent data model (AiIntent) that captures the parsed semantic meaning of a message; a local rule-based parser (AiIntentParser) that classifies queries client-side before any network call is made; a data resolver (AiDataResolver) that generates structured, deterministic responses for recognised intents directly from Riverpod provider snapshots; a Riverpod provider (AiAssistantNotifier) that orchestrates the full message lifecycle and falls back to the cloud LLM when the intent cannot be resolved locally; a Supabase service and repository pair (AiAssistantService, AiAssistantRepository) that package conversation history and mood context for transmission to the Edge Function; and the acadia-ai Supabase Edge Function (Deno/TypeScript) that performs LLM-based intent detection, structured data fetching, and email delivery via NVIDIA NIM and Resend.

F.1 — Intent Data Model

File: lib/models/ai_intent.dart

Purpose: AiIntent is the structured output of the intent parser. It carries a typed AiIntentType enum value, an AiFilter that captures time range, completion state, and optional course name extracted from the user's message, and the original raw message text for fallback use. Keeping intent and filter as separate value objects allows the data resolver to apply date and completion filters generically across all query types without branching on raw strings.

```dart
enum AiIntentType {
  planWeek, planToday, rankTasks, focusNow,
  getTasksByTimeRange, getCompletedTasks,
  getUpcomingDeadlines, getStudySummary,
  getProgress, getRiskStatus,
  unknown, // Triggers LLM fallback path in the provider.
}

enum TimeRange { today, thisWeek, lastWeek, lastTwoWeeks, custom }

class AiFilter {
  final TimeRange? timeRange;
  final bool? isCompleted;  // tri-state: true=completed only, false=incomplete only, null=all
  final String? course;
  final DateTime? startDate;
  final DateTime? endDate;

  const AiFilter({ this.timeRange, this.isCompleted, this.course,
                   this.startDate, this.endDate });
}

class AiIntent {
  final AiIntentType type;
  final AiFilter filter;
  final String originalMessage; // Preserved so the Edge Function can still use the raw text.

  const AiIntent({ required this.type, required this.filter,
                   required this.originalMessage });
}
```

F.2 — Intent Parser (Client-Side Classification)

File: lib/services/ai/intent_parser.dart

Purpose: AiIntentParser is a purely local, synchronous classifier. It applies a priority-ordered sequence of regex pattern groups against the lowercased message to resolve one of the ten supported intent types, a time range, and an optional course filter — all without a network call. Email delivery requests are detected first and returned as unknown so they always escalate to the Edge Function, which has access to the Resend API. Absolute month names are extracted into a custom date range so queries like "deadlines in April" work correctly.

```
class AiIntentParser {
  AiIntent parse(String message) {
    final lower = message.toLowerCase();

    // Email delivery requests must reach the Edge Function (Resend API is server-only).
    // Return unknown so the provider routes them directly to the LLM fallback path.
    if (_wantsEmailDelivery(lower)) {
      return AiIntent(type: AiIntentType.unknown, filter: const AiFilter(),
                      originalMessage: message);
    }

    // Intent classification — evaluated in priority order.
    // Planning intents are checked before the generic task pattern to avoid
    // "plan today" being captured by the broader r'\btask' match below.
    AiIntentType type = AiIntentType.unknown;
    bool? completionFilter;

    if (_matches(lower, [r'plan my week', r'weekly plan'])) {
      type = AiIntentType.planWeek;
    } else if (_matches(lower, [r'plan today', r'what should i study today'])) {
      type = AiIntentType.planToday;
    } else if (_matches(lower, [r'focus now', r'start session', r'study now'])) {
      type = AiIntentType.focusNow;
    } else if (_matches(lower, [r'what should i do first', r'rank my tasks'])) {
      type = AiIntentType.rankTasks;
    } else if (_matches(lower, [r'\btask', r'\bassignment', r'\bdo\b'])) {
      // Tri-state completion filter: avoids conflating "show tasks" with
      // "show completed tasks" or "show incomplete tasks".
      if (_matches(lower, [r'pending', r'incomplete', r'not done'])) {
        type = AiIntentType.getTasksByTimeRange;
        completionFilter = false; // Incomplete only.
      } else if (_matches(lower, [r'completed', r'done', r'finished'])) {
        type = AiIntentType.getCompletedTasks;
        completionFilter = true;  // Completed only.
      } else {
        type = AiIntentType.getTasksByTimeRange;
        completionFilter = null;  // All tasks.
      }
    } else if (_matches(lower, [r'deadline', r'due', r'upcoming', r'when is'])) {
      type = AiIntentType.getUpcomingDeadlines;
    } else if (_matches(lower, [r'gpa', r'cgpa', r'grade', r'result', r'performance'])) {
      type = AiIntentType.getProgress;
    } else if (_matches(lower, [r'risk', r'fail', r'struggling', r'warning'])) {
      type = AiIntentType.getRiskStatus;
    } else if (_matches(lower, [r'summary', r'recap', r'how am i doing'])) {
      type = AiIntentType.getStudySummary;
    }

    // Absolute month extraction: "deadlines in April" → custom date range April 1–30.
    // Checked before relative ranges so month names are not misclassified as "this week".
    final explicitRange = _extractAbsoluteDateRange(lower);
    TimeRange? timeRange = explicitRange != null ? TimeRange.custom : null;
    DateTime? startDate = explicitRange?['start'];
    DateTime? endDate   = explicitRange?['end'];

    // Relative time range classification.
    if (_matches(lower, [r'\btoday', r'tonight'])) timeRange = TimeRange.today;
    else if (_matches(lower, [r'this week', r'next 7 days'])) timeRange = TimeRange.thisWeek;
    else if (_matches(lower, [r'last week', r'past week'])) timeRange = TimeRange.lastWeek;
    else if (_matches(lower, [r'last 2 weeks', r'last two weeks'])) timeRange = TimeRange.lastTwoWeeks;

    // Course extraction: "tasks for Machine Learning" → course = "machine learning".
    // Suppressed if the captured phrase looks like a date (avoids "for April").
    final courseMatch = RegExp(r'(?:for|in|about)\s+([a-z\d\s]+)$').firstMatch(lower);
    final course = (courseMatch != null && !_looksLikeDatePhrase(courseMatch.group(1)!))
        ? courseMatch.group(1)?.trim()
        : null;

    return AiIntent(
      type: type,
      filter: AiFilter(timeRange: timeRange, isCompleted: completionFilter,
                       course: course, startDate: startDate, endDate: endDate),
      originalMessage: message,
    );
  }
}
```

F.3 — AI Provider (Message Lifecycle Orchestration)

File: lib/providers/ai_assistant_provider.dart

Purpose: AiAssistantNotifier is the central coordinator of the assistant's message lifecycle. It enforces a strict academic scope gate — out-of-scope messages receive an immediate local refusal without a network call. For recognised intents, it resolves responses deterministically from the live Riverpod provider snapshots, guaranteeing that figures in the chat match what the UI displays. Only when the intent remains unknown after local reclassification is a request forwarded to the Edge Function, with risk, burnout, and mood context serialised from the same providers. A character-by-character typing animation is applied to every assistant response.

```dart
class AiAssistantNotifier extends StateNotifier<AiAssistantState> {

  Future<void> sendMessage(String message) async {
    // 1. Scope gate — block non-academic queries without a network call.
    //    Small-talk (hi, who are you) gets a hardcoded local reply.
    final localSmallTalk = _localSmallTalkReply(trimmed, semesterId);
    if (localSmallTalk != null) {
      await _animateTyping(localSmallTalk, semesterId);
      return;
    }

    if (!_isAcademicScopedQuery(trimmed)) {
      // Out-of-scope: respond locally, do not call the Edge Function.
      await _animateTyping(AiMessage(
        text: "I can only answer Acadia academic questions. Ask about your summary, "
              "tasks, deadlines, study sessions, risk, GPA, or progress.",
        isUser: false, semesterId: semesterId,
      ), semesterId);
      return;
    }

    // 2. Parse intent locally.
    var intent = _parser.parse(trimmed);

    // 3. Re-classify unknown academic queries before reaching the LLM.
    //    Email delivery is intentionally excluded — it must go to the Edge Function.
    if (intent.type == AiIntentType.unknown &&
        _isAcademicScopedQuery(trimmed) && !_isEmailDeliveryQuery(trimmed)) {
      intent = AiIntent(
        type: _inferLocalAcademicIntent(trimmed),
        filter: intent.filter, originalMessage: intent.originalMessage,
      );
    }

    AiMessage aiMessage;
    if (intent.type != AiIntentType.unknown) {
      // 4a. LOCAL RESOLUTION — reads from the same Riverpod snapshots as the UI.
      //     This guarantees chat values are identical to Home/Analytics screens.
      final analyticsSnapshot  = _ref.read(analyticsProvider).valueOrNull;
      final riskSnapshot       = _ref.read(riskProvider);
      final adaptiveProfile    = _ref.read(adaptiveProfileProvider);

      aiMessage = await _resolver.resolve(
        intent, user.id, semesterId,
        analyticsSnapshot: analyticsSnapshot,
        riskSnapshot: riskSnapshot.hasData ? riskSnapshot : null,
        adaptiveProfileSnapshot: adaptiveProfile,
        burnoutRiskSnapshot: analyticsSnapshot?.burnoutScore ?? 0.0,
      );
    } else {
      // 4b. LLM FALLBACK — forward to the Edge Function with rich context.
      //     moodContext serialises live risk/stress state so the LLM can
      //     cite accurate figures without querying the database itself.
      final moodContext = {
        'dailyStress': analytics.avgDailyStress,
        'sessionStress': analytics.avgSessionStress,
        'studyDeficit': analytics.studyDeficit,
        'consistency': analytics.consistency,
        'risk': risk.result.totalScore,
        'riskDetail': {
          'score': risk.result.totalScore, 'level': risk.result.riskLevel,
          'overdueTasks': risk.input.overdueTasks,
          'studyDeficit': risk.input.studyDeficit,
          if (risk.trendDelta != null) 'trendDelta': risk.trendDelta,
        },
        'burnout': analytics.burnoutScore,
      };
      aiMessage = await _repository.sendMessage(
        history: updatedMessages, semesterId: semesterId, moodContext: moodContext,
      );
    }

    await _animateTyping(fullAiMessage, semesterId);
  }

  // 15 ms character-by-character typing effect; cancelled immediately on stopResponse().
  Future<void> _animateTyping(AiMessage message, String semesterId) async {
    for (int i = 0; i <= message.text.length; i++) {
      if (_isCancelled) { message.displayedText = message.text; _triggerRebuild(); return;
}     message.displayedText = message.text.substring(0, i);
      _triggerRebuild();
      await Future.delayed(const Duration(milliseconds: 15));
    }
  }
}
```

F.4 — Edge Function Communication (Service and Repository)

File: lib/services/supabase/ai_assistant_service.dart · lib/repositories/ai_assistant_repository.dart

Purpose: AiAssistantService packages the conversation history and context payload for the acadia-ai Edge Function. The history window is capped at the last six messages to keep the LLM context window small and reduce latency. A clientContext object carries the most recent assistant response text and structured data, enabling the Edge Function to handle "send that to my email" without re-querying the database. AiAssistantRepository maps the raw Edge Function response JSON into a typed AiMessage, preserving the structured data, intent, suggestions, and summaryMetadata fields that the UI renders as rich cards.

```dart
class AiAssistantService {
  Future<Map<String, dynamic>> sendMessage(
    List<AiMessage> history,
    String? semesterId, {
    Map<String, dynamic>? moodContext,
  }) async {
    // Cap history at 6 messages — avoids large context windows while retaining
    // enough conversational context for follow-up questions.
    final messages = history.reversed.take(6).toList().reversed.map((m) => {
      'role':    m.isUser ? 'user' : 'assistant',
      'content': m.text,
      'intent':  m.intent,
      'data':    m.data,
    }).toList();

    // clientContext carries the last assistant turn so the Edge Function can
    // reference it for "send this to email" without an additional DB query.
    Map<String, dynamic>? clientContext;
    for (var i = history.length - 2; i >= 0; i--) {
      final m = history[i];
      if (!m.isUser) {
        clientContext = {
          'lastAssistantText': m.text,
          'lastAssistantType': m.type,
          if (m.intent != null) 'lastAssistantIntent': m.intent,
          if (m.data != null)   'lastData': m.data,
        };
        break;
      }
    }

    // 12-second timeout: the NVIDIA NIM endpoint can be slow under load;
    // the provider's catch block shows a graceful fallback message on timeout.
    final response = await _client.functions.invoke(
      'acadia-ai',
      body: {
        'messages': messages, 'semesterId': semesterId,
        if (moodContext != null)    'moodContext': moodContext,
        if (clientContext != null)  'context': clientContext,
      },
    ).timeout(const Duration(seconds: 12));

    if (response.status != 200) throw Exception('Failed to get response from AI');
    return response.data as Map<String, dynamic>;
  }
}

class AiAssistantRepository {
  Future<AiMessage> sendMessage({
    required List<AiMessage> history,
    required String? semesterId,
    Map<String, dynamic>? moodContext,
  }) async {
    final result = await _service.sendMessage(history, semesterId, moodContext:
moodContext);
    // Edge Function JSON fields to the typed AiMessage used throughout the UI.
    return AiMessage(
      text:            result['reply']           as String? ?? '',
      isUser:          false,
      type:            result['type']            as String? ?? 'text',
      data:            result['data'],
      intent:          result['intent']          as String?,
      summaryMetadata: result['summaryMetadata'] as Map<String, dynamic>?,
      suggestions:     (result['suggestions'] as List<dynamic>?)
                           ?.map((e) => e.toString()).toList(),
    );
  }
}
```

F.5 — Edge Function: Intent Routing and LLM Integration

File: supabase/functions/acadia-ai/index.ts

Purpose: The acadia-ai Deno Edge Function is the server-side brain of the assistant. It authenticates each request via the Supabase JWT, then applies a three-stage intent pipeline: a hard keyword router for high-confidence terms (email, summary, focus), an LLM classifier call to the NVIDIA NIM endpoint for ambiguous phrasing, and a regex fallback for cases the LLM cannot confidently resolve. Fetched data is passed back to the LLM for natural-language reply generation, ensuring the model always speaks from real student data rather than hallucinated figures. Email delivery is handled via Resend with per-content-kind HTML templates; the risk kind reads from the moodContext payload rather than re-querying the database.

```
serve(async (req) => {
  // Authenticate via Supabase JWT — the service role key has no RLS bypass here;
  // auth.getUser() validates the client token sent from the Flutter app.
  const { data: { user } } = await supabase.auth.getUser(token);
  if (!user) return new Response("Unauthorized", { status: 401 });

  const { messages, semesterId, context: clientContext, moodContext } = await req.json();
  const lastMessage = messages[messages.length - 1].content;
  const lower = lastMessage.toLowerCase().trim();

  // STAGE 1 — Hard keyword routing for high-confidence intents.
  // Email is checked first so "email my summary" is not swallowed by get_summary.
  let intent: Intent = "ai";
  if (userWantsEmailDelivery(lower))     intent = "email_tasks";
  else if (lower.includes("summary"))    intent = "get_summary";
  else if (lower.includes("focus now")) intent = "get_focus";

  // STAGE 2 — LLM-based intent classification (only when hard routing fails).
  // The system prompt asks the model to return ONE word: tasks/deadlines/summary/etc.
  if (intent === "ai") {
    intent = await aiDetectIntent(client, model, messages);
  }

  // STAGE 3 — Regex fallback (only when LLM also returns "ai").
  if (intent === "ai") {
    intent = resolveIntent(lastMessage, clientContext);
  }

  // --- EXECUTION: structured data fetch → natural-language reply via LLM ---
  switch (intent) {
    case "get_summary": {
      const sum = await computeSummaryData(supabase, user.id, resolvedSemesterId);
      responseData = sum.data;
      reply        = sum.reply; // Pre-built from structured data, not LLM hallucination.
      break;
    }
    case "get_tasks": {
      const tasks = await getTasks(supabase, user.id, resolvedSemesterId);
      // Priority scoring: overdue=4, high priority=4, <24h=3, <72h=2, else=1.
      reply = tasks.all.length === 0
        ? "This semester, you don't have any tasks yet."
        : `This semester, you have ${tasks.all.length} tasks…`;
      break;
    }
    case "email_tasks": {
      // Email kind detection: risk/tasks/summary/deadlines/last_reply/bundle.
      // Risk data is read from moodContext (sent by Flutter provider) — no DB round-trip.
      const kind = detectEmailKind(lower, clientContext);
      const html = kind === "risk"
        ? buildRiskEmail(moodContext.riskDetail, displayName)
        : kind === "last_reply"
          ? buildLastReplyEmail(clientContext.lastAssistantText, displayName)
          : buildTasksEmail(displayName, groups);

      await resend.emails.send({
        from: fromHeader, to: toAddr, subject, html
      });
      reply = emailSentLabel(kind); // "I've sent your risk assessment to your email."
      break;
    }
    default: {
      // Pure conversational fallback — LLM generates the reply with real-data context.
      reply = await aiReply(client, model, messages, contextData);
    }
  }

  return jsonResponse({ reply, type: responseType, data: responseData });
});

// aiReply injects real student metrics into the system prompt so the LLM
// never has to guess — it reads exact figures and labels scope correctly.
async function aiReply(client, model, messages, contextData) {
  const systemPrompt = `You are Acadia, a helpful academic assistant.
STUDENT DATA (always label scope when citing numbers):
- This week (study sessions): ${contextData.studyHours}h (goal: ${contextData.studyGoal}h)
- Study deficit (this week vs goal): ${contextData.studyDeficit}h
- Rolling consistency (last 7 days): ${(contextData.consistency * 100).toFixed(0)}%
- Overdue (this semester): ${contextData.overdue}
Never mix scopes without labeling them. Always use the exact numbers above.`;

  const res = await client.chat.completions.create({
    model,
    messages: [{ role: "system", content: systemPrompt }, ...messages],
  });
  return res.choices[0].message.content;
}
```

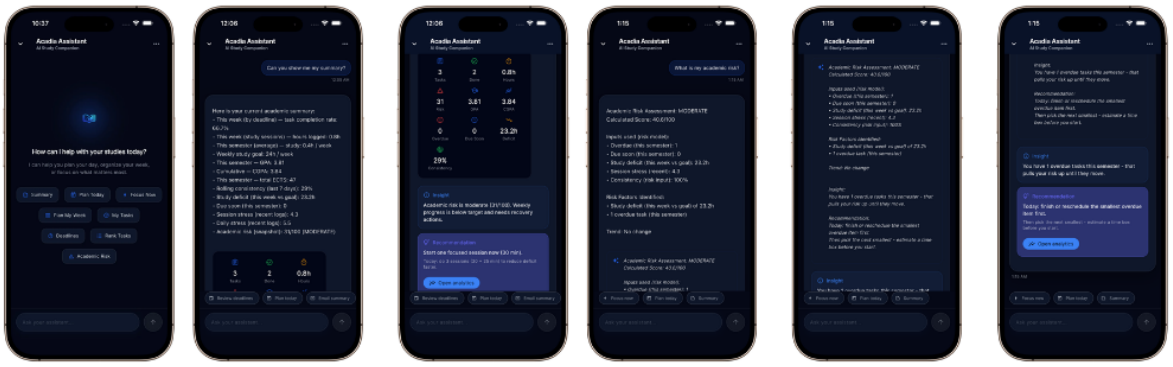
F.6 — Data Resolver (Local Recommendation Generation)

File: lib/services/ai/data_resolver.dart

Purpose: AiDataResolver generates deterministic chat responses for all recognised intents without touching the network. For planning intents (planToday, planWeek, focusNow), it fetches tasks and deadlines from the local repositories, scores each candidate using a priority formula (urgency based on days-remaining + priority label + estimated effort), and distributes sessions across time slots. Session duration is adjusted through DecisionRules.getFinalRecommendedMinutes, which accounts for the student's live stress level, consistency score, and adaptive profile — the same calculation the DecisionEngine uses for its recommendations. Study deficit recovery blocks are injected into daily plans when the weekly deficit exceeds 10 hours.

```
class AiDataResolver {
  Future<AiMessage> resolve(AiIntent intent, String userId, String semesterId, {
    AnalyticsData? analyticsSnapshot,
    RiskState? riskSnapshot,
    AdaptiveProfile? adaptiveProfileSnapshot,
    double? burnoutRiskSnapshot,
  }) async {
    switch (intent.type) {
      case AiIntentType.planToday:
        return _resolvePlanToday(userId, semesterId, analyticsSnapshot,
                                 adaptiveProfileSnapshot ?? const AdaptiveProfile.defaults(),
                                 burnoutRiskSnapshot ?? 0);
      case AiIntentType.focusNow:
        return _resolveFocusNow(userId, semesterId, analyticsSnapshot, riskSnapshot,
                                adaptiveProfileSnapshot ?? const AdaptiveProfile.defaults(),
                                burnoutRiskSnapshot ?? 0);
      case AiIntentType.rankTasks:    return _resolveTaskRanking(userId, semesterId);
      case AiIntentType.getStudySummary: return _resolveStudySummary(...);
      case AiIntentType.getRiskStatus:   return _resolveRiskStatus(semesterId,
riskSnapshot);
    }
  }

  // Priority score = urgency (days-remaining) + overdue bonus + priority label + effort.
  // Caps at 100 — ensures overdue tasks always surface above low-priority future tasks.
  double _taskPriorityScore(Map<String, dynamic> task, DateTime now) {
    double score = 0;
    final daysLeft = task['deadline']?.difference(now).inDays;

    if (daysLeft != null) {
      score += math.max(0.0, 40 - (daysLeft * 3.0)); // Urgency decays as deadline recedes.
    }
    if (task['deadline'] != null && task['deadline'].isBefore(now)) {
      score += 30; // Overdue bonus — always surfaces above non-overdue tasks.
    }
    switch (task['priority']?.toLowerCase()) {
      case 'high':   score += 20; break;
      case 'medium': score += 10; break;
      default:       score +=  5;
    }
    if ((task['estimatedTime'] ?? 0) > 180) score += 10; // Heavy tasks ranked higher.
    return score.clamp(0, 100).toDouble();
  }

  Future<AiMessage> _resolvePlanToday(String userId, String semesterId,
      AnalyticsData? analytics, AdaptiveProfile adaptiveProfile, double burnoutRisk) async {
    // Session duration from DecisionRules — same algorithm as DecisionEngine.
    // Ensures chat recommendations match the behavioural guidance on the Home screen.
    final baseMinutes = DecisionRules.getFinalRecommendedMinutes(
      consistency: analytics?.consistency ?? 1.0,
      stress:      analytics?.avgSessionStress ?? 0.0,
      profile:     adaptiveProfile,
      burnoutRisk: burnoutRisk,
    );

    // Inject a deficit recovery block when weekly hours are severely short.
    if ((analytics?.studyDeficit ?? 0) > 10 && sessions.length < 2) {
      sessions.add({
        'title':    'Deficit recovery block',
        'duration': math.min(baseMinutes, 25),
        'slot':     remainingSlot,
        'source':   'deficit',
        'reasons':  ['High study deficit requires extra recovery today'],
      });
    }

    return AiMessage(
      text: buffer.toString(), isUser: false,
      semesterId: semesterId, type: 'daily_plan',
      data: { 'type': 'daily_plan', 'sessions': sessions },
    );
  }

  // Risk status response reads exclusively from riskProvider snapshot —
  // parity with Home screen is guaranteed by using the same RiskState object.
  AiMessage _resolveRiskStatus(String semesterId, RiskState? risk) {
    final score = risk!.result.totalScore;
    final level = score < 30 ? "LOW" : score < 60 ? "MODERATE" : "HIGH";

    buffer.writeln("Academic Risk Assessment: $level");
    buffer.writeln("Calculated Score: ${score.toStringAsFixed(1)}/100");
    buffer.writeln("• Overdue: ${risk.input.overdueTasks}");
    buffer.writeln("• Study deficit: ${risk.input.studyDeficit.toStringAsFixed(1)}h");
    buffer.writeln("• Session stress: ${risk.input.avgSessionStress.toStringAsFixed(1)}");
    if (risk.trendDelta != null) {
      buffer.writeln("Trend: ${risk.trendDelta! > 0 ? '+${risk.trendDelta} (worsening)'
                                                    : '${risk.trendDelta} (improving)'}");
    }

    return AiMessage(
      text: buffer.toString(), isUser: false,
      semesterId: semesterId, type: 'recommendation',
      data: {
        'score': score, 'level': level,
        'breakdown': {
          'overdue':     risk.result.overdueContribution,
          'deficit':     risk.result.studyContribution,
          'stress':      risk.result.sessionStressContribution,
          'consistency': risk.result.consistencyContribution,
        },
      },
    );
  }
}
```

Figure A.7. AI Assistant Integration Workflow



13.2. Appendix B: Sprint Backlog

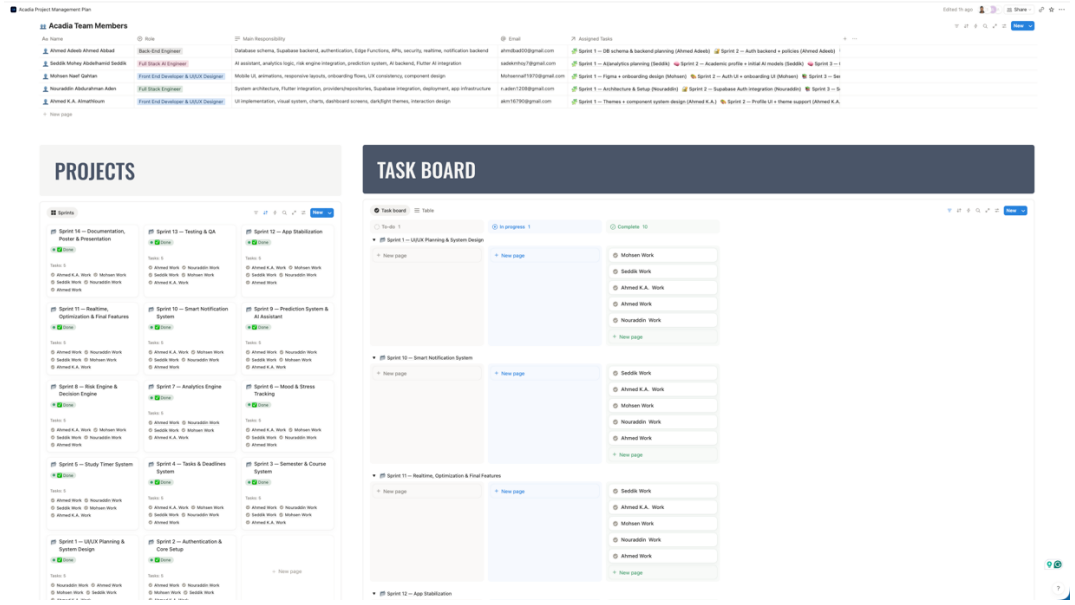
This appendix summarizes the Scrum sprint management process used throughout the Acadia development lifecycle. The appendix should provide evidence of sprint planning, task organization, backlog management, implementation tracking, testing activities, and project coordination.

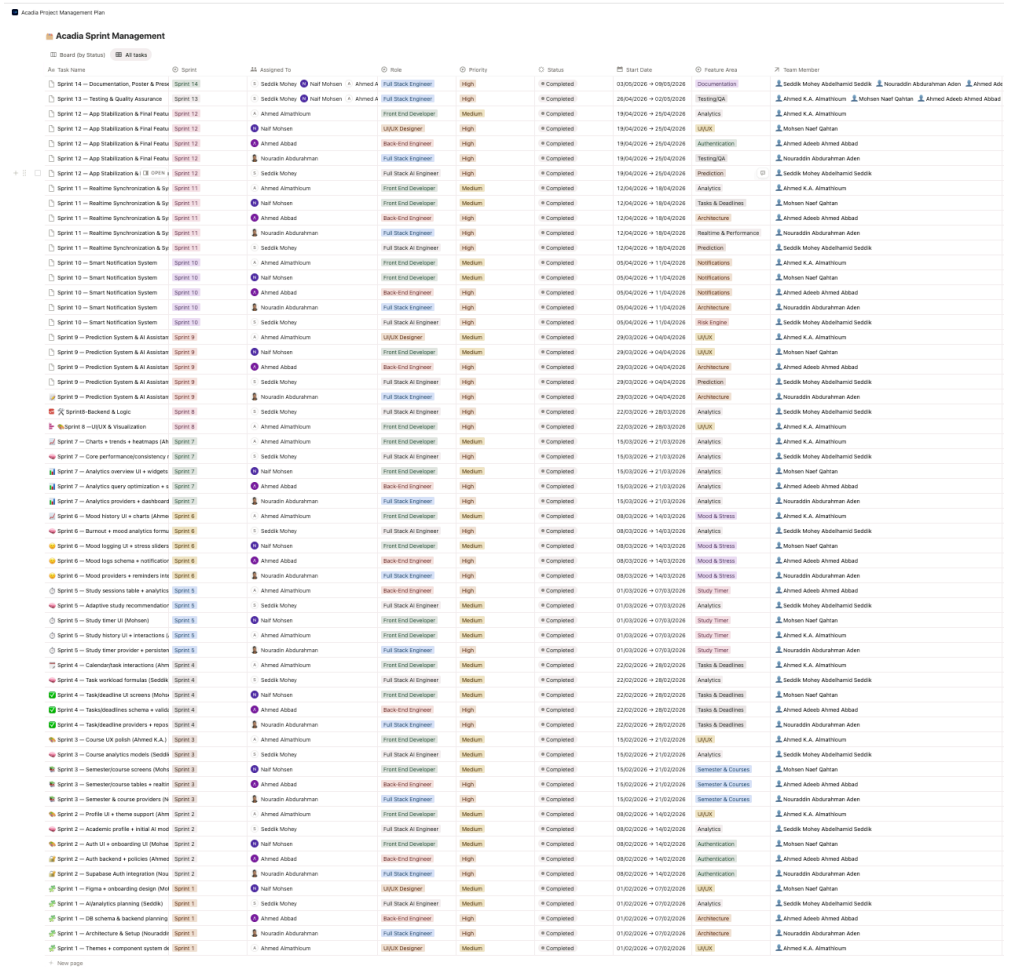
The project followed a structured Agile Scrum workflow consisting of fourteen weekly sprints covering:

- architecture planning
- authentication
- semester management
- task systems
- study sessions
- analytics
- AI integration
- notification systems
- optimization
- testing
- documentation

Sprint Planning

Figure B.1. Acadia Sprint Management Workspace





User Stories and Acceptance Criteria

Include representative examples of:

- User stories
- Acceptance criteria
- Feature priorities
- Sprint deliverables
- Testing objectives

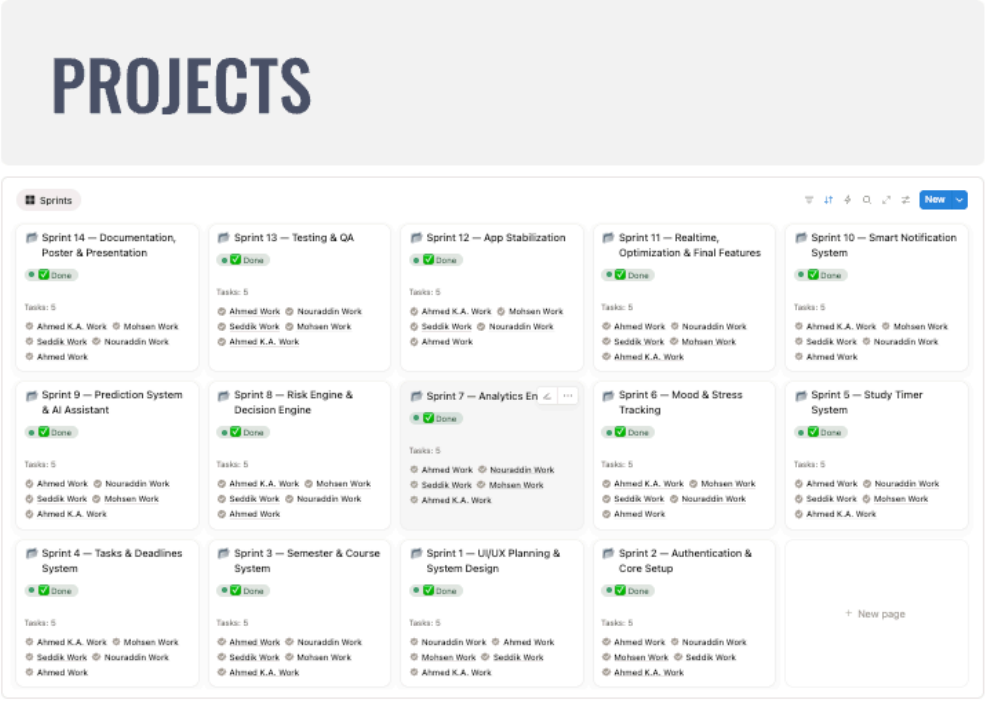
Example User Story Format:

User Story ID	Description	Acceptance Criteria
US-01	Student login system	User can authenticate successfully
US-02	Task creation	Task stored and displayed correctly
US-03	Study session tracking	Session analytics updated after completion

---

Sprint Progress and Completion

Figure B.2. Sprint Completion and Progress Tracking



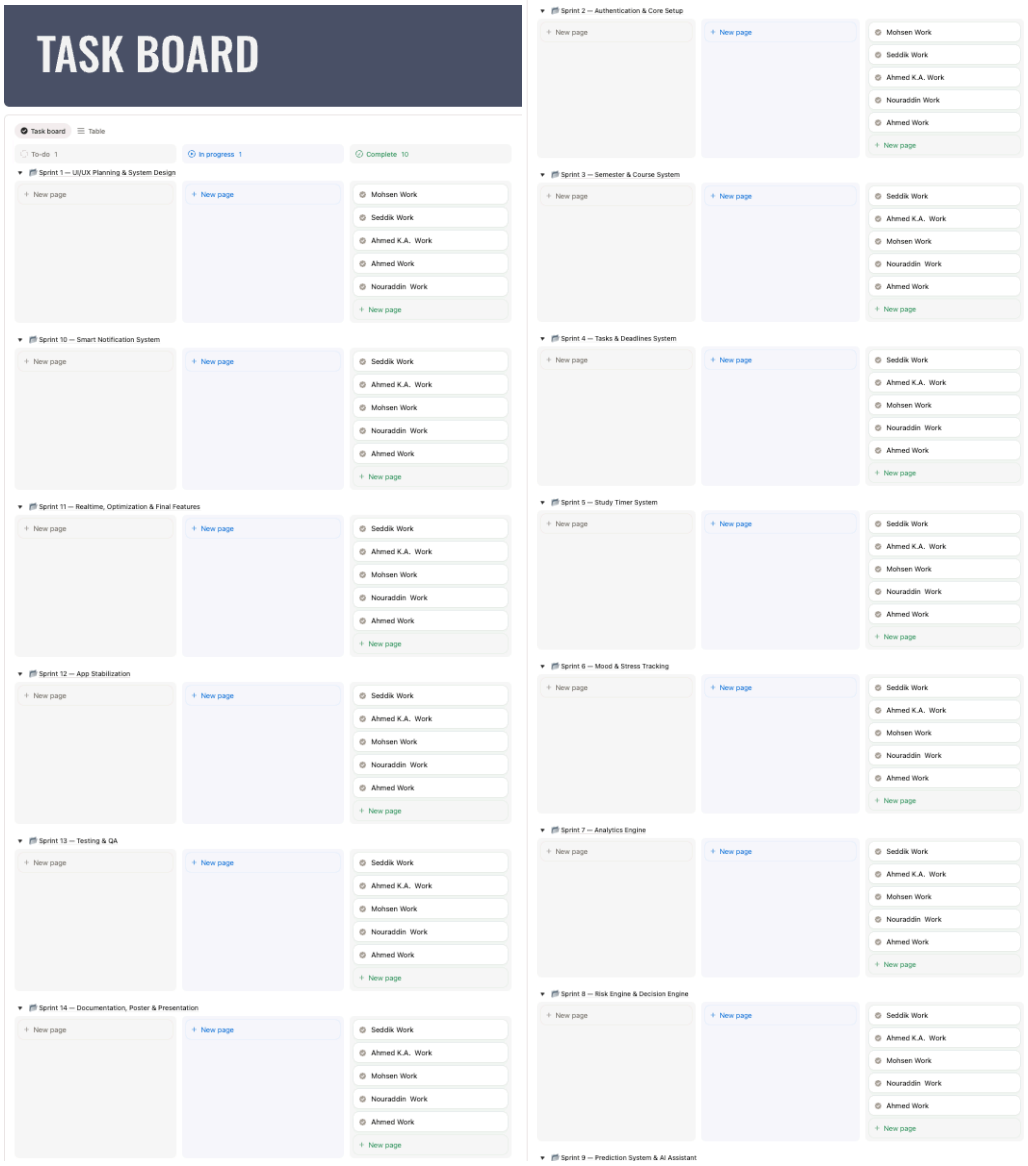
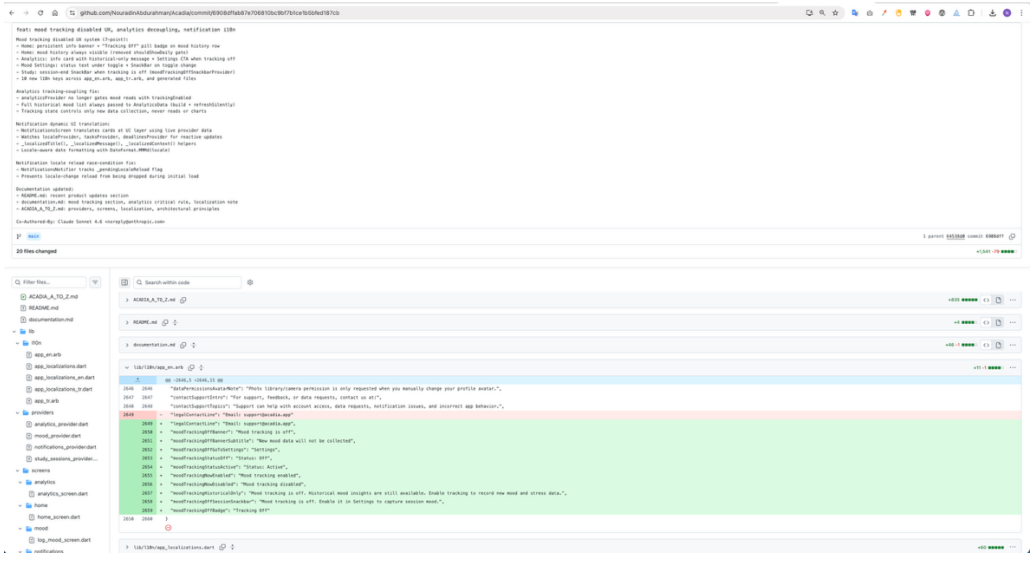


Figure B.3. GitHub commit history



13.3. Appendix C: User Guide

C.1. Account Registration and Login

The authentication module allows students to securely create accounts, access the system, and recover forgotten passwords. The authentication workflow includes language selection, signup, OTP verification, login validation, and password reset functionality using Supabase Authentication services.

Language Selection Screen

The language selection screen appears when the application is launched for the first time. Users can select their preferred application language before continuing to onboarding and authentication workflows.

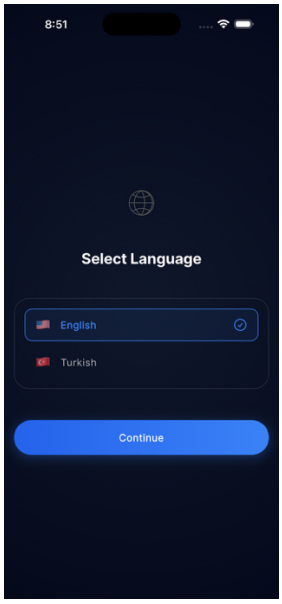


Figure C.1. Language Selection Interface

Login Screen

The login screen allows registered users to authenticate using their email and password. The interface includes remember-me functionality, password recovery, validation handling, and secure authentication using Supabase Auth.

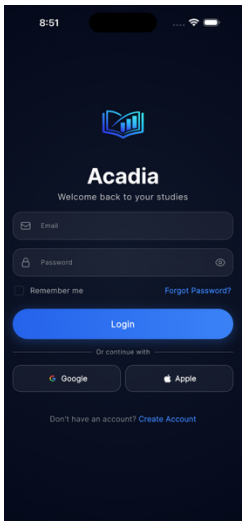


Figure C.2. User Login Interface

Signup Screen

The signup screen allows new students to create accounts using email-based registration. The system validates user credentials and initiates OTP verification after successful registration.

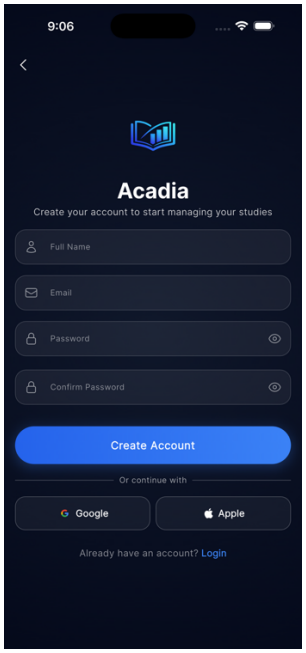


Figure C.3. User Registration Interface

OTP Verification Screen

The OTP verification screen validates the email confirmation code sent during registration or password recovery workflows. Successful verification activates the student account and continues the onboarding process.

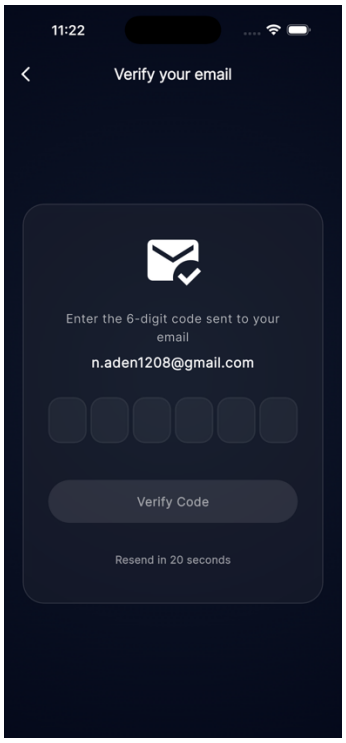


Figure C.4. OTP Verification Workflow

Password Reset Screen

The password reset workflow allows users to securely recover accounts through email verification and OTP validation. Users can create new passwords after successful identity verification.

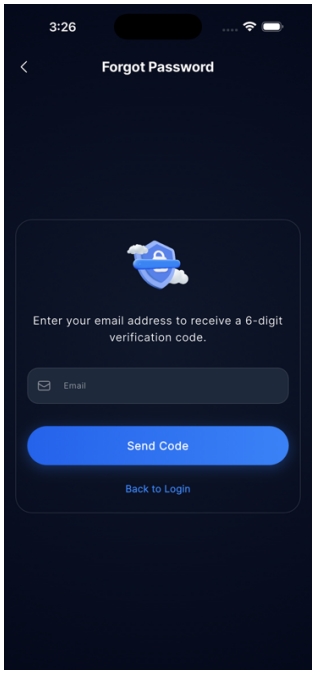


Figure C.5. Password Recovery Interface

C.2. Academic Onboarding

The onboarding system collects academic profile information required for analytics generation, semester organization, and intelligent recommendation workflows.

Academic Profile Setup

The academic profile setup screen allows students to configure university information, degree details, academic year, study preferences, and productivity settings.

Figure C.6. Academic Profile Setup Interface

Semester Configuration

The semester setup screen allows students to create academic semesters, configure semester dates, and define weekly study goals used by the analytics engine.

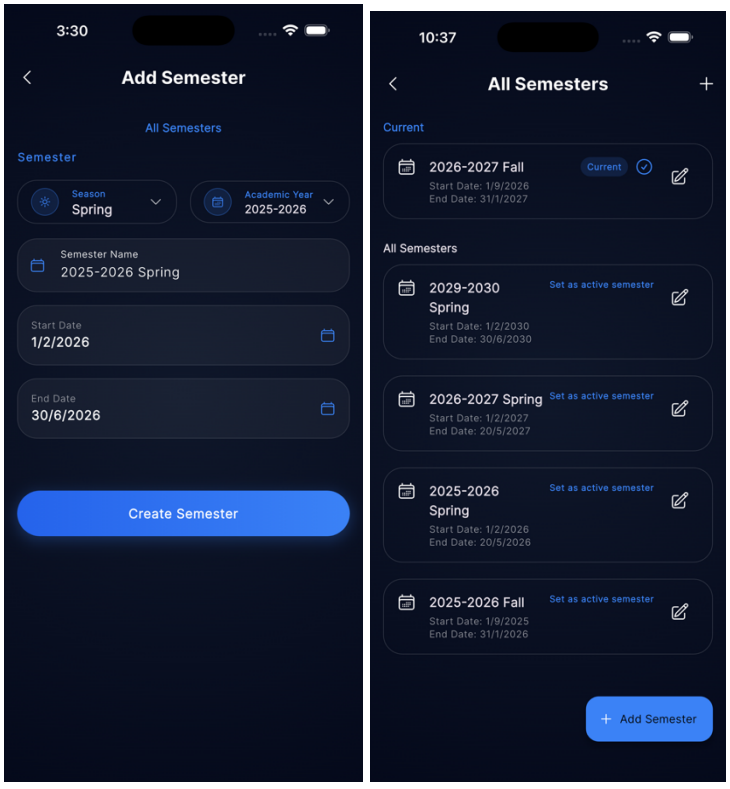


Figure C.7. Semester Configuration Screen

Course Management Setup

The course setup interface allows students to add courses, ECTS values, instructors, and course-related information for semester organization and GPA calculations.

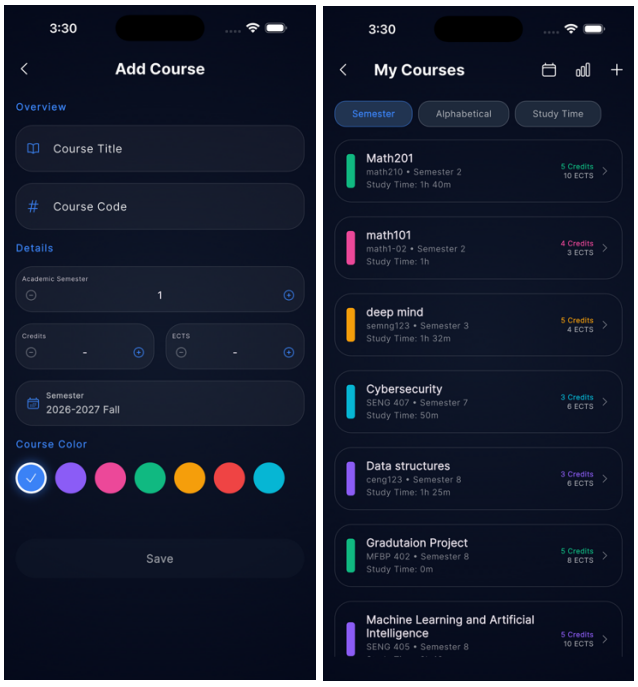


Figure C.8. Course Setup Interface

C.3. Task and Deadline Management

The academic management system allows students to organize assignments, exams, projects, and deadlines within semester-specific workflows.

Task Dashboard

The task dashboard displays active tasks, overdue activities, priorities, and completion progress using realtime synchronization.

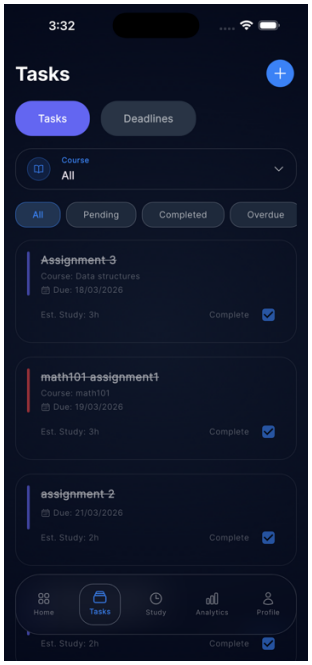


Figure C.9. Academic Task Dashboard

Task Creation Screen

The task creation interface allows students to create academic tasks with deadlines, priorities, estimated durations, and linked courses.

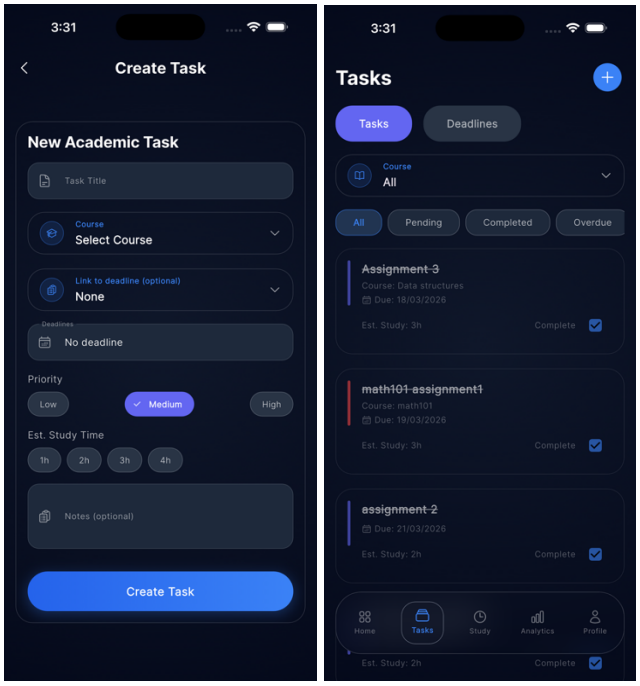


Figure C.10. Task Creation Interface

Deadline Management Screen

The deadline management system allows students to monitor upcoming assignments, exams, quizzes, and projects within semester timelines.

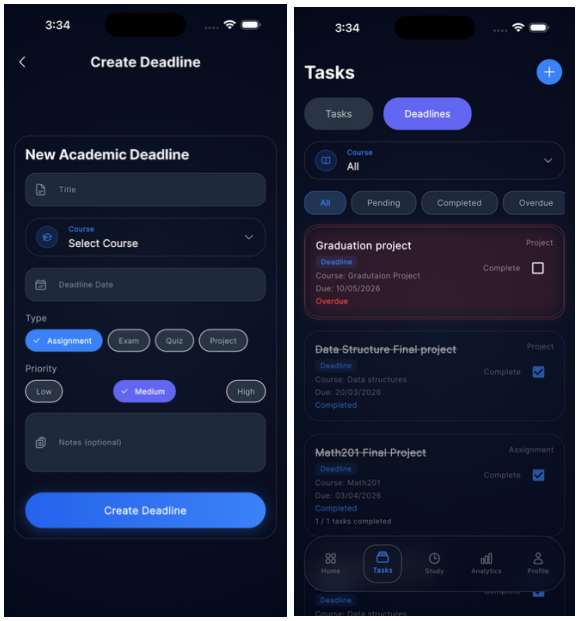


Figure C.11. Deadline Management Interface

C.4. Study Session System

The study session system provides Pomodoro timers, deep work tracking, focus analytics, and productivity monitoring.

Study Timer Screen

The study timer interface allows students to start Pomodoro and deep-work sessions with realtime focus tracking and break management.

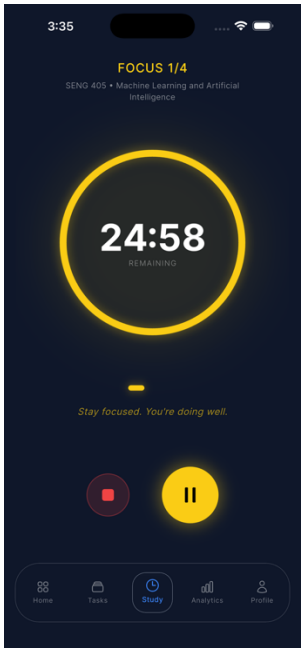


Figure C.12. Pomodoro Study Session Interface

Active Focus Session

The active session screen displays realtime timer progress, completed cycles, focus duration, and break transitions during study sessions.

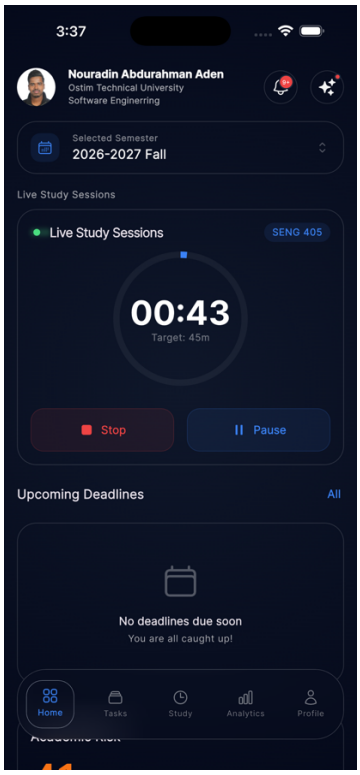


Figure C.13. Active Focus Session Workflow

Study History Screen

The study history dashboard displays completed sessions, productivity statistics, focus duration summaries, and historical study trends.

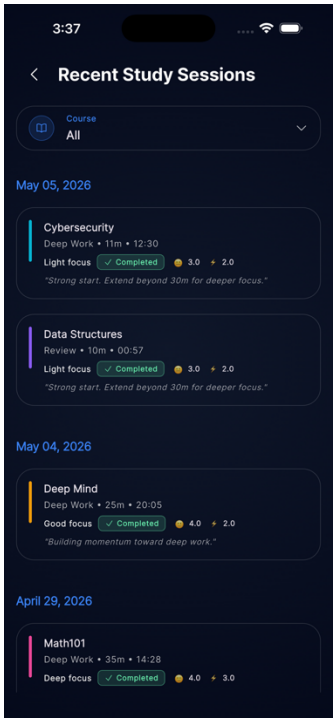


Figure C.14. Study History and Productivity Analytics

C.5. Analytics and Risk Monitoring

The analytics engine visualizes academic performance, productivity metrics, burnout indicators, and academic risk evaluations.

Analytics Dashboard

The analytics dashboard provides visual summaries of GPA trends, study consistency, workload density, and productivity indicators.

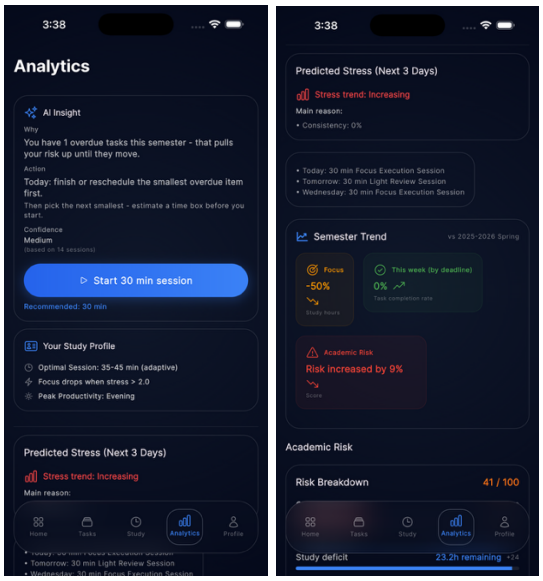


Figure C.15. Academic Analytics Dashboard

Academic Risk Monitoring

The risk monitoring system evaluates missed deadlines, study inconsistency, stress escalation, and workload pressure to generate adaptive academic warnings.

Figure C.16. Academic Risk Monitoring Interface

Burnout and Productivity Charts

The analytics charts visualize emotional trends, burnout indicators, study consistency, and productivity statistics across academic periods.

Figure C.17. Burnout and Productivity Visualization

C.6. Smart Notifications and AI Assistant

The intelligent notification system and AI assistant provide adaptive academic support and realtime productivity recommendations.

Notification Inbox

The notification screen displays deadline reminders, risk alerts, study recommendations, and academic summary notifications generated by the smart notification engine.

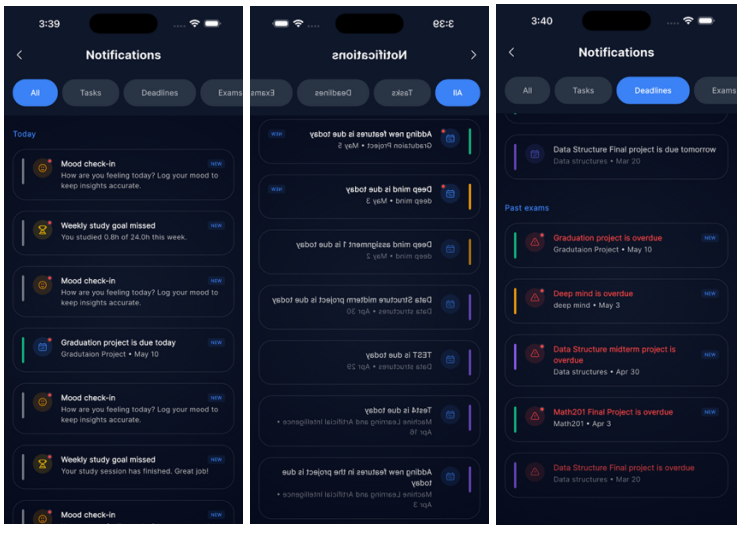


Figure C.18. Smart Notification Interface

Push Notification Example

Push notifications are delivered in realtime using Firebase Cloud Messaging even when the application is inactive.

Figure C.19. Realtime Push Notification Example

AI Assistant Interface

The AI assistant interface allows students to request productivity recommendations, semester summaries, academic explanations, and workload insights.

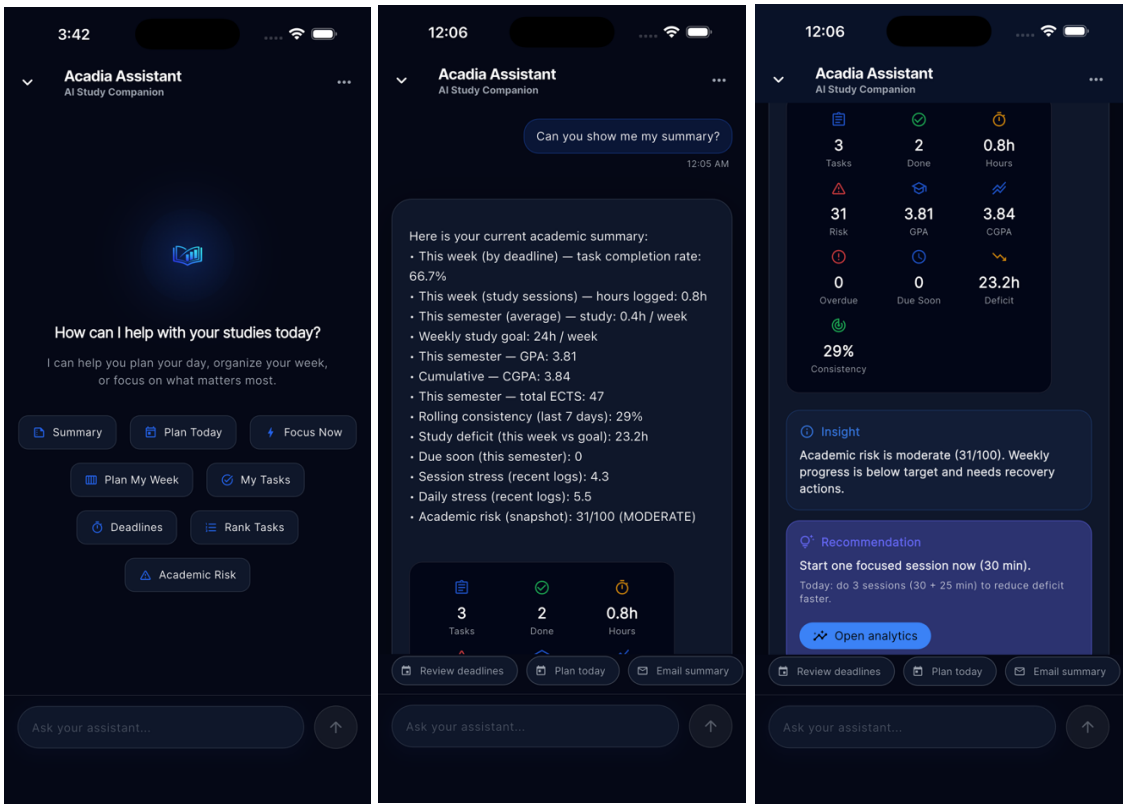


Figure C.20. AI Assistant Interface

AI Recommendation Workflow

The recommendation system generates adaptive study guidance based on analytics, workload density, academic risk, and productivity trends.

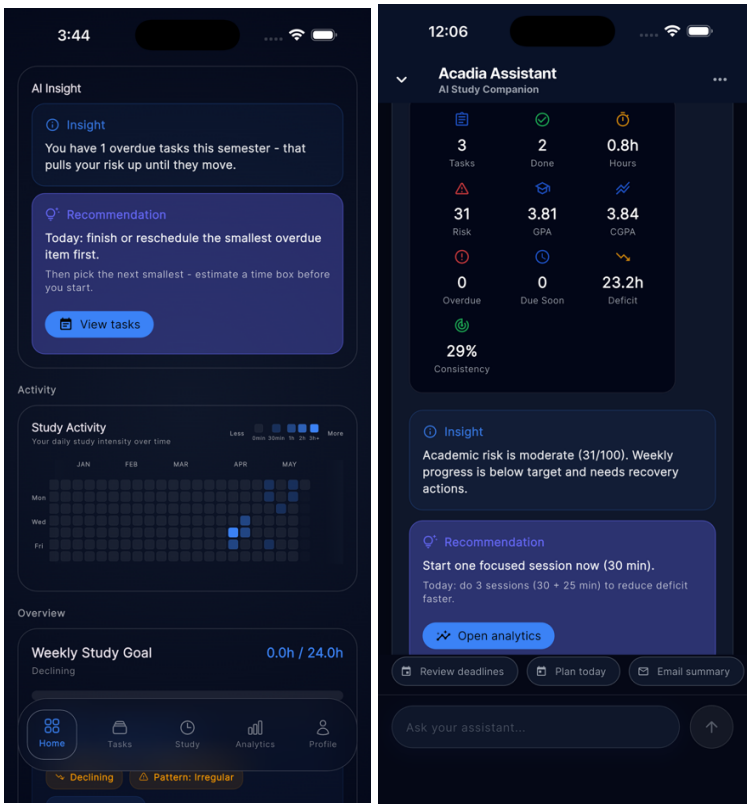


Figure C.21. AI Recommendation System

C.7. Troubleshooting Notes

Problem	Solution
Login issue	Verify internet connection and account credentials
Notifications not received	Enable notification permissions in device settings
Study timer not restoring	Restart the application and reopen the study module
Analytics not updating	Refresh the dashboard or reconnect to the internet
Realtime synchronization delay	Verify stable network connectivity
OTP code expired	Request a new verification code
AI assistant not responding	Verify backend connectivity and internet access